

CASO DE ESTUDIO: EL SISTEMA DE PUNTO DE VENTA NUEVAERA

Pocas cosas son más difíciles de soportar que un buen ejemplo.

Mark Twain

Introducción

Este capítulo describe brevemente el caso de estudio. Si se conoce el dominio del problema, podría saltarse. De hecho, se eligió este problema porque es familiar, pero suficientemente rico, con problemas interesantes de arquitectura y diseño, y, por tanto, nos permite concentrarnos en cómo llevar a cabo el análisis y diseño, en lugar de explicar el problema y el dominio.

3.1. El sistema de punto de venta NuevaEra

El caso de estudio es el sistema de punto de venta (PDV) NuevaEra. En este dominio del problema, en apariencia sencillo, veremos que hay requisitos muy interesantes y problemas de diseño que solucionar. Además, es un problema real; las organizaciones realmente escriben sistemas PDV utilizando la tecnología de objetos.

Un sistema PDV es una aplicación informática utilizada (en parte) para registrar ventas y realizar pagos; normalmente se utiliza en tiendas. Incluye componentes hardware, como un ordenador y un lector de códigos de barras, y software para ejecutar el sistema. Interactúa con varias aplicaciones de servicios, como un servicio de cálculo de impuestos y un control de inventario, de terceras partes. Estos sistemas deben ser, relativamente, tolerantes a fallos; es decir, incluso si los servicios remotos no están disponibles temporalmente (como el sistema de inventario), todavía deben ser capaces de

capturar las ventas y gestionar, al menos, los pagos en efectivo (de manera que no se impida que el negocio funcione de manera adecuada).

Un sistema PDV, progresivamente, debe soportar múltiples y variados terminales e interfaces del lado del cliente, como un terminal con un navegador Web de una arquitectura “cliente delgado” (*thin client*), un ordenador personal normal con una interfaz gráfica de usuario hecha con las clases Swing de Java, entrada de datos mediante una pantalla táctil, PDAs inalámbricos, etcétera.

Además, estamos creando un sistema PDV comercial que se venderá a diferentes clientes con necesidades dispares en términos de procesamiento de reglas del negocio. Cada cliente deseará un conjunto exclusivo de lógica a ejecutar en ciertos puntos predecibles en escenarios de uso del sistema, como cuando se inicia una venta o cuando se añade una nueva línea. Por tanto, necesitaremos un mecanismo para proporcionar esta flexibilidad y personalización.

Utilizando una estrategia de desarrollo iterativo, vamos a realizar las fases de requisitos y análisis, diseño e implementación orientados a objetos.

3.2. Capas arquitectónicas y el énfasis del caso de estudio

Un sistema de información orientado a objetos típico se diseña en función de varias capas arquitectónicas o subsistemas (ver Figura 3.1). A continuación, presentamos algunos ejemplos, no una lista completa:

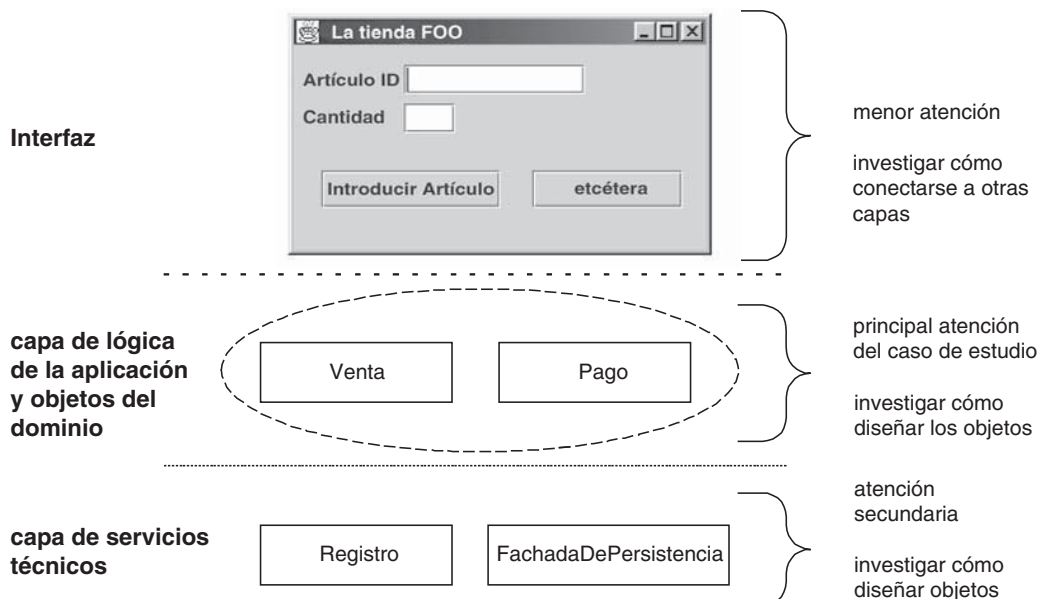


Figura 3.1. Capas y objetos de ejemplo en un sistema orientado a objetos, y el enfoque del caso de estudio.

- **Interfaz de Usuario:** interfaz gráfica; ventanas.
- **Lógica de la aplicación y Objetos del Dominio:** objetos software que representan conceptos del dominio (por ejemplo una clase software denominada *Venta*) que satisfacen los requisitos de la aplicación.
- **Servicios técnicos:** objetos de propósito general y subsistemas que proporcionan servicios técnicos de apoyo, como conexión con una base de datos o registrar los errores. Normalmente, estos servicios son independientes de la aplicación y se pueden reutilizar entre varios sistemas.

El A/DOO es, por lo general, más relevante para modelar la lógica de la aplicación y las capas de servicios técnicos.

El caso de estudio NuevaEra se centra principalmente en los objetos del dominio del problema, asignándoles responsabilidades para satisfacer los requisitos de la aplicación. El diseño orientado objetos también se aplica para crear un subsistema de servicio técnico para interactuar con una base de datos.

En este enfoque de diseño, la capa de interfaz de usuario tiene muy poca responsabilidad; se dice que es *delgada*. Las ventanas *no* contienen código que ejecute o procese la lógica de la aplicación, sino que, las peticiones de realización de tareas se envían a otras capas.

3.3. Estrategia del libro: aprendizaje y desarrollo iterativo

Este libro se organiza para seguir una estrategia de desarrollo iterativo. El A/DOO se aplica al sistema PDV NuevaEra en varias iteraciones; la primera iteración es para determinar algunas funciones básicas. Las iteraciones posteriores expanden la funcionalidad del sistema (ver Figura 3.2). Junto con el desarrollo iterativo, se introduce incrementalmente y de manera iterativa, la *presentación* de las cuestiones del análisis y diseño, la notación UML y los patrones. En la primera iteración, se presenta un núcleo básico de cuestiones de análisis, diseño y notación. La segunda iteración amplía con nuevas ideas, la notación UML y los patrones. Y, de igual modo, la tercera iteración.

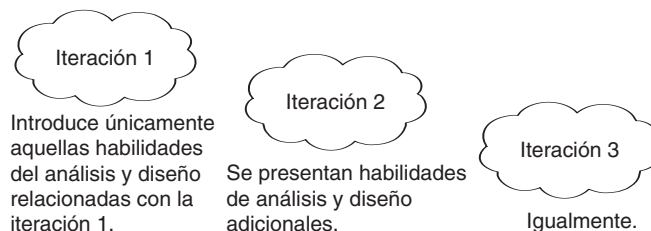


Figura 3.2. El camino de aprendizaje sigue iteraciones.

Parte 2

INICIO

Le mieux est l'ennemi du bien (Lo mejor es enemigo de lo bueno.)

Voltaire

Objetivos

- Definir la etapa de inicio.
- Motivar los capítulos siguientes de esta sección.

Introducción

Este capítulo define la fase de inicio de un proyecto. Si las ideas del proceso no son del interés del lector, o prefiere centrarse en primer lugar en aprender la actividad práctica principal de esta fase —modelado de casos de uso— entonces puede saltarse este capítulo.

La mayoría de los proyectos requieren una etapa inicial breve en la que se estudian los siguientes tipos de preguntas:

- ¿Cuál es la visión y el análisis del negocio para este proyecto?
- ¿Es viable?
- ¿Comprar y/o construir?
- Estimación aproximada del coste: ¿cuesta 10K-100K o millones de euros?
- ¿Deberíamos abordarlo o no seguir?

Para definir la visión y obtener una estimación (de la que no te puedes fiar) del orden de magnitud es necesario llevar a cabo alguna exploración de los requisitos. Sin embargo, el objetivo de la etapa de inicio no es definir todos los requisitos, o generar una estimación creíble o plan de proyecto. Aún a riesgo de simplificar demasiado, la idea es hacer la investigación justa para formar una opinión racional y justificable del propósito global y la viabilidad del nuevo sistema potencial, y decidir si merece la pena invertir en un estudio más profundo (el objetivo de la fase de elaboración).

Por tanto, la fase de inicio debería ser relativamente corta en la mayoría de los proyectos, una duración de una a unas pocas semanas. De hecho, en muchos proyectos, si la duración es superior a una semana, entonces se pierde la idea fundamental de la etapa de inicio: decidir si merece la pena una investigación seria (durante la elaboración), no llevar a cabo esta investigación.

La fase de inicio en una frase:

Vislumbrar el alcance del producto, visión y análisis del negocio.

El principal problema resuelto en una frase:

¿Está de acuerdo el personal involucrado en la visión del proyecto, y merece la pena invertir en un estudio serio?

4.1. Inicio: una analogía

En el negocio del petróleo, cuando se está considerando una nueva zona, algunos de los pasos que hay que abordar son:

1. Decidir si hay evidencias suficientes o un análisis del negocio que justifique perforaciones de exploración.
2. De ser así, llevar a cabo medidas y perforaciones de exploración.
3. Proporcionar información del alcance y estimación.
4. Pasos adicionales...

La fase de inicio es como el primer paso en esta analogía. En el primer paso, no se predice cuánto petróleo hay, o el coste o esfuerzo para extraerlo. Es prematuro —no hay suficiente información—. Aunque estaría bien ser capaz de responder a las preguntas de “cuánto” y “cuándo” sin el coste de la exploración, en el negocio del petróleo se entiende que no es realista.

En términos del UP, el paso de exploración realista se corresponde con la fase de elaboración. La fase de inicio que la precede es parecida a un estudio de viabilidad para decidir si incluso merece la pena invertir en perforaciones de exploración. Sólo después de la exploración (elaboración) tenemos los datos y los conocimientos para hacer, de algún modo, planes y estimaciones creíbles. Por tanto, en el desarrollo iterativo y el UP, los planes y estimaciones de la fase de inicio no deben considerarse fiables. Simplemente proporcionan una percepción del orden de magnitud del grado de esfuerzo, para ayudar en la decisión de continuar o no.

4.2. La fase de inicio podría ser muy breve

El propósito de la fase de inicio es establecer una visión común inicial de los objetivos del proyecto, determinar si es viable y decidir si merece la pena llevar a cabo algunas investigaciones serias en la fase de elaboración. Si se ha decidido de antemano que el proyecto se hará sin ninguna duda, y es claramente viable (quizás porque el equipo ha desarrollado proyectos parecidos antes), entonces la fase de inicio será especialmente breve. Podría incluir los primeros talleres de requisitos¹, planificación de la primera iteración y, entonces, rápidamente, cambiar a la elaboración.

4.3. ¿Qué artefactos podrían crearse en la fase de inicio?

La Tabla 4.1 presenta un listado de los artefactos comunes de la fase de inicio (o principio de la elaboración) e indica las cuestiones que deben abordarse. Los capítulos siguientes estudiarán algunos de ellos con más detalle, especialmente el Modelo de Ca-

Tabla 4.1. Ejemplo de artefactos de la fase de inicio.

<i>Artefacto†</i>	<i>Comentario</i>
Visión y Análisis del Negocio	Describe los objetivos y las restricciones de alto nivel, el análisis del negocio y proporciona un informe para la toma de decisiones.
Modelo de Casos de Uso	Describe los requisitos funcionales y los no funcionales relacionados.
Especificación Complementaria	Describe otros requisitos.
Glosario	Terminología clave del dominio.
Lista de Riesgos & Plan de Gestión del Riesgo darles respuesta.	Describe los riesgos del negocio, técnicos, recursos, planificación, y las ideas para mitigarlos o
Prototipos y pruebas-de-conceptos	Para clarificar la visión y validar las ideas técnicas.
Plan de Iteración	Describe qué hacer en la primera iteración de la elaboración.
Fase Plan de & Plan de Desarrollo de Software	Estimación de poca precisión de la duración y esfuerzo de la fase de elaboración. Herramientas, personas, formación y otros recursos.
Marco de Desarrollo	Una descripción de los pasos del UP y los artefactos adaptados para este proyecto. El UP siempre se debe adaptar al proyecto.

† Estos artefactos se completan sólo parcialmente en esta fase. Se refinarán de manera iterativa en las siguientes iteraciones. El nombre en mayúsculas indica que es un artefacto UP con ese nombre oficial.

¹ *N. del T.*: Se ha traducido “requirements workshop” por “taller de requisitos” para indicar una reunión de discusión sobre requisitos.

sos de Uso. Una idea clave con respecto al desarrollo iterativo es comprender que estos artefactos sólo se completan parcialmente en esta fase, se refinarán en iteraciones posteriores, e incluso no deberían crearse a menos que se considere probable que añadirán valor práctico real. Y, puesto que estamos en el inicio, la investigación y el contenido de los artefactos deberían ser ligeros.

Por ejemplo, el Modelo de Casos de Uso (que se describirá en los capítulos siguientes) podría listar los *nombres* de la mayoría de los casos de uso y actores esperados, pero quizás sólo describiría en detalle el 10% de los casos de uso —hecho con el propósito de desarrollar una visión de alto nivel y sin detalles del alcance, objetivo y riesgos del sistema.

Nótese que en la fase de inicio se podrían realizar algunas tareas de programación con el objeto de crear prototipos de “pruebas de conceptos”, para clarificar unos pocos requisitos mediante (generalmente) prototipos orientados a la interfaz de usuario, y hacer experimentos de programación para cuestiones técnicas críticas.

¿No es eso mucha documentación?

Hay que recordar que los artefactos se deberían considerar opcionales. Se deben elegir sólo aquellos que realmente añadan valor al proyecto, y desecharlos si no se prueba que merezcan la pena.

Lo importante de un artefacto no es el documento o el diagrama en sí mismo, sino el pensamiento, análisis y disposición activa (y entonces se registra, para evitar re-inventaciones o tener que repetir las cosas de palabra). Como dijo el general Eisenhower: “Al preparar una batalla siempre he encontrado que los planes no son útiles, pero planificar indispensable” [Nixon90, BF00].

Hay que almacenar los artefactos digitalmente y on-line —disponibles en el sitio web del proyecto— en lugar de en papel.

Obsérvese también que los artefactos del UP de los proyectos anteriores se pueden utilizar en otros posteriores. Es normal que haya muchas similitudes entre los proyectos en cuanto a los artefactos de riesgos, gestión del proyecto, pruebas y entorno. Todos los proyectos UP organizarán (o deberían organizar) los artefactos de la misma manera, con los mismos nombres (Lista de Riesgos, Marco de Desarrollo, etc.). Esto simplifica la búsqueda de artefactos reutilizables de los proyectos precedentes en las nuevas aplicaciones del UP.

4.4. No se entendió la fase de inicio cuando...

- La duración es mayor de “unas pocas” semanas en la mayoría de los proyectos.
- Se intenta definir la mayoría de los requisitos.
- Se espera que los planes y estimaciones sean fiables.
- Se define la arquitectura; en lugar de hacerlo de manera iterativa en la fase de elaboración.

- Se cree que la secuencia adecuada de trabajo debería ser: 1) definición de los requisitos; 2) diseño de la arquitectura; 3) implementación.
- No hay artefacto de Análisis del Negocio o Visión.
- No se identificaron la mayoría de los nombres de los casos de uso y los actores.
- Se escribieron todos los casos de uso en detalle.
- Ninguno de los casos de uso se escribió en detalle; cuando del 10-20% se deberían escribir con detalle para obtener algún conocimiento realista del alcance del problema.

COMPRENSIÓN DE LOS REQUISITOS

*El nuestro es un mundo donde la gente no sabe lo que quiere
y está deseando atravesar el infierno para conseguirlo.*

Don Marquis

Objetivos

- Definir el modelo FURPS+.
- Relacionar los tipos de requisitos con los artefactos del UP.

Introducción

No todos los requisitos se crean igual. Este capítulo presenta la clasificación de requisitos FURPS+.

Los **requisitos** son capacidades y condiciones con las cuales debe ser conforme el sistema —y más ampliamente, el proyecto [JBR99]—. El primer reto del trabajo de los requisitos es encontrar, comunicar y recordar (que normalmente significa registrar) lo que se necesita realmente, de manera que tenga un significado claro para el cliente y los miembros del equipo de desarrollo.

El UP fomenta un conjunto de buenas prácticas, una de las cuales es la *gestión de requisitos*. Esto no hace referencia a la actitud del ciclo de vida en cascada de definir completamente y estabilizar los requisitos en la primera fase del proyecto, sino más bien —en el contexto de que inevitablemente los deseos del personal involucrado son

cambiantes y poco claros— “un enfoque sistemático para encontrar, documentar, organizar y seguir la pista de los requisitos cambiantes de un sistema” [RUP]; en concreto, haciéndolo con destreza y sin ser descuidado. Fíjese en la palabra *cambiantes*; el UP acepta el cambio en los requisitos como un motor fundamental del proyecto. Otro término importante es *encontrar*; es decir, elicitación cuidadosamente mediante técnicas tales como escritura de casos de uso y talleres de requisitos.

Como se indica en la Figura 5.1, un estudio sobre los costes en proyectos reales en diferentes empresas reveló que el 37% de ellos estaban relacionados con los requisitos, de manera que las cuestiones de requisitos constituyen la principal causa de problemas [Standish94]. En consecuencia, es importante adquirir dominio en la gestión de requisitos. La respuesta del ciclo de vida en cascada a este dato sería intentar con ahínco pulir, estabilizar y fijar los requisitos antes de cualquier diseño o implementación, pero la historia demuestra que es una batalla perdida. La respuesta iterativa es utilizar un proceso que acepte el cambio y la retroalimentación como motores centrales en el descubrimiento de los requisitos.

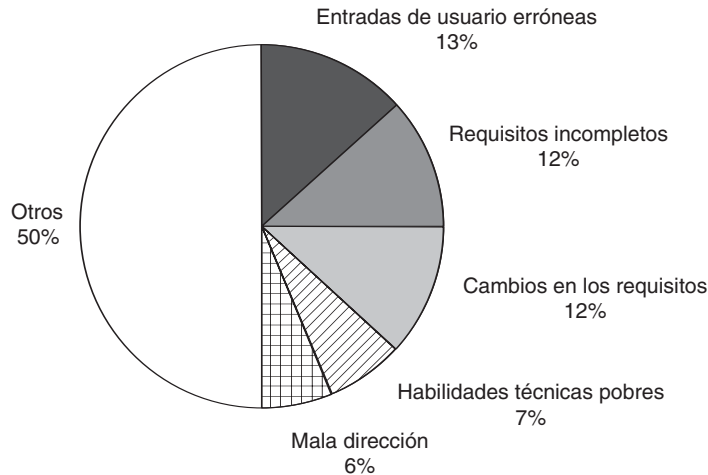


Figura 5.1. Factores del coste en proyectos software reales.

5.1. Tipos de requisitos

En el UP, los requisitos se clasifican de acuerdo con el modelo FURPS+ [Grady92], un útil nemotécnico que significa los siguientes cinco tipos de requisitos¹:

- **Funcional (*Functional*)**: características, capacidades y seguridad.
- **Facilidad de uso (*Usability*)**: factores humanos, ayuda, documentación.
- **Fiabilidad (*Reliability*)**: frecuencia de fallos, capacidad de recuperación de un fallo y grado de previsión.

¹ Hay varios sistemas de clasificación de requisitos y atributos de calidad publicados en libros y por organizaciones estándares, como el ISO 9126 (que es similar a la lista del FURPS+), y varias del Instituto de Ingeniería del Software (SEI, *Software Engineering Institute*); cualquiera de ellas se puede utilizar en un proyecto UP.

- **Rendimiento (*Performance*):** tiempos de respuesta, productividad, precisión, disponibilidad, uso de los recursos.
- **Soporte (*Supportability*):** adaptabilidad, facilidad de mantenimiento, internacionalización, configurabilidad.

El + en FURPS+ indica requisitos adicionales, tales como:

- **Implementación:** limitación de recursos, lenguajes y herramientas, hardware.
- **Interfaz:** restricciones impuestas para la interacción con sistemas externos.
- **Operaciones:** gestión del sistema en su puesta en marcha.
- **Empaquetamiento**
- **Legales:** licencias, etcétera.

Resulta útil utilizar las categorías del FURPS+ (o algún esquema de clasificación) como una lista para comprobar que se cubren los requisitos, de manera que reducimos el riesgo de no considerar alguna faceta importante del sistema.

Algunos de estos requisitos se denominan colectivamente **atributos de calidad, requisitos de calidad**, o las “-ilities”² de un sistema. Estos comprenden facilidad de uso (*usability*), fiabilidad (*reliability*), rendimiento (*performance*) y soporte (*supportability*). Lo normal es dividir los requisitos en **funcionales** (comportamiento) y **no funcionales** (todo lo demás); a algunos no les gusta esta amplia generalización [BCK98], pero se utiliza de manera muy extendida.

Los requisitos funcionales se estudian y recogen en el Modelo de Casos de Uso, el tema del siguiente capítulo, y en la lista de características del sistema del artefacto Visión. Los otros requisitos se pueden recoger en los casos de usos con los que están relacionados, o en el artefacto Especificación Complementaria. El artefacto Visión resume los requisitos de alto nivel que se elaboran en estos otros documentos. El Glosario agrupa y clarifica los términos que se utilizan en los requisitos. El Glosario en el UP también comprende el concepto de **diccionario de datos**, que reúne los requisitos relacionados con los datos, como reglas de validación, valores aceptables, etcétera. Los prototipos son un mecanismo para clarificar qué es lo que se quiere o es posible.

Como veremos cuando estudiemos el análisis arquitectural, los requisitos de calidad influyen fuertemente en la arquitectura de un sistema. Por ejemplo, un requisito de alto rendimiento y alta fiabilidad influirá en la elección de componentes software y hardware, y en sus configuraciones. La necesidad de fácil adaptación, debido a cambios frecuentes en los requisitos funcionales, igualmente dará forma al diseño del software.

5.2. Lecturas adicionales

En los capítulos siguientes se cubrirán referencias relacionadas con casos de uso. Se recomiendan como punto de partida para el estudio de los requisitos, textos sobre requisitos orientados a los casos de uso, como *Writing Effective Use Cases* [Cockburn01], en lugar de textos más generales (y normalmente, tradicionales).

² N. del T.: Terminación plural en inglés de los nombres de tales requisitos.

Hay un amplio debate sobre requisitos —y una extensa variedad de temas de ingeniería del software— bajo el paraguas de Cuerpo de Conocimientos en Ingeniería del Software (**SWEBOK**, *Software Engineering Body of Knowledge*) disponible en www.swebok.org.

El SEI (www.sei.cmu.edu) tiene varias propuestas relacionadas con los requisitos de calidad. El ISO 9126, el IEEE Std. 830, y el IEEE Std. 1061 son estándares relacionados con los requisitos y atributos de calidad, y están disponibles en la web en varios sitios.

Algunas advertencias con respecto a los libros generales sobre requisitos, incluso aquellos que proponen cubrir los casos de uso, desarrollo iterativo o, de hecho, incluso los requisitos en el UP:

1. La mayoría están escritos bajo la influencia de un ciclo de vida en cascada planteando la definición “precisa” de los requisitos por adelantado, antes de pasar al diseño y la implementación. Esto no pretende invalidar su valor más amplio o, a menudo, profundas y útiles visiones de los requisitos independientes del método, sino que pretende aclarar que no constituyen una visión fiable del desarrollo iterativo. Esto es debido a que la experiencia de los autores proviene principalmente de proyectos de ciclo de vida en cascada, trabajando para refinar, definir cuidadosamente y precisamente los requisitos, y terminar con la fase de requisitos antes de continuar con el diseño. Aquellos libros que también mencionan el desarrollo iterativo, lo abordan muy superficialmente, quizás añadiendo material “iterativo” apelando a las tendencias modernas. Por tanto, los libros y artículos de requisitos deberían leerse con cautela; uno podría sentirse seguro con la idea de intentar definir de manera cuidadosa todos los requisitos en la fase inicial, que no es consistente con un proceso iterativo.
2. Muchos libros generales sobre requisitos que también dicen incluir casos de uso, lo hacen muy superficialmente, o entienden mal el significado real de requisitos dirigidos por casos de uso. Esto podría deberse a que los autores poseen una experiencia de muchos años en métodos de requisitos tradicionales, y recientemente, han intentado incluir los casos de uso en sus métodos anteriores, sin darse cuenta de que la idea principal de los casos de uso, tal y como la concibe Ivar Jacobson y el UP, es hacer los casos de uso el elemento central del enfoque de requisitos global —sustituyendo a otros documentos de requisitos como elemento central—; los casos de uso impregnan y dirigen el trabajo de requisitos, en lugar de considerarse como una técnica de apoyo auxiliar a nivel inferior o medio, añadida a los documentos o enfoques de requisitos tradicionales.

En resumen, los libros generales sobre requisitos ofrecen consejos útiles sobre técnicas y cuestiones para recopilar los requisitos, escritos por personas expertas, pero normalmente, los presentan en el contexto de un proceso en cascada, sin tener un gran conocimiento de las implicaciones más profundas de los casos de uso. Cualquier consejo sobre el proceso que sea una variante de: “trate de definir la mayoría de los requisitos, y entonces pase al diseño y la implementación” no es consistente con el desarrollo iterativo y el UP.

MODELO DE CASOS DE USO: ESCRITURA DE REQUISITOS EN CONTEXTO

*El primer paso indispensable para conseguir
las cosas que quieres de la vida: decidir qué quieres.*

Ben Stein

Objetivos

- Identificar y escribir casos de uso.
- Relacionar los casos de uso con los objetivos de los usuarios y los procesos de negocio básicos.
- Utilizar los formatos breve, informal y completo, en un estilo esencial.
- Relacionar el trabajo con casos de uso con el desarrollo iterativo.

Introducción

Merece la pena estudiar este capítulo durante la primera lectura del libro porque los casos de uso son un mecanismo ampliamente utilizado para descubrir y registrar los requisitos (especialmente los funcionales); influyen muchos aspectos de un proyecto, incluyendo el A/DOO. Merece la pena tanto saber sobre los casos de uso como crearlos.

La escritura de casos de uso —historias del uso de un sistema— es una técnica excelente para entender y describir los requisitos. Este capítulo explora los conceptos claves de los casos de uso y presenta casos de uso de ejemplo para la aplicación NuevaEra.

El UP define el **Modelo de Casos de Uso** en la disciplina Requisitos. Básicamente, es el conjunto de todos los casos de uso; es un modelo de la funcionalidad y entorno del sistema.

6.1. Objetivos e historias

Los clientes y los usuarios finales tienen objetivos (también conocidos como *necesidades*) y quieren sistemas informáticos que les ayuden a conseguirlos, que varían desde registrar las ventas hasta estimar el flujo de petróleo de futuros pozos. Hay varias formas de capturar estos objetivos y requisitos del sistema; las mejores son simples y familiares, porque esto hace que sea más fácil —especialmente para clientes y usuarios finales— contribuir a su definición o evaluación. Eso reduce el riesgo de perder el hilo.

Los casos de uso son un mecanismo para ayudar a mantenerlo simple y entendible para todo el personal involucrado. De manera informal, son historias del uso de un sistema para alcanzar los objetivos. A continuación presentamos un ejemplo de caso de uso en *formato breve*:

Procesar Venta: Un cliente llega a una caja con artículos para comprar. El cajero utiliza el sistema PDV para registrar cada artículo comprado. El sistema presenta una suma parcial y detalles de cada línea de venta. El cliente introduce los datos del pago, que el sistema valida y registra. El sistema actualiza el inventario. El cliente recibe un recibo del sistema y luego se va con los artículos.

A menudo, los casos de uso necesitan una elaboración mayor que ésta, pero la esencia es descubrir y registrar los requisitos funcionales, mediante la escritura de historias del uso de un sistema, para ayudar a cumplir los objetivos de varias de las personas involucradas; esto es, los *casos de uso*¹. Se supone que no es una idea difícil, aunque, de hecho podría ser difícil descubrir o decidir lo que es necesario, y escribirlo de manera coherente con un nivel de detalle útil.

Se ha escrito mucho acerca de los casos de uso, y si bien es útil, existe el riesgo entre las personas inteligentes y creativas, de oscurecer una idea sencilla con niveles de sofisticación. Normalmente es posible distinguir a un modelador de casos de uso novato (o a un analista serio de Tipo A) preocupándose en exceso con cuestiones secundarias como diagramas de casos de uso, relaciones de casos de uso, paquetes de casos de uso, atributos opcionales, etcétera, en lugar de escribir las historias. En otras palabras, el poder del mecanismo de casos de uso es la capacidad tanto de aumentar como de disminuir, en términos de sofisticación y formalidad, dependiendo de la necesidad.

6.2. Antecedentes

La idea de utilizar los casos de uso para describir los requisitos funcionales fue introducida en 1986 por Ivar Jacobson [Jacobson92], uno de los contribuidores principales al UML y UP. La idea de caso de uso de Jacobson ha tenido una gran influencia y ha sido ampliamente reconocida; siendo sus principales virtudes la simplicidad y utilidad. Aunque muchos han contribuido en este campo, se puede sostener que el siguiente paso más coherente, comprensible e influyente en la definición de qué son (o deberían ser) los casos de uso y cómo escribirlos, procede de Alistair Cockburn, resumido en su popular texto *Writing Effective Use Cases* [Cockburn01], basado en sus primeros trabajos y escritos publicados de 1992 en adelante. Esta introducción, por tanto, se basa y es consistente con este último trabajo.

¹ El término original en sueco se traduce literalmente como “caso de costumbre”.

6.3. Casos de uso y valor añadido

En primer lugar, algunas definiciones informales: un **actor** es algo con comportamiento, como una persona (identificada por un rol), sistema informatizado u organización; por ejemplo, un cajero.

Un **escenario** es una secuencia específica de acciones e interacciones entre los actores y el sistema objeto de estudio; también se denomina **instancia de caso de uso**. Es una historia particular del uso de un sistema, o un camino a través del caso de uso; por ejemplo, el escenario de éxito de compra de artículos con pago en efectivo, o el escenario de fallo al comprar debido al rechazo de la transacción de pago con la tarjeta de crédito.

Informalmente entonces, un **caso de uso** es una colección de escenarios con éxito y fallo relacionados, que describe a los actores utilizando un sistema para satisfacer un objetivo. Por ejemplo, a continuación presentamos un caso de uso en *formato informal* que incluye algunos escenarios alternativos:

Gestionar Devoluciones

Escenario principal de éxito: Un cliente llega a una caja con artículos para devolver. El cajero utiliza el sistema PDV para registrar cada uno de los artículos devueltos...

Escenarios alternativos:

Si se pagó con tarjeta de crédito, y se rechaza la transacción de reembolso a su cuenta, informar al cliente y pagarle en efectivo.

Si el identificador del artículo no se encuentra en el sistema, notificar al Cajero y sugerir la entrada manual del código de identificación (quizás esté alterado).

Si el sistema detecta fallos en la comunicación con el sistema de contabilidad externo...

El RUP proporciona una definición alternativa, aunque similar, de un caso de uso:

Un conjunto de instancias de caso de uso, donde cada instancia es una secuencia de acciones que un sistema ejecuta, produciendo un resultado observable de valor para un actor particular [RUP].

La expresión “*un resultado observable de valor*” es sutil pero importante, porque destaca el hecho de que el comportamiento del sistema debería preocuparse de proporcionar valor al usuario.

Una actitud clave en el trabajo con casos de uso es centrarse en la pregunta: “¿Cómo puedo, utilizando el sistema, proporcionar un valor observable al usuario, o cumplir sus objetivos?” en lugar de, simplemente, pensar en los requisitos del sistema en términos de una “lista de la lavandería” de características o funciones.

Quizás parece obvio destacar que se proporcione un valor observable para el usuario, pero la industria de software está plagada de proyectos fracasados que no proporciona-

ron lo que la gente realmente necesitaba. El enfoque de la lista de características y funciones para capturar los requisitos, puede contribuir a este resultado negativo, puesto que no fomenta que el personal involucrado considere los requisitos en un contexto amplio de uso del sistema, en un escenario para alcanzar algún resultado observable de valor, o algún objetivo. Por el contrario, los casos de uso sitúan las características y funciones en un contexto orientado al objetivo. De ahí el título del capítulo².

Ésta es la idea clave que Jacobson intentaba transmitir con el concepto de caso de uso: trabaja con los requisitos centrándote en cómo puede un sistema añadir valor y cumplir los objetivos.

6.4. Casos de uso y requisitos funcionales

Los casos de uso son requisitos; ante todo son requisitos funcionales que indican qué hará el sistema. En términos de los tipos de requisitos FURPS+, los casos de uso se refieren fundamentalmente a la “F” (funcional o de comportamiento), pero también pueden utilizarse para otros tipos, especialmente cuando esos otros tipos están estrechamente relacionados con un caso de uso. En el UP —y métodos más modernos— los casos de uso son el mecanismo principal que se recomienda para su descubrimiento y definición. Los casos de uso definen una promesa o contrato de la manera en la que se comportará un sistema.

Para ser claros: los casos de uso *son* requisitos (aunque no todos los requisitos). Algunos piensan en requisitos sólo como listas de características y funciones de la forma “el sistema deberá hacer...”. No es así, y una idea clave de los casos de uso es (por lo general) reducir la importancia o el uso de listas de características detalladas al estilo antiguo y más bien, escribir casos de uso para los requisitos funcionales. Veremos más sobre este punto en una sección posterior.

Los casos de uso son documentos de texto, no diagramas, y el modelado de casos de uso es, sobre todo, una acción de escribir texto, no dibujar. Sin embargo, UML define un diagrama de casos de uso para ilustrar los nombres de casos de uso y actores, y sus relaciones.

6.5. Tipos de casos de uso y formatos

Casos de uso de caja negra y las responsabilidades del sistema

Los casos de uso de caja negra son la clase más común y recomendada; no describen el funcionamiento interno del sistema, sus componentes o diseño, sino que se describe el sistema en base a las *responsabilidades* que tiene, que es una metáfora común y unificadora en el pensamiento orientado a objetos —los elementos software tiene responsabilidades y colaboran con otros elementos que tienen responsabilidades—.

² Procede del libro titulado muy apropiadamente *Uses Cases: Requirements in Context* [GK00] (el título del capítulo se adaptó con permiso de los autores).

A través de la definición de las responsabilidades del sistema con casos de uso de caja negra, es posible especificar *qué* debe hacer el sistema (los requisitos funcionales) sin decidir *cómo* lo hará (el diseño). De hecho, la definición de “análisis” frente al “diseño” se resume algunas veces como el “qué” frente al “cómo”. Éste es un tema importante en un buen desarrollo de software: evite durante el análisis de requisitos tomar decisiones acerca del “cómo”, y especifique el comportamiento externo del sistema, como una caja negra. Después, durante el diseño, cree una solución que satisfaga la especificación.

<i>Estilo de caja negra</i>	<i>No</i>
El sistema registra la venta	El sistema escribe la venta en una base de datos... o (incluso peor). El sistema genera una sentencia SQL INSERT para la venta...

Tipos de formalidad

Los casos de uso se escriben con formatos diferentes, dependiendo de la necesidad. Además del tipo de *visibilidad*, de caja negra frente a caja blanca, los casos de uso se escriben con varios grados de *formalidad*:

- **Breve:** resumen conciso de un párrafo, normalmente del escenario principal con éxito. El anterior ejemplo de *Procesar Venta* era breve.
- **Informal:** formato de párrafo en un estilo informal. Múltiples párrafos que comprenden varios escenarios. El anterior ejemplo de *Gestionar Devoluciones* era informal.
- **Completo:** el más elaborado. Se escriben con detalle todos los pasos y variaciones, y hay secciones de apoyo como precondiciones y garantías de éxito.

El ejemplo siguiente es un caso de uso en formato completo del caso de estudio NuevaEra.

6.6. Ejemplo completo: Procesar Venta

Los casos de uso completos muestran más detalles y están estructurados; son útiles para entender en profundidad los objetivos, tareas y requisitos. En el caso de estudio del PDV NuevaEra, se crearían en uno de los primeros talleres de requisitos con la colaboración del analista del sistema, expertos en la materia de estudio y los desarrolladores.

El formato usecases.org

Hay varias plantillas disponibles para los casos de uso completos. Sin embargo, quizás el formato más ampliamente extendido y compartido es la plantilla disponible en www.usecases.org. El siguiente ejemplo muestra este estilo.

El lector debe darse cuenta que éste es el principal ejemplo de caso de uso detallado para el caso de estudio del libro; muestra muchos elementos y cuestiones comunes.

Caso de uso UC1: Procesar Venta

Actor principal: Cajero.

Personal involucrado e intereses:

- Cajero: quiere entradas precisas, rápidas, y sin errores de pago, ya que las pérdidas se deducen de su salario.
- Vendedor: quiere que las comisiones de las ventas estén actualizadas.
- Compañía: Quiere registrar las transacciones con precisión y satisfacer los intereses de los clientes. Quiere asegurar que se registran los pagos aceptados por el Servicio de Autorización de Pagos. Quiere cierta tolerancia a fallos que permita capturar las ventas incluso si los componentes del servidor (ej. validación remota de crédito) no están disponibles. Quiere actualización automática y rápida de la contabilidad y el inventario.
- Agencia Tributaria del Gobierno: quiere recopilar los impuestos de cada venta. Podrían ser múltiples agencias: nacional, provincial y local.
- Servicio de Autorización de Pagos: Quiere recibir peticiones de autorización digital con el formato y protocolo correctos. Quiere registrar de manera precisa las cuentas por cobrar de la tienda.

Precondiciones: El cajero se identifica y autentica.

Garantías de éxito (Postcondiciones): Se registra la venta. El impuesto se calcula de manera correcta. Se actualizan la contabilidad y el inventario. Se registran las comisiones. Se genera el recibo. Se registran las autorizaciones de pago aprobadas.

Escenario principal de éxito (o Flujo Básico):

1. El Cliente llega a un terminal PDV con mercancías y/o servicios que comprar.
 2. El Cajero comienza una nueva venta.
 3. El Cajero introduce el identificador del artículo.
 4. El Sistema registra la línea de la venta y presenta la descripción del artículo, precio y suma parcial. El precio se calcula a partir de un conjunto de reglas de precios.
- El Cajero repite los pasos 3-4 hasta que se indique.*
5. El Sistema presenta el total con los impuestos calculados.
 6. El Cajero le dice al Cliente el total y pide que le pague.
 7. El Cliente paga y el Sistema gestiona el pago.
 8. El Sistema registra la venta completa y envía la información de la venta y el pago al sistema de Contabilidad externo (para la contabilidad y las comisiones) y al sistema de Inventario (para actualizar el inventario).
 9. El Sistema presenta el recibo.
 10. El Cliente se va con el recibo y las mercancías (si es el caso).

Extensiones (o Flujos Alternativos):

- *a. En cualquier momento el Sistema falla:
- Para dar soporte a la recuperación y registro correcto, asegura que todos los estados y eventos significativos de una transacción puedan recuperarse desde cualquier paso del escenario.
1. El Cajero reinicia el Sistema, inicia la sesión, y solicita la recuperación al estado anterior.
 2. El Sistema reconstruye el estado anterior.
- 2a. El Sistema detecta anomalías intentando la recuperación:
1. El Sistema informa del error al Cajero, registra el error, y pasa a un estado limpio.
 2. El Cajero comienza una nueva venta.

- 3a. Identificador no válido:
 1. El Sistema señala el error y rechaza la entrada.
- 3b. Hay muchos artículos de la misma categoría y tener en cuenta una única identidad del artículo no es importante (ej. 5 paquetes de hamburguesas vegetales):
 1. El Cajero puede introducir el identificador de la categoría del artículo y la cantidad.
- 3-6a. El Cliente le pide al Cajero que elimine un artículo de la compra:
 1. El Cajero introduce el identificador del artículo para eliminarlo de la compra.
 2. El Sistema muestra la suma parcial actualizada.
- 3-6b. El Cliente le pide al Cajero que cancele la venta:
 1. El Cajero cancela la venta en el Sistema.
- 3-6c. El Cajero detiene la venta:
 1. El sistema registra la venta para que esté disponible su recuperación en cualquier terminal PDV.
- 4a. El Sistema genera el precio de un artículo que no es el deseado (ej. el Cliente se queja por algo y se le ofrece un precio más bajo):
 1. El Cajero introduce el precio alternativo.
 2. El Sistema presenta el precio nuevo.
- 5a. El sistema encuentra algún fallo para comunicarse con el servicio externo del sistema de cálculo de impuestos.
 1. El Sistema reinicia el servicio en el nodo PDV y continúa.
 - 1a. El Sistema detecta que el servicio no se reinicia.
 1. El Sistema señala el error.
 2. El Cajero podría calcular e introducir manualmente el impuesto, o cancelar la venta.
- 5b. El Cliente dice que le son aplicables descuentos (ej. empleado, cliente preferente):
 1. El Cajero señala la petición de descuento.
 2. El Cajero introduce la identificación del Cliente.
 3. El Sistema presenta el descuento total, basado en las reglas de descuento.
- 5c. El Cliente dice que tiene crédito en su cuenta, para aplicar a la venta:
 1. El Cajero señala la petición de crédito.
 2. El Cajero introduce la identificación del Cliente.
 3. El Sistema aplica el crédito hasta que el precio = 0, y reduce el crédito que queda.
- 6a. El Cliente dice que su intención era pagar en efectivo pero que no tiene suficiente:
 - 1a. El Cliente utiliza un método de pago alternativo.
 - 1b. El Cliente le dice al Cajero que cancele la venta. El Cajero cancela la venta en el Sistema.
- 7a. Pago en efectivo:
 1. El Cajero introduce la cantidad de dinero en efectivo entregada.
 2. El Sistema muestra la cantidad de dinero a devolver y abre el cajón de caja.
 3. El Cajero deposita el dinero entregado y devuelve el cambio al Cliente.
 4. El Sistema registra el pago en efectivo.
- 7b. Pago a crédito:
 1. El Cliente introduce la información de su cuenta de crédito.
 2. El Sistema envía la petición de autorización del pago al Sistema externo de Servicio de Autorización de Pagos, y solicita la aprobación del pago.
 - 2a. El Sistema detecta un fallo en la colaboración con el sistema externo:
 1. El Sistema señala el error al Cajero.
 2. El Cajero le pide al Cliente un modo de pago alternativo.
 3. El Sistema recibe la aprobación del pago y lo notifica al Cajero.
 - 3a. El Sistema recibe la denegación del pago:
 1. El Sistema señala la denegación al Cajero.
 2. El Cajero le pide al Cliente un modo de pago alternativo.

4. El Sistema registra el pago a crédito, que incluye la aprobación del pago
5. El Sistema presenta el mecanismo de entrada para la firma del pago a crédito.
6. El Cajero le pide al Cliente que firme el pago a crédito. El Cliente introduce la firma.
- 7c. Pago con cheque...
- 7d. Pago a cuenta...
- 7e. El Cliente presenta cupones:
 1. Antes de gestionar el pago, el Cajero recoge cada cupón y el Sistema reduce el pago como sea oportuno. El sistema registra los cupones utilizados por razones de contabilidad.
 - 1a. El cupón introducido no es válido para ninguno de los artículos comprados
 1. El Sistema señala el error al Cajero.
- 9a. Hay rebajas en los artículos:
 1. El Sistema presenta los formularios de rebaja y los recibos de descuento para cada artículo con una rebaja.
- 9b. El Cliente solicita un vale-regalo (sin precio visible):
 1. El Cajero solicita el vale-regalo y el Sistema lo proporciona.

Requisitos especiales:

- Interfaz de Usuario con pantalla táctil en un gran monitor de pantalla plana. El texto debe ser visible a un metro de distancia.
- Tiempo de respuesta para la autorización de crédito de 30 segundos el 90% de las veces.
- De algún modo, queremos recuperación robusta cuando falla el acceso a servicios remotos, como el sistema de inventario.
- Internacionalización del lenguaje del texto que se muestra.
- Reglas de negocio que se puedan añadir en tiempo de ejecución en los pasos 3 y 7.
- ...

Lista de tecnología y variaciones de datos:

- 3a. El identificador del artículo se introduce mediante un escáner láser de código de barras (si está presente el código de barras) o a través del teclado.
- 3b. El identificador del artículo podría ser cualquier esquema de código UPC, EAN, JAN o SKU.
- 7a. La entrada de la información de la cuenta de crédito se lleva a cabo mediante un lector de tarjetas o el teclado.
- 7b. La firma de los pagos a crédito se captura en un recibo de papel. Pero en dos años, pronosticamos que muchos clientes querrán que se capture la firma digital.

Frecuencia: Podría ser casi continuo.

Temas abiertos:

- ¿Cuáles son las variaciones de la ley de impuestos?
- Explorar las cuestiones de recuperación de servicios remotos.
- ¿Cuál es la adaptación que se tiene que hacer para diferentes negocios?
- ¿Un cajero debe llevarse el dinero de la caja cuando salga del sistema?
- ¿Puede utilizar el cliente directamente el lector de tarjetas o tiene que hacerlo el cajero?

Este caso de uso es más bien ilustrativo que exhaustivo (aunque está basado en los requisitos de un sistema PDV real). Sin embargo, muestra suficiente detalle y complejidad para exponer de manera realista que los casos de uso completos pueden documentar muchos detalles de los requisitos. Este ejemplo nos servirá bien como modelo para muchos problemas de los casos de uso.

La variación de dos-columnas

Algunos prefieren el formato en dos columnas o conversacional, que destaca el hecho de que se establece una interacción entre los actores y el sistema. Se propuso por primera vez por Rebecca Wirfs-Brock en [Wirfs-Brock93], y también promueven su uso Constantine y Lockwood para ayudar en el análisis e ingeniería de usabilidad [CL99]. A continuación presentamos el mismo contenido utilizando el formato en dos columnas:

Caso de uso UC1: Procesar Venta

Actor principal: ...

...como antes...

Escenario principal de éxito:

Acción del actor (o intención)

1. El Cliente llega a un terminal PDV con mercancías y/o servicios que comprar.
2. El Cajero comienza una nueva venta.
3. El Cajero introduce el identificador del artículo.
- El Cajero repite los pasos 3-4 hasta que se indique*
6. El Cajero le dice al Cliente el total y pide que le pague.
7. El Cliente paga.

...

Responsabilidad del Sistema

4. Registra cada línea de la venta y presenta la descripción del artículo y la suma parcial.
5. El Sistema presenta el total con los impuestos calculados.
8. El Sistema gestiona el pago.
9. Registra la venta completa y envía la información de la venta y el pago al sistema de Contabilidad externo (para la contabilidad y las comisiones) y al sistema de Inventario (para actualizar el inventario). El Sistema presenta el recibo.

...

¿Cuál es el mejor formato?

No existe un mejor formato; unos prefieren el estilo de una columna, otros el de dos columnas. Las secciones se pueden añadir y quitar; los nombres de los títulos podrían cambiar. Ninguna de estas cosas es especialmente importante; la clave es escribir los detalles del escenario principal de éxito y sus extensiones de alguna forma. [Cockburn1] resume muchos formatos utilizables:

Práctica personal

Ésta es mi práctica, no una recomendación. Durante algunos años, utilicé el formato en dos columnas debido a su clara separación visual de la conversación. Sin embargo, he vuelto al estilo en una columna ya que es más compacto y más fácil de formatear, y el pequeño valor de la separación visual de la conversación, para mí, no supera estos beneficios. Encuentro que es todavía sencillo identificar visualmente las diferentes partes en la conversación (Cliente, Sistema,...) si cada parte y las respuestas del Sistema se asignan normalmente a sus propios pasos.

6.7. Explicación de las secciones

Elementos del prólogo

Son posibles muchos elementos opcionales en el prólogo. Sitúe al principio sólo los elementos que son importantes que se lean antes del escenario principal de éxito. Mueva el material de “encabezamiento” ajeno, al final de los casos de uso.

Actor principal: El actor principal que recurre a los servicios del sistema para cumplir un objetivo.

Importante: Personal involucrado y lista de intereses

Esta lista es más importante y práctica de lo que podría parecer a primera vista. Sugiere y delimita qué es lo que debe hacer el sistema. Citando a Cockburn:

El [sistema] funciona siguiendo un contrato entre el personal involucrado, donde los casos de usos detallan la parte de comportamiento del contrato... El caso de uso, como contrato de comportamiento, captura *todo y sólo* el comportamiento relacionado con la satisfacción de los intereses del personal involucrado [Cockburn01].

Esto contesta la pregunta: ¿Qué debería estar en un caso de uso? La respuesta es: lo que satisfaga los intereses de todo el personal involucrado. Además, empezando con el personal involucrado y sus intereses antes de escribir el resto del caso de uso, tenemos un modo de recordarnos cuáles deberían ser las responsabilidades más detalladas del sistema. Por ejemplo, ¿habría identificado una responsabilidad para gestionar la comisión del vendedor si no hubiese listado en primer lugar la persona involucrada vendedor y sus intereses? En el mejor de los casos al final, pero quizás lo habría echado en falta durante la primera sesión de análisis. El punto de vista del interés del personal involucrado proporciona un procedimiento metódico y completo para el descubrimiento y registro de todos los comportamientos requeridos.

Personal involucrado e intereses:

- Cajero: quiere entradas precisas, rápidas, y sin errores de pago, ya que las pérdidas se deducen de su salario.
- Vendedor: quiere que las comisiones de las ventas estén actualizadas.
- ...

Precondiciones y garantías de éxito (postcondiciones)

Las **precondiciones** establecen lo que *siempre debe* cumplirse antes de comenzar un escenario en el caso de uso. Las precondiciones *no* se prueban en el caso de uso, sino que son condiciones que se asumen que son verdad. Normalmente, una precondición implica un escenario de otro caso de uso que se ha completado con éxito, como inicio de sesión o el más general “el cajero se identifica y autentica”. Nótese que hay condiciones

que deben ser verdad, pero no tienen un valor práctico para que se escriban, como “el sistema tiene energía”. Las precondiciones comunican suposiciones importantes de las que el escritor del caso de uso piensa que los lectores deberían ser avisados.

Las **garantías de éxito** (o **postcondiciones**) establecen qué debe cumplirse cuando el caso de uso se completa con éxito —o bien el escenario principal de éxito o algún camino alternativo—. La garantía debería satisfacer las necesidades de todo el personal involucrado.

Precondiciones: El cajero se identifica y autentica.

Garantías de éxito (Postcondiciones): Se registra la venta. El impuesto se calcula de manera correcta. Se actualizan la Contabilidad y el Inventario. Se registran las comisiones. Se genera el recibo.

Escenario principal de éxito y pasos (o Flujo Básico)

También recibe el nombre de escenario del “camino feliz”, o más prosaico “Flujo Básico”. Describe el camino de éxito típico que satisface los intereses del personal involucrado. Nótese que, a menudo, *no* incluye ninguna condición o bifurcación. Aunque no es incorrecto o ilegal, se puede suponer que es más comprensible y extensible ser muy consistente, y postergar todo el manejo de caminos condicionales a la sección Extensiones.

Sugerencia

Posponga todas las sentencias condicionales y de bifurcación a la sección Extensiones.

El escenario recoge los pasos, que pueden ser de tres tipos:

1. Una interacción entre actores³.
2. Una validación (normalmente a cargo del sistema).
3. Un cambio de estado realizado por el sistema (por ejemplo, registrando o modificando algo).

El primer paso de un caso de uso, normalmente no está incluido en esta clasificación, sino que indica el evento que desencadena el comienzo del escenario.

Un estilo habitual es poner con mayúsculas los nombres de los actores para facilitar la identificación. También podemos observar el estilo que se utiliza para indicar una repetición.

Escenario principal de éxito (o Flujo Básico):

1. El Cliente llega a un terminal PDV con mercancías para comprar.
 2. El Cajero comienza una nueva venta.
 3. El Cajero introduce el identificador del artículo.
 4. ...
- El Cajero repite los pasos 3-4 hasta que se indique.
5. ...

³ Nótese que el sistema que se está estudiando en sí mismo debería considerarse un actor cuando juega un rol de actor colaborando con otros sistemas.

Extensiones (o Flujos Alternativos)

Las extensiones son muy importantes. Indican todos los otros escenarios o bifurcaciones, tanto de éxito como de fracaso. Podemos observar que en el ejemplo de caso de uso completo, la sección Extensiones es considerablemente más larga y compleja que la correspondiente al Escenario Principal de Éxito; esto es normal y de esperar. También se conocen como “Flujos Alternativos”.

En la escritura de casos de uso completos, la combinación del camino feliz y los escenarios de extensión deberían satisfacer “casi” todos los intereses del personal involucrado. Este punto está limitado, puesto que algunos intereses se podrían capturar mejor como requisitos no funcionales escritos en la Especificación Complementaria en lugar de en los casos de uso.

Los escenarios de extensión son bifurcaciones del escenario principal de éxito y, por tanto, pueden ser etiquetados de acuerdo con él. Por ejemplo, en el Paso 3 del escenario principal podría haber un identificador de artículo inválido, bien porque no se introdujo correctamente o bien porque el sistema no lo conoce. Una extensión se etiqueta como “3a”; primero identifica la condición y después la respuesta. Una extensión alternativa al Paso 3 se etiqueta como “3b” y así sucesivamente.

Extensiones (o Flujos Alternativos):

3a. Identificador no válido:

1. El Sistema señala el error y rechaza la entrada.

3b. Hay muchos artículos de misma categoría y tener en cuenta una única identidad del artículo no es importante (ej. 5 paquetes de hamburguesas vegetales):

1. El Cajero puede introducir el identificador de la categoría del artículo y la cantidad.

Una extensión tiene dos partes: la condición y el manejo.

Guía: Escriba la condición como algo que pueda ser *detectado* por el sistema o un actor. Para contrastar:

5a. El Sistema detecta un fallo en la comunicación con el servicio externo del sistema de cálculo de impuestos:

5a. El sistema de cálculo de impuestos externo no funciona:

Se prefiere el primer estilo porque se trata de algo que el sistema puede detectar; el último es una inferencia.

El manejo de la extensión se puede resumir en un paso, o incluir una secuencia, como en este ejemplo, que también ilustra la notación utilizada para indicar que una condición puede tener lugar en una serie de pasos:

3-6a. El Cliente le pide al Cajero que elimine un artículo de la compra:

1. El Cajero introduce el identificador del artículo para eliminarlo de la compra.
2. El Sistema muestra la suma parcial actualizada.

Al final del manejo de la extensión, por defecto, el escenario se une de nuevo con el escenario principal de éxito, a menos que la extensión indique otra cosa (como interrumpir el sistema).

Algunas veces, un punto de extensión particular es bastante complejo, como en la extensión “pago a crédito”. Esto puede ser un motivo para expresar la extensión como un caso de uso aparte.

Este ejemplo de extensión también muestra la notación para expresar fallos en las extensiones.

- 7b. Pago a crédito:
1. El Cliente introduce la información de su cuenta de crédito.
 2. El Sistema envía la petición de autorización del pago al Sistema externo de Servicio de Autorización de Pagos, y solicita la aprobación del pago.
 - 2a. El Sistema detecta un fallo en la colaboración con el sistema externo:
 1. El Sistema señala el error al Cajero.
 2. El Cajero le pide al Cliente un modo de pago alternativo.
 - 3 ...

Si es deseable describir una condición de extensión que puede ser posible durante cualquiera (o al menos la mayoría) de los pasos, se pueden utilizar las etiquetas *a, *b,...

- *a. En cualquier momento el Sistema falla:
- Para dar soporte a la recuperación y registro correcto, asegura que todos los estados y eventos significativos de una transacción puedan recuperarse desde cualquier paso del escenario.
1. El Cajero reinicia el Sistema, inicia la sesión, y solicita la recuperación al estado anterior.
 2. El Sistema reconstruye el estado anterior.

Requisitos especiales

Si un requisito no funcional, atributo de calidad o restricción se relaciona de manera específica con un caso de uso, se recoge en el caso de uso. Esto incluye cualidades tales como rendimiento, fiabilidad y facilidad de uso, y restricciones de diseño (a menudo, en dispositivos de entrada/salida) que son obligados o se consideran probables.

Requisitos especiales:

- Interfaz de usuario con pantalla táctil en un gran monitor de pantalla plana. El texto debe ser visible a un metro de distancia.
- Tiempo de respuesta para la autorización de crédito de 30 segundos el 90% de las veces.
- Internacionalización del lenguaje del texto que se muestra.
- Reglas de negocio que se puedan añadir en ejecución en los pasos 3 y 7.

Un consejo clásico del UP es registrar estos requisitos con el caso de uso, y constituye un lugar razonable al escribir primero el caso de uso. Sin embargo, muchos expertos encuentran útil reunir al final todos los requisitos no funcionales en la Especificación Complementaria, para favorecer la gestión del contenido, comprensión y legibilidad por-

que, normalmente, se tienen que considerar estos requisitos en conjunto durante el análisis arquitectural.

Lista de tecnología y variaciones de datos

A menudo, encontramos variaciones técnicas en *cómo* se debe hacer algo, pero no en *qué*, y es importante registrarlo en el caso de uso. Un ejemplo típico es una restricción técnica impuesta por el personal involucrado con respecto a las tecnologías de entrada o salida de datos. Por ejemplo, uno de los interesados podría decir, “El sistema PDV debe soportar la entrada de la cuenta de crédito utilizando un lector de tarjetas y el teclado”. Nótese que éstos son ejemplos de decisiones o restricciones de diseño anticipadas; en general, deben evitarse decisiones de diseño prematuras, pero algunas veces son obvias o inevitables, especialmente en relación con las tecnologías de entrada/salida.

También es necesario entender las variaciones en los esquemas de los datos, como utilizar UPCs o EANs para los identificadores de los artículos, codificados mediante el código de barras.

Esta lista es el lugar para situar tales variaciones. También es útil registrar las variaciones en los datos que podrían capturarse en un paso particular.

Lista de tecnología y variaciones de datos

- 3a. El identificador del artículo se introduce mediante un escáner láser de código de barras (si está presente el código de barras) o a través del teclado.
- 3b. El identificador del artículo podría ser cualquier esquema de código UPC, EAN, JAN o SKU.
- 7a. La entrada de la información de la cuenta de crédito se lleva a cabo mediante un lector de tarjetas o el teclado.
- 7b. La firma de los pagos a crédito se captura en un recibo de papel. Pero en dos años, pronosticamos que muchos clientes querrán que se capture la firma digital.

Sugerencia

Esta sección no debería contener múltiples pasos para representar la variación de comportamiento en diferentes casos. Si es necesario, dígalos en la sección Extensiones.

6.8. Objetivos y alcance de un caso de uso

¿Cómo deberían descubrirse los casos de uso? Es típico no estar seguro si algo es un caso de uso válido (o de manera más práctica, útil). Las tareas se pueden agrupar a muchos niveles de granularidad, desde uno o unos pocos pasos pequeños, hasta actividades de nivel de empresa.

¿A qué nivel y alcance deberían expresarse los casos de uso?

Las siguientes secciones presentan las ideas sencillas de los procesos y objetivos del negocio elementales, como marco para la identificación de los casos de uso de una aplicación.

Casos de uso para los procesos del negocio elementales

¿Cuál de éstos es un caso de uso válido?

- Negociar un Contrato con el Proveedor
- Gestionar las Devoluciones
- Iniciar Sesión

Podría argumentarse que todos ellos son casos de uso *a diferentes niveles*, dependiendo de los límites del sistema, actores y objetivos. La evaluación de estos candidatos se presenta después de una introducción a los procesos del negocio elementales.

En lugar de preguntar en general: “¿qué es un caso de uso válido?”, una pregunta más relevante para el caso de estudio PDV es: ¿cuál es el nivel útil para expresar los casos de uso en el análisis de requisitos de la aplicación?

Guía: El caso de uso EBP

Para el análisis de requisitos de una aplicación informática, céntrese en los casos de uso al nivel de **procesos del negocio elementales** (EBPs, *Elementary Business Processes*).

EBP es un término que procede del campo de la ingeniería de procesos del negocio⁴, y se define como:

Una tarea realizada por una persona en un lugar, en un instante, como respuesta a un evento del negocio, que añade un valor cuantificable para el negocio y deja los datos en un estado consistente, ej. Autorizar Crédito, o Solicitar Precio (se perdió la fuente original).

Esto se puede tomar demasiado literalmente: ¿Está mal considerar un caso de uso como un EBP si se requieren dos personas, o si una persona tiene que pasear? Probablemente no; sin embargo, la impresión general de la definición es casi correcta. No se trata de un pequeño paso como “eliminar una línea de pedido” o “imprimir el documento”. Sino que el escenario principal de éxito está formado probablemente por cinco o diez pasos. No tarda días y múltiples sesiones, como “negociar un contrato con el proveedor”; es una tarea que se aborda en una única sesión. Probablemente dura entre unos pocos minutos y una hora. Como con la definición del UP, hace hincapié en añadir valor observable y cuantificable al negocio, y llega a un acuerdo en el que el sistema y los datos se encuentran en un estado estable y consistente.

⁴ EBP es parecido al término **tarea de usuario** en la ingeniería de usabilidad, aunque el significado es menos estricto en ese dominio.

Un error típico de los casos de uso es definir muchos casos de uso a un nivel muy bajo; es decir, como un paso simple, subfunción o subtarea en un EBP.

Violaciones razonables de la guía EBP

Aunque los casos de uso “base” de una aplicación deberían satisfacer la guía EBP, normalmente es útil crear “sub” casos de uso separados que representan subtareas, o pasos, en un caso de uso base. Pueden existir casos de uso que no sean EBP; potencialmente existen muchos a un nivel inferior. La guía sólo se utiliza para encontrar el nivel dominante de casos de uso en el análisis de requisitos de una aplicación; esto es, el nivel en el que nos tenemos que centrar para nombrarlos y escribirlos.

Por ejemplo, una subtarea o extensión como “pago a crédito” podría repetirse en varios casos de uso base. Es conveniente separarlo en un caso de uso propio (que no satisfice la guía EBP) y conectarlo a varios casos de uso base, para evitar duplicaciones del texto.

En el Capítulo 25 estudiaremos las cuestiones acerca de las relaciones de casos de uso.

Casos de uso y objetivos

Los actores tienen objetivos (o necesidades) y utilizan las aplicaciones para ayudarles a satisfacerlos. En consecuencia, un caso de uso de nivel EBP se denomina caso de uso de nivel de **objetivo de usuario**, para remarcar que sirve (o debería servir) para satisfacer un objetivo de un usuario del sistema, o el actor principal.

Esto nos lleva a recomendar el siguiente procedimiento:

1. Encontrar los objetivos de usuario.
2. Definir un caso de uso para cada uno.

Esto supone un ligero cambio de énfasis para el modelador de casos de uso. En lugar de preguntar: “¿Cuáles son los casos de uso?”, uno comienza preguntando: “¿Cuáles son tus objetivos?”. De hecho, el nombre del caso de uso para un objetivo de usuario debería reflejar dicho objetivo, para resaltar este punto de vista. Objetivo: capturar o procesar una venta; caso de uso: *Procesar Venta*.

Nótese que debido a esta simetría, la guía EBP se puede aplicar igualmente para decidir si un objetivo o un caso de uso se encuentran a un nivel adecuado.

Por tanto, he aquí una idea clave con respecto a la investigación de los objetivos de usuario frente a la investigación de casos de uso:

Imagine que estamos juntos en un taller de requisitos. Nos podríamos preguntar:

- “¿Qué haces?” (una pregunta bastante orientada al caso de uso) o;
- “¿Cuáles son tus objetivos?”

Es más probable que las respuestas a la primera pregunta reflejen soluciones y procedimientos actuales, y las complicaciones asociadas a ellos.

Las respuestas a la segunda pregunta, especialmente combinada con una investigación para ascender en la jerarquía de objetivos (“¿cuál es el objetivo de ese objetivo?”), abren la visión de soluciones nuevas y mejoradas, centradas en añadir valor al negocio, y llegar al corazón de lo que el personal involucrado quiere del sistema que se está estudiando.

Ejemplo: aplicación de la guía EBP

Un analista de sistemas responsable del descubrimiento de los requisitos del sistema NuevaEra, está investigando los objetivos de usuario. La conversación transcurre de esta forma durante un taller de requisitos:

Analista de sistemas: “¿Cuáles son algunos de sus objetivos en el contexto de uso de un sistema PDV?”

Cajero: “Uno, iniciar la sesión rápidamente. También, capturar las ventas.”

Analista de sistemas: “¿Cuál cree que es el objetivo de nivel más alto que motiva el inicio de sesión?”

Cajero: “Intento identificarme en el sistema, de este modo puede validar que estoy autorizado para utilizar el sistema que captura ventas y otras tareas.”

Analista de sistemas: “¿Más alto que ése?”

Cajero: “Evitar robos, alteración de datos, y mostrar información privada de la compañía.”

Obsérvese que la estrategia del analista de buscar de manera ascendente en la jerarquía de objetivos para encontrar los objetivos de usuario de un nivel superior que todavía satisfagan la guía EBP, para obtener el objetivo real detrás de la acción, y también para entender el contexto de los objetivos.

“Evitar robos...” es de un nivel superior a un objetivo de usuario; podría llamarse objetivo de empresa, y no es un EBP. Por tanto, aunque puede inspirar nuevas formas de pensar en el problema y las soluciones (como eliminar sistemas PDV y cajeros completamente), lo vamos a dejar a un lado por ahora.

Disminuyendo el nivel del objetivo a “identificarme en el sistema y ser validado” se acerca al nivel de objetivo de usuario. ¿Pero es el nivel EBP? No añade valor observable o cuantificable al negocio. Si el responsable del comercio pregunta: “¿Qué hiciste hoy?” y dices “¡Inicié la sesión 20 veces!”, no se impresionaría. En consecuencia, es un objetivo secundario, siempre disponible para hacer algo útil, y no es un EBP u objetivo de usuario. En cambio, “capturar una venta” cumple el criterio para ser un EBP u objetivo de usuario.

Otro ejemplo podría ser, en algunas tiendas existe un proceso denominado “abrir caja”, en el cual un cajero inserta su propia bandeja del cajón de caja en el terminal, inicia la sesión, e indica al sistema el dinero que hay en caja. *Abrir caja* es un caso de uso de nivel EBP (o nivel de objetivo de usuario); el paso de inicio de sesión, en lugar de ser un caso de uso de nivel EBP, es un objetivo de subfunción para llevar a cabo el objetivo de abrir caja.

Objetivos y casos de uso de subfunción

Aunque “identificarme y ser validado” (o “iniciar sesión”) se ha eliminado como objetivo de usuario, es un objetivo de nivel más bajo, denominado **objetivo de subfunción**—subobjetivos que dan soporte a un objetivo de usuario—. Sólo deberían escribirse casos de uso de manera ocasional para estos objetivos de subfunción, aunque es un problema típico que observan los expertos cuando se les pide que evalúen y mejoren (normalmente que simplifiquen) un conjunto de casos de uso.

No es ilegal escribir casos de uso para objetivos de subfunción, pero no siempre es útil, ya que añade complejidad al modelo de casos de uso; puede haber cientos de objetivos de subfunción—o casos de uso de subfunción— en un sistema.

Un punto importante es que el número y granularidad de los casos de uso influyen en el tiempo y la dificultad para entender, mantener y gestionar los requisitos.

El motivo válido, más común para representar un objetivo de subfunción como un caso de uso, es cuando la subfunción se repite o es una precondition en muchos casos de uso de nivel de objetivos de usuario. Este hecho se cumple probablemente en el caso de “identificarme y ser validado”, que es una precondition de la mayoría, si no todos, los otros casos de uso de nivel de objetivos de usuario.

En consecuencia, podría escribirse como el caso de uso *Autenticar Usuario*.

Objetivos y casos de uso pueden ser compuestos

Normalmente, los objetivos son compuestos, desde el nivel de empresa (“ser rentable”), que incluyen muchos objetivos intermedios a nivel de uso de la aplicación (“se capturan las ventas”), que a su vez incluyen objetivos de subfunción dentro de las aplicaciones (“la entrada es válida”).

De manera análoga, los casos de uso se pueden escribir a niveles diferentes para satisfacer estos objetivos, y pueden estar compuestos de casos de uso de nivel inferior.

Estos diferentes niveles de objetivos y casos de uso son una fuente típica de confusión en la identificación del nivel adecuado de los casos de uso de una aplicación. La guía EBP proporciona una orientación para eliminar casos de uso de nivel excesivamente bajo.

6.9. Descubrimiento de actores principales, objetivos y casos de uso

Los casos de uso se definen para satisfacer los objetivos de usuario de actores principales. Por tanto, el procedimiento básico es:

1. Elegir los límites del sistema. ¿Es sólo una aplicación software, el hardware y la aplicación como un todo, que lo utiliza más de una persona o una organización completa?

2. Identificar los actores principales —aquellos que tienen objetivos de usuario que se satisfacen mediante el uso de los servicios del sistema—.
3. Para cada uno, identificar sus objetivos de usuario. Elevarlos al nivel de objetivos de usuario más alto que satisfaga la guía EBP.
4. Definir los casos de uso que satisfagan los objetivos de usuario; nombrarlos de acuerdo con sus objetivos. Normalmente, los casos de uso del nivel de objetivo de usuario se corresponderán uno a uno con los objetivos de usuario, aunque hay al menos una excepción, como se verá.

Paso 1: Elegir el límite del sistema

Para este caso de estudio, el propio sistema PDV es el sistema que se está diseñando; todo lo que queda fuera de él, está fuera de los límites del sistema, incluyendo el cajero, el servicio de autorización de pagos, etcétera.

Si no está clara la definición de los límites del sistema que se está diseñando, se puede aclarar definiendo lo que está fuera —los actores principales externos y de apoyo—. Una vez identificados los actores externos, los límites se vuelven más claros. Por ejemplo, ¿se encuentra la responsabilidad de autorización de pagos completa en los límites del sistema? No, hay un actor del servicio externo de autorización de pagos.

Pasos 2 y 3: Identificar los actores principales y objetivos

Es artificial establecer de manera estricta que la identificación de los actores principales es antes que los objetivos de usuario; en un taller de requisitos, la gente pone en común sus ideas y dan lugar a una mezcla de ambos. Algunas veces los objetivos ponen de manifiesto a los actores, o viceversa.

Guía: Centrar la discusión en los actores principales en primer lugar, ya que establece el marco para las investigaciones posteriores.

Preguntas útiles para encontrar los actores principales y objetivos

Además de los actores principales y objetivos de usuario obvios, las preguntas siguientes ayudan a identificar otros que se podrían haber pasado:

- | | |
|---|--|
| ¿Quién arranca y para el sistema? | ¿Quién se encarga de la administración del sistema? |
| ¿Quién gestiona a los usuarios y la seguridad? | ¿Es un actor el “tiempo” porque el sistema hace algo como respuesta a un evento de tiempo? |
| ¿Existe un proceso de control que reinicie el sistema si falla? | ¿Quién evalúa la actividad o el rendimiento del sistema? |

¿Cómo se gestionan
las actualizaciones de software?
¿Actualizaciones automáticas o no?

¿Quién evalúa los registros?
¿Se recuperan de manera remota?

Actores principales y de apoyo

Recordemos que los actores principales tienen objetivos de usuario que se satisfacen mediante el uso de los servicios del sistema. Acuden al sistema para que les ayude. Al contrario que los *actores de apoyo*, que proporcionan servicios al sistema que se está diseñando. Por ahora, nos centraremos en encontrar los actores principales, no los de apoyo.

Recordemos también que los actores principales pueden ser —entre otras cosas— otros sistemas informáticos, como procesos software “guardianes” (“*watchdog*”).

Sugerencia

Desconfía si ninguno de los actores principales se corresponde con un sistema informático externo.

La lista actor-objetivo

Recoge los actores principales y sus objetivos de usuario en una lista actor-objetivo. En términos de los artefactos del UP, debería corresponderse con una sección del artefacto Visión (que se describirá en el capítulo siguiente).

Por ejemplo:

<i>Actor</i>	<i>Objetivo</i>	<i>Actor</i>	<i>Objetivo</i>
Cajero	procesar ventas procesar alquileres gestionar las devoluciones abrir caja cerrar caja ...	Administrador del Sistema	añadir usuarios modificar usuarios eliminar usuarios gestionar seguridad gestionar las tablas del sistema ...
Director	poner en marcha suspender operación ...	Sistema de Actividad de Ventas	analizar los datos de ventas y rendimiento
...	...	...	

El Sistema de Actividad de Ventas es una aplicación remota a la que se le solicitará con frecuencia datos de ventas desde cada nodo PDV en la red.

Dimensión de la planificación del proyecto

En la práctica, esta lista tiene columnas adicionales para la prioridad, esfuerzo y riesgo; esto se tratará brevemente en el Capítulo 36.

La complicada realidad

Esta lista parece ordenada, pero la realidad de su creación es cualquier cosa salvo eso. Son necesarias muchas “tormentas de ideas” y discusiones durante los talleres de requisitos. Consideremos el ejemplo anterior que ilustraba la aplicación de la regla EBP al objetivo de “iniciar sesión”. Durante el taller, mientras se crea esta lista, el cajero podría proponer “iniciar sesión” como un objetivo de usuario. El analista de sistemas profundizará y subirá el nivel del objetivo, más allá del mecanismo de bajo nivel de inicio de sesión (el cajero posiblemente estaba pensando en utilizar un cuadro de diálogo en una GUI), al nivel de “identificar y autenticar al usuario”. Pero, el analista entonces se da cuenta de que no cumple la guía EBP, y lo descarta como objetivo de usuario. Por supuesto, la realidad es incluso algo diferente a esto puesto que un analista con experiencia cuenta con un conjunto de heurísticas, procedentes de experiencias o estudios anteriores, una de las cuales es “la autenticación de los usuarios es rara vez un EBP”, y de este modo es probable que se elimine rápidamente.

El actor principal y los objetivos de usuario dependen del límite del sistema

¿Por qué es el cajero, y no el cliente, el actor principal del caso de uso de *Procesar Venta*? ¿Por qué no aparece el cliente en la lista actor-objetivo?

La respuesta depende del límite del sistema que se esté diseñando, como se ilustra en la Figura 6.1. Si consideramos la empresa o el servicio de caja como un sistema agregado, el cliente *es* un actor principal, con el objetivo de obtener artículos o servicios, y marcharse. Sin embargo, desde el punto de vista de únicamente el sistema PDV (que es la elección del límite del sistema para este caso de estudio), éste atiende el objetivo del cajero (y de la tienda) de procesar la venta del cliente.

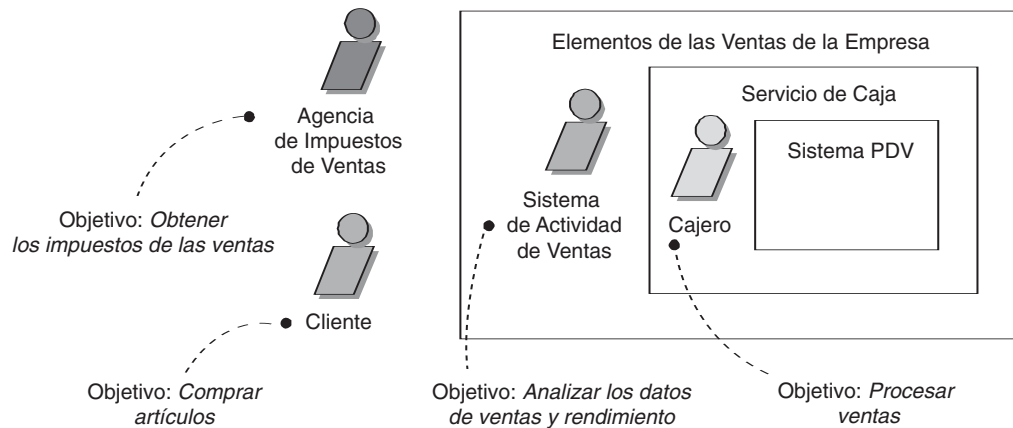


Figura 6.1. Actores principales y objetivos con diferentes límites del sistema.

Actores y objetivos por medio del análisis de eventos

Otro enfoque para ayudar en la búsqueda de los actores, objetivos y casos de uso, es identificar los eventos externos. ¿Cuáles son, de dónde proceden y por qué? A menudo,

un grupo de eventos pertenecen al mismo objetivo o caso de uso de nivel EBP. Por ejemplo:

<i>Evento Externo</i>	<i>Parte del Actor</i>	<i>Objetivo</i>
introducir línea de venta	Cajero	procesar una venta
introducir el pago	Cliente o Cajero	procesar una venta
...	...	

Paso 4: Definir los casos de uso

Por lo general, definimos un caso de uso de nivel EBP por cada objetivo de usuario. Nombramos el caso de uso de manera similar al objetivo de usuario; por ejemplo, Objetivo: procesar una venta; Caso de Uso: *Procesar Venta*.

También, nombre los casos de uso comenzando con un verbo.

Una excepción típica a un caso de uso por objetivo, es agrupar objetivos separados CRUD⁵ (crear, recuperar, actualizar, eliminar) en un caso de uso CRUD, llamado, por convención, *Gestionar<X>*. Por ejemplo, los objetivos “editar usuario”, “eliminar usuario”, etcétera, todos se satisfacen en el caso de uso *Gestionar Usuarios*.

“Definir casos de uso” comprende varios niveles de esfuerzo, variando desde unos pocos minutos, para recoger simplemente los nombres, hasta semanas, con el fin de escribir las versiones en formato completo. La última sección del proceso UP de este capítulo, sitúa este trabajo —cuándo y cuánto— en el contexto del desarrollo iterativo y del UP.

6.10. Enhorabuena: se han escrito los casos de uso y no son perfectos

La necesidad de comunicación y participación

El equipo del PDV NuevaEra va escribiendo casos de uso en múltiples talleres de requisitos, a través de una serie de iteraciones de desarrollo cortas, añadiendo incrementalmente al conjunto, y refinando y adaptando en base a la retroalimentación. Los expertos en la materia de estudio, los cajeros y programadores participan activamente en el proceso de escritura. No existen intermediarios entre los cajeros, otros usuarios y los desarrolladores, sino que se establece una comunicación directa.

Está bien, pero no es suficiente. Las especificaciones de requisitos escritas dan la impresión de ser correctas; pero no lo son. Los casos de uso y otros requisitos todavía

⁵ N. del T.: Acrónimo de Create-Retrieve-Uppdate-Delete.

no serán correctos —garantizado—. Carecen de información crítica y contienen afirmaciones erróneas. La solución no es la aptitud del proceso “en cascada” de esforzarse mucho por recopilar los requisitos perfectos y completos al principio, aunque, por supuesto, lo hacemos lo mejor que podemos en el tiempo disponible. Pero nunca será suficiente.

Se necesita un enfoque diferente. Una buena parte del enfoque es el desarrollo iterativo, pero se necesita algo más: *comunicación personal continua*. Comunicación y participación cercana y continua —diaria— entre los desarrolladores y alguien que entienda el dominio y pueda tomar decisiones sobre los requisitos. Alguno de los programadores se puede acercar y en cuestión de segundos obtener aclaraciones, en cualquier momento que surja una duda. Por ejemplo, las prácticas de XP [Beck00] incluyen una excelente recomendación: *los usuarios deben tener dedicación a tiempo completo al proyecto, permaneciendo en la sala del proyecto*.

6.11. Escritura de casos de uso en un estilo esencial independiente de la interfaz de usuario

¡Nuevo y mejorado! Razones a favor de utilizar las huellas dactilares

Investigar y preguntar acerca de los objetivos, en lugar de las tareas y procedimientos fomenta que se centre la atención en la esencia de los requisitos —la intención detrás de ellos—. Por ejemplo, durante un taller de requisitos, el cajero podría decir que uno de sus objetivos es “iniciar la sesión”. El cajero, probablemente, estaría pensando en una GUI, cuadro de diálogo, identificador de usuario (ID) y contraseña (*password*). Éste es un mecanismo para alcanzar un objetivo, en lugar del objetivo en sí mismo. Mediante la investigación ascendente de la jerarquía de objetivos (“¿Cuál es el objetivo del objetivo?”), el analista del sistema llega hasta el objetivo independiente del mecanismo: “identificarse y conseguir la autenticación”, o incluso un objetivo de nivel superior: “prevenir robos...”.

Este proceso de descubrimiento puede abrir la visión a soluciones nuevas y mejoradas. Por ejemplo, los teclados y ratones con lectores biométricos, normalmente para las huellas dactilares son ahora habituales y baratos. Si el objetivo es “identificación y autenticación”, ¿por qué no hacerlo rápido y fácil, utilizando un lector biométrico en el teclado? Pero, la respuesta adecuada conlleva también un análisis de usabilidad, como conocer el perfil de los usuarios típicos del sistema. ¿Qué pasa si tienen los dedos cubiertos de grasa? ¿Tienen dedos?

Escritura en estilo esencial

Esta idea se ha resumido en varias guías de casos de uso como “no considere la interfaz de usuario; céntrese en la intención”. [Cockburn01]. La motivación y notación la ha estudiado Larry Constantine de manera más completa, en el contexto de la creación de interfaces de usuario (UIs) mejores y de la ingeniería de usabilidad [Constantine94,

CL99]. Constantine denomina al estilo de escritura **esencial** cuando evita los detalles de UI y se centra en la intención real del usuario⁶.

En un estilo de escritura esencial, la narración se expresa al nivel de las *intenciones* de los usuarios y *responsabilidades* del sistema, en lugar de sus acciones concretas. Son independientes de los detalles acerca de la tecnología y los mecanismos, especialmente aquellos relacionados con la UI.

Escriba casos de uso en un estilo esencial; no considere la interfaz de usuario y céntrese en la intención del actor.

Todos los ejemplos de casos de uso anteriores de este capítulo, como *Procesar Venta*, se escribieron con la intención de seguir un estilo esencial.

Nótese que el diccionario define *objetivo* como sinónimo de intención [MW89], lo que ilustra la conexión entre la idea del estilo *esencial* de Constantine y el punto de vista orientado al objetivo que se puso de relieve anteriormente en este capítulo. De hecho, muchos pasos de las *intenciones* de los actores, también se pueden caracterizar como *objetivos* de subfunción.

Ejemplos de contraste

Estilo esencial

Supongamos que el caso de uso *Gestionar Usuarios* requiere identificación y autenticación. El estilo esencial, inspirado en Constantine, utiliza el formato en dos columnas. Sin embargo, se puede escribir en una columna.

...	
Intención del Actor	Responsabilidad del Sistema
1. El Administrador se identifica	2. Autenticar la identidad
3. ...	

En el formato de una columna esto se muestra como:

...
1. El Administrador se identifica
2. El Sistema autentica la identidad
3. ...

La solución de diseño para estas intenciones y responsabilidades está muy abierta: lectores biométricos, interfaces gráficas de usuario (GUIs), etcétera.

⁶ El término procede de los “modelos esenciales” en el *Análisis de Sistemas Esenciales* [MP84].

Estilo concreto—evitar durante el trabajo de requisitos inicial

En contraste, hay un estilo de **caso de uso concreto**. En este estilo, se incluyen en el texto del caso de uso las decisiones acerca de la interfaz de usuario. El texto podría incluso mostrar instantáneas de las pantallas de las ventanas, discutir la navegación de las ventanas, la manipulación de los elementos de la GUI, etcétera. Por ejemplo:

-
- ...
1. El Administrador introduce su ID y contraseña en el cuadro de diálogo (ver Dibujo 3).
 2. El Sistema autentica al Administrador.
 3. El Sistema muestra la ventana de “edición de usuarios” (ver Dibujo 4).
 4. ...
-

Estos casos de uso concretos podrían ser útiles para ayudar en el diseño de la GUI detallada y concreta en etapas posteriores, pero no son adecuados durante el trabajo del análisis de requisitos inicial. Durante el trabajo de requisitos inicial, “no considere la interfaz de usuario, céntrese en la intención”.

6.12. Actores

Un actor es cualquier cosa con comportamiento, incluyendo el propio sistema que se está estudiando (SuD, *System under Discussion*) cuando solicita los servicios de otros sistemas⁷. Los actores principales y de apoyo aparecerán en los pasos de acción del texto del caso de uso. Los actores no son solamente roles que juegan personas, sino también organizaciones, software y máquinas. Hay tres tipos de actores externos con relación al SuD:

- **Actor principal:** tiene objetivos de usuario que se satisfacen mediante el uso de los servicios del SuD. Por ejemplo, el cajero.
 - ¿Por qué se identifica? Para encontrar los objetivos de usuario, los cuales dirigen los casos de uso.
- **Actor de apoyo:** proporciona un servicio (por ejemplo, información) al SuD. El servicio de autorización de pago es un ejemplo. Normalmente se trata de un sistema informático, pero podría ser una organización o una persona.
 - ¿Por qué se identifica? Para clarificar las interfaces externas y los protocolos.
- **Actor pasivo:** está interesado en el comportamiento del caso de uso, pero no es principal ni de apoyo; por ejemplo, la agencia tributaria del gobierno.
 - ¿Por qué se identifica? Para asegurar que *todos* los intereses necesarios se han identificado y satisfecho. Los intereses de los actores pasivos algunas veces son sutiles o es fácil no tenerlos en cuenta, a menos que estos actores sean identificados explícitamente.

⁷ Éste fue un refinamiento y mejora para las definiciones de actores alternativas, incluyendo las de las primeras versiones de UML y el UP [Cockburn97]. Las antiguas definiciones excluían, de manera inconsistente, el SuD como actor, incluso cuando recurría a servicios de otros sistemas. Todas las entidades podrían jugar múltiples *roles*, incluyendo el SuD.

6.13. Diagramas de casos de uso

UML proporciona notación para los diagramas de casos de uso con el fin de ilustrar los nombres de los casos de uso y los actores, y las relaciones entre ellos (ver Figura 6.2).

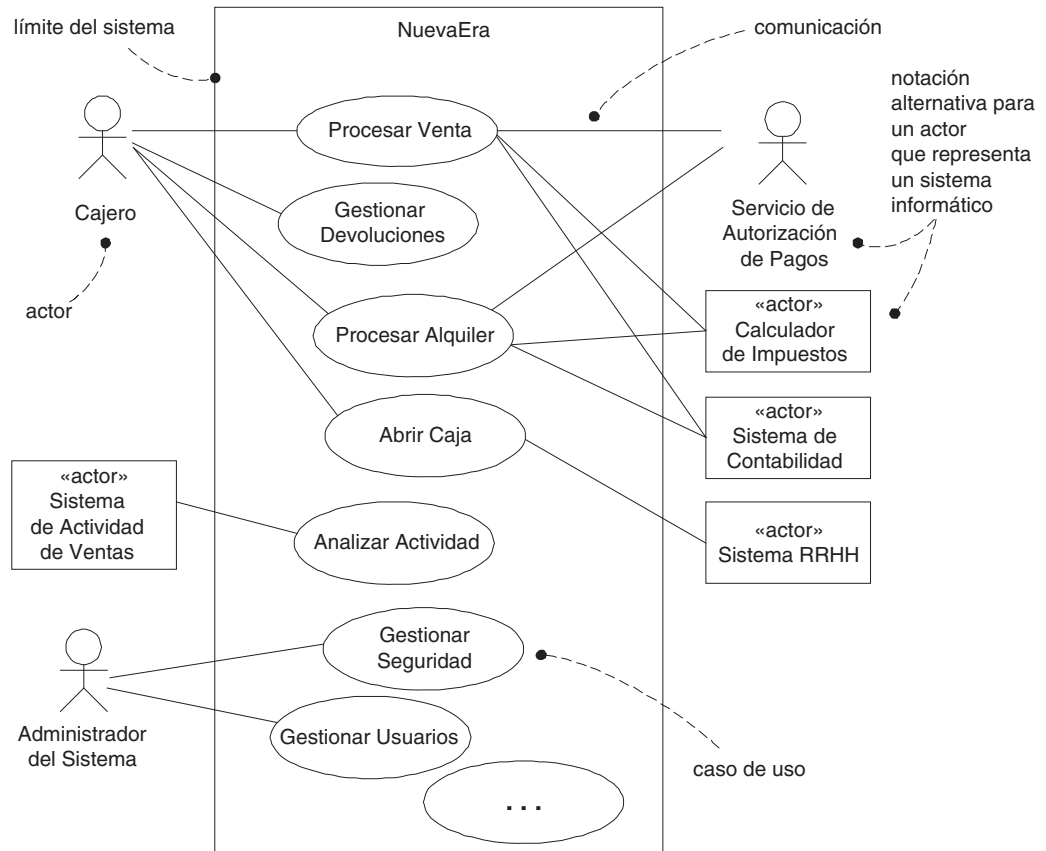


Figura 6.2. Diagrama de contexto de casos de uso parcial.

Los diagramas de caso de uso y las relaciones entre los casos de uso son secundarios en el trabajo con los casos de uso. Los casos de uso son documentos de texto. Trabajar con los casos de uso significa escribir texto.

Un signo típico de un modelador de casos de uso novato (o un estudiante) es la preocupación por los diagramas de casos de uso y las relaciones entre los casos de uso, en lugar de escribir texto. Los expertos mundiales en casos de uso, como Anderson, Fowler, Cockburn, entre otros, minimizan la importancia de los diagramas de casos de uso y las relaciones entre los casos de uso, y en lugar de eso se centran en escribir. Con eso como advertencia, un simple diagrama de casos de uso proporciona un conciso diagrama de contexto visual del sistema, que muestra los actores externos y cómo utilizan el sistema.

Sugerencia

Dibuje un diagrama de casos de uso sencillo junto con la lista actor-objetivo.

Un diagrama de casos de uso es una excelente representación del contexto del sistema; conforma un buen **diagrama de contexto**, esto es, muestra los límites de un sistema, lo que permanece fuera de él, y cómo se utiliza. Sirve como herramienta de comunicación que resume el comportamiento de un sistema y sus actores. La Figura 6.2 presenta una muestra de diagrama de contexto de casos de uso *parcial* para el sistema NuevaEra.

Sugerencias en la realización de los diagramas

La Figura 6.3 muestra algunos consejos sobre los diagramas. Nótese que la caja del actor contiene el símbolo «actor». Este símbolo se denomina **estereotipo UML**; se trata de un mecanismo para clasificar un elemento en cierto modo. El nombre de un estereotipo se escribe entre comillas francesas —signo ortográfico especial formado por un *único* carácter (no “<<” y “>>”) conocido sobre todo por su uso en la tipografía francesa para indicar una cita—.

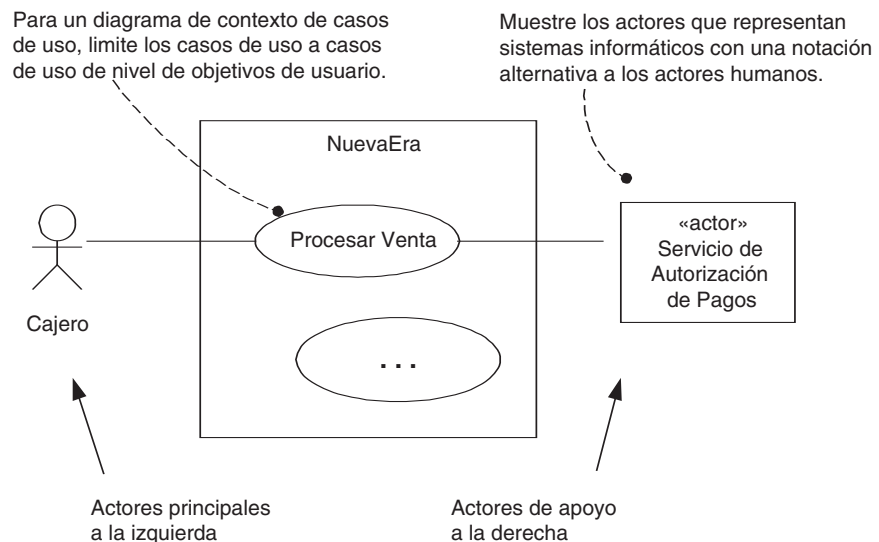


Figura 6.3. Sugerencias sobre la notación.

Para clarificar, algunos prefieren destacar los actores que se corresponden con sistemas informáticos externos con una notación alternativa, como se ilustra en la Figura 6.4.

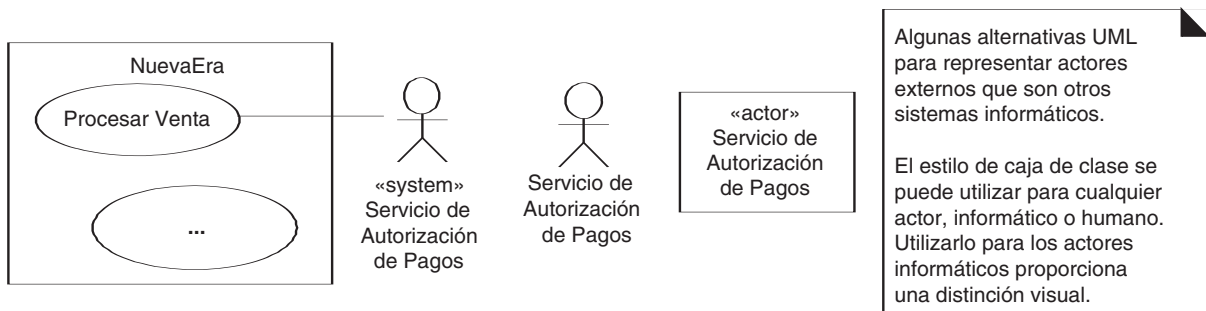


Figura 6.4. Notación alternativa para los actores.

Advertencia sobre el exceso de diagramas

Reiteramos que la importancia del trabajo de los casos de uso es escribir texto, no diagramas o centrarse en las relaciones entre los casos de uso. Si una organización dedica muchas horas (o peor, días) trabajando en los diagramas de casos de uso y debatiendo las relaciones en los casos de uso, en lugar de centrarse en escribir texto, se ha desaprovechado el esfuerzo.

6.14. Requisitos en contexto y listas de características de bajo nivel

Como queda implícito en el título del libro *Uses Cases: Requirements in Context* [GK00], una motivación clave de la idea de caso de uso es considerar y organizar los requisitos en el contexto de los objetivos y escenarios de uso de un sistema. Eso es bueno —mejora la cohesión y comprensión—. Sin embargo, los casos de uso no son los únicos artefactos de requisitos necesarios. Algunos requisitos no funcionales, reglas de dominio y contexto, y otros elementos difíciles de ubicar, se capturan mejor en la Especificación Complementaria, que se describirá en el siguiente capítulo.

Una idea detrás de los casos de uso es sustituir las listas de características de bajo nivel (típicas de los métodos de requisitos tradicionales) por los casos de uso (con algunas excepciones). Estas listas tendían a ser como sigue, normalmente agrupadas por áreas funcionales:

<i>ID</i>	<i>Característica</i>
CARAC1.9	El Sistema aceptará entradas de los identificadores de los artículos.
...	...
CARAC2.4	El Sistema registrará los pagos a crédito en el sistema de cuentas por cobrar.

De tales listas detalladas de las características de bajo nivel se puede aprovechar algo. Sin embargo, la lista completa no se recoge en media página; es más probable que ocupe docenas o cientos de páginas. Esto nos conduce a algunos obstáculos, que los casos de uso ayudan a abordar. Estos incluyen:

- Listas de funciones largas y detalladas no relacionan los requisitos en un contexto cohesivo; las diferentes funciones y características aparecen progresivamente como una “lista de la lavandería” inconexa de elementos. En cambio, los casos de uso sitúan los requisitos en el contexto de las historias y objetivos de uso del sistema.
- Si se utilizan tanto los casos de uso como las listas de características detalladas, hay duplicación. Más trabajo, mayor volumen para escribir y leer, más problemas de consistencia y sincronización.

Sugerencia

Esfuércese en sustituir las listas detalladas de características de bajo nivel por casos de uso.

Son aceptables las listas de características de alto nivel del sistema

Es típico y útil resumir la funcionalidad del sistema con una lista breve de características de alto nivel, denominadas características del sistema, en un documento de Visión. A diferencia de las 100 páginas de características detalladas de bajo nivel, la lista de características del sistema tiende a incluir sólo unas pocas docenas de elementos. La lista proporciona un resumen muy conciso de la funcionalidad del sistema, independiente de la vista del caso de uso. Por ejemplo:

Resumen de características del sistema

- capturar las ventas.
- autorización del pago (crédito, débito, cheque).
- administración del sistema para los usuarios, seguridad, tablas de códigos y constantes, etcétera.
- procesamiento automático de ventas sin conexión, cuando fallan los componentes externos.
- transacciones en tiempo real, en base a los estándares industriales, con sistemas de terceras partes, que incluye inventario, contabilidad, recursos humanos, cálculo de impuestos y servicios de autorización de pagos.
- definición y ejecución de reglas de negocio adaptadas y que se pueden añadir en ejecución en puntos típicos, fijados en los escenarios de proceso.
- ...

Esto se estudiará en el siguiente capítulo.

¿Cuándo son apropiadas las listas de características detalladas?

Algunas veces los casos de uso no encajan realmente; algunas aplicaciones exigen un punto de vista dirigido por las características. Por ejemplo, los servidores de aplicaciones, productos de bases de datos y otros sistemas *middleware* o *back-end* necesitan ante todo ser considerados y evolucionar en términos de las *características* (“Necesitamos el soporte de XML en la siguiente versión”). Los casos de uso no se ajustan de manera natural a estas aplicaciones o al modo en que necesitan evolucionar en términos de las fuerzas del mercado.

6.15. Los casos de uso no son orientados a objetos

No hay nada orientado a objetos en los casos de uso; uno no está realizando un análisis orientado a objetos si escribe casos de uso. Esto no es un defecto sino una aclaración. De hecho, los casos de uso constituyen una herramienta para el análisis de requisitos ampliamente aplicable, que se puede utilizar en proyectos no orientados a objetos, lo cual

incrementa su utilidad como método de requisitos. Sin embargo, como estudiaremos, los casos de uso son una entrada fundamental en las actividades clásicas de A/DOO.

6.16. Casos de uso en el UP

Los casos de uso son vitales y centrales en el UP, que fomentan el **desarrollo dirigido por casos de uso**. Esto implica:

- Los requisitos se recogen principalmente en casos de uso (el Modelo de Casos de Uso); otras técnicas de requisitos (como las listas de funciones) son secundarias, si es que se utilizan.
- Los casos de uso son una parte importante de la planificación iterativa. El trabajo de una iteración se define —en parte— eligiendo algunos escenarios de caso de uso, o casos de uso completos. Los casos de uso son una entrada clave para hacer estimaciones.
- Las **realizaciones de los casos de uso** dirigen el diseño. Es decir, el equipo diseña objetos y subsistemas que colaboran para ejecutar o realizar los casos de uso.
- Los casos de uso, a menudo, influyen en la organización de los manuales de usuario.

El UP diferencia entre casos de uso del sistema y del negocio. Los **casos de uso del sistema** son los que se han estudiado en este capítulo, como *Procesar Venta*. Se crean en la disciplina Requisitos, y forman parte del Modelo de Casos de Uso.

Los **casos de uso del negocio** se escriben con menos frecuencia. Si se hace, se crean en la disciplina Modelado del Negocio, como parte de un esfuerzo de reingeniería de los procesos de negocio a gran escala, o para ayudar a entender el contexto de un nuevo sistema en el negocio. Describen una secuencia de acciones de un negocio como un todo para cumplir un objetivo de un **actor del negocio** (un actor en el entorno del negocio, como un cliente o un proveedor). Por ejemplo, en un restaurante, un caso de uso del negocio es *Servir una Comida*.

Casos de uso y especificación de requisitos a lo largo de las iteraciones

Esta sección reitera la idea clave del UP y el desarrollo iterativo: medir el tiempo y el nivel de esfuerzo de la especificación de requisitos a lo largo de las iteraciones. La Tabla 6.1 presenta una muestra (no una receta) de la estrategia del UP sobre el modo de desarrollar los requisitos.

Nótese que el equipo técnico comienza construyendo los fundamentos de la producción del sistema cuando, quizás, sólo se han detallado el 10% de los requisitos, y de hecho, se retrasa de manera deliberada la continuación del trabajo de los requisitos concertados hasta casi el final de la primera iteración de elaboración.

Ésta es la diferencia clave entre el proceso iterativo y el proceso en cascada: el desarrollo con calidad de producción de los fundamentos del sistema comienza rápidamente, mucho antes de conocer todos los requisitos.

Obsérvese que cerca del final de la primera iteración de elaboración, hay un segundo taller de requisitos, durante el que quizás, el 30% de los casos de uso se escriben en

Tabla 6.1. Muestra del esfuerzo de los requisitos a lo largo de las primeras iteraciones; no es una receta.

Disciplina	Artefacto	Comentarios y nivel de esfuerzo de los requisitos				
		Inicio 1 semana	Elab 1 4 semanas	Elab 2 4 semanas	Elab 3 3 semanas	Elab 4 3 semanas
Requisitos.	Modelo de Casos de Uso.	2 días de taller de requisitos. Se identifican por el nombre la mayoría de los casos de uso y se resumen en un párrafo breve. Sólo el 10% se escribe en detalle.	Cerca del final de esta iteración, tiene lugar un taller de requisitos de 2 días. Se obtiene un mejor entendimiento y retroalimentación a partir del trabajo de implementación, entonces se completa el 30% de los casos de uso en detalle.	Cerca del final de esta iteración, tiene lugar un taller de requisitos de dos días. Se obtiene una mejor comprensión y retroalimentación a partir del trabajo de implementación, entonces se completa el 50% de los casos de uso en detalle.	Repetir, se completa el 70% de todos los casos de uso en detalle.	Repetir con la intención de clarificar y escribir en detalle del 80-90% de los casos de uso. Sólo una pequeña parte de éstos se construyen durante la elaboración; el resto se aborda durante la construcción.
Diseño.	Modelo de Diseño.	Nada.	Diseño de un pequeño conjunto de requisitos de alto riesgo significativos desde el punto de vista de la arquitectura.	Repetir.	Repetir.	Repetir. Deberían ahora estabilizarse los aspectos de alto riesgo significativos para la arquitectura.
Implementación.	Modelo de Implementación (código, etc.)	Nada.	Implementar esto.	Repetir. Se construye el 5% del sistema final.	Repetir. Se construye el 10% del sistema final.	Repetir. Se construye el 15% del sistema final.
Gestión del Proyecto.	Plan de Desarrollo de SW.	Estimación muy imprecisa del esfuerzo total.	La estimación comienza a tomar forma.	Un poco mejor...	Un poco mejor...	Ahora se pueden establecer racionalmente la duración global del proyecto, los hitos más importantes, estimación del coste y esfuerzo.

detalle. Este análisis de requisitos escalonado se beneficia de la retroalimentación a partir de la construcción de un poco del núcleo del software. La retroalimentación incluye la evaluación del usuario, pruebas y “conocimiento de lo que no conocemos” mejorado. Es decir, el acto de construir software rápidamente hace que surjan suposiciones y preguntas que necesitan aclararse.

Momento de la creación de los artefactos del UP

La Tabla 6.2 muestra algunos de los artefactos del UP y un ejemplo de la planificación de sus momentos de comienzo y refinamiento. El Modelo de Casos de Uso comienza en la fase de inicio, con quizás sólo el 10% de los casos de uso escritos con algo de detalle. La mayoría se escriben incrementalmente a lo largo de las iteraciones de la fase de elaboración, de manera que, al final de la elaboración se ha escrito un gran cuerpo de casos de uso detallados y otros requisitos (en la Especificación Complementaria), proporcionando una base realista para hacer una estimación precisa hasta el final del proyecto.

Tabla 6.2. Muestra de los artefactos UP y evolución temporal. c - comenzar; r – refinar.

<i>Disciplina</i>	<i>Artefacto</i> <i>Iteración →</i>	<i>Inicio</i> <i>I1</i>	<i>Elab.</i> <i>E1...En</i>	<i>Const.</i> <i>C1...Cn</i>	<i>Trans.</i> <i>T1...T2</i>
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c		
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Casos de uso en la fase de inicio

La siguiente discusión detalla la información presentada en la Tabla 6.1.

No todos los casos de uso se escriben en formato completo durante la fase de inicio. Más bien, suponga que se lleva a cabo un taller de requisitos durante dos días al comienzo del estudio de NuevaEra. La primera parte del día se dedica a identificar los objetivos y el personal involucrado, y especular sobre lo que queda dentro y fuera del alcance del proyecto. Se escribe una tabla de casos de uso actor-objetivo y se presenta con el proyector del ordenador. Se inicia el diagrama de contexto de casos de uso. Tras unas pocas horas, quizás se identifican unos 20 objetivos de usuario (y por tanto, casos de uso de nivel de usuario), que incluye *Procesar Venta*, *Gestionar Devoluciones*, etcétera. La mayoría de los casos de uso interesantes, complejos o arriesgados, se escriben en formato breve; cada uno escrito con una duración media de dos minutos. El equipo comienza a formarse un esquema de alto nivel de la funcionalidad del sistema.

Después de esto, entre el 10% y el 20% de los casos de uso que representan las funciones complejas principales, o que son especialmente arriesgadas en alguna dimensión, se escriben en formato completo; el equipo investiga con algo más de profundidad para entender mejor la magnitud, complejidad y demonios ocultos en el proyecto, a través de una pequeña muestra de casos de uso interesantes. Quizás esto puede implicar dos casos de uso: *Procesar Venta* y *Gestionar Devoluciones*.

Se utiliza una herramienta de gestión de requisitos integrada con un procesador de texto para la escritura, y el trabajo se muestra por medio de un proyector mientras el equipo colabora en el análisis y la escritura. Se escriben las listas de *Intereses* y *Personal Involucrado* para estos casos de uso, para descubrir requisitos más refinados (y quizás costosos) funcionales y no funcionales —o cualidades del sistema— claves, como la fiabilidad y el rendimiento.

El objetivo del análisis no es completar los casos de uso de manera exhaustiva, sino dedicar unas horas a comprenderlos mejor.

El promotor del proyecto necesita decidir si merece la pena un estudio profundo (esto es, la fase de elaboración). La intención del trabajo de inicio no es hacer este estudio, sino adquirir una idea de poca fidelidad (y claramente propensa a errores) acerca del alcance, riesgo, esfuerzo, viabilidad técnica, y análisis del negocio, para decir avanzar, dónde comenzar si se hace, o si parar.

Quizás la fase de inicio del proyecto NuevaEra duró cinco días. La combinación del taller de requisitos de dos días y su análisis de casos de uso breve, y otros estudios durante la semana, condujeron a tomar la decisión de continuar con la fase de elaboración para el sistema.

Casos de uso en la elaboración

La siguiente discusión detalla la información presentada en la Tabla 6.1.

Se trata de una fase de múltiples iteraciones de duración fija (por ejemplo, cuatro iteraciones) en las cuales se construyen incrementalmente partes del sistema arriesgadas, de alto valor o significativas desde el punto de vista de la arquitectura, y se identifican y clarifican la “mayoría” de los requisitos. La retroalimentación de los pasos concretos de programación influye e informa del conocimiento de los requisitos por parte del equipo, que se refina de manera iterativa y adaptable. Quizás se aborda un taller de requisitos de dos días en cada iteración —cuatro talleres—. Sin embargo, no se estudian todos los casos de uso en cada taller. Se priorizan; los primeros talleres se centran en un subconjunto de los casos de uso más importantes.

En cada siguiente taller de requisitos breve, es el momento de adaptar y refinar la visión de los requisitos principales, que serán inestables en las primeras iteraciones y se irán estabilizando en las últimas. Por tanto, hay una interacción iterativa entre el descubrimiento de los requisitos y la construcción de partes del software.

Durante cada taller de requisitos se refinan los objetivos de usuario y la lista de casos de uso. Se escriben, y reescriben, la mayoría de los casos de uso, en formato completo. Al final de la elaboración, se escriben en detalle del “80 al 90%” de los casos de uso. Para el sistema PDV con 20 casos de uso de nivel de objetivo de usuario, 15 o más de los más complejos y arriesgados deberían investigarse, escribirse y reescribirse en formato completo.

Nótese que la elaboración conlleva programar partes del sistema. Al final de esta etapa, el equipo NuevaEra no sólo debería tener una mejor definición de los casos de uso, sino también algo de software ejecutable de calidad.

Casos de uso en la construcción

La etapa de construcción está compuesta de iteraciones de duración fija (por ejemplo, 20 iteraciones de dos semanas cada una) que se centra en completar el sistema, una vez que las principales cuestiones arriesgadas e inestables se han establecido en la elaboración. Tendrá lugar todavía la escritura de casos de uso menores y quizás talleres de requisitos,

pero mucho menos de lo que se hizo en la elaboración. En esta etapa, la mayoría de los requisitos funcionales y no funcionales principales deberían haberse estabilizado de manera iterativa y adaptable. La intención no es dar a entender que los requisitos se congelan o el estudio termina, sino que el grado de cambio es mucho menor.

6.17. Caso de estudio: casos de uso en la fase de inicio de NuevaEra

Como se ha descrito en la sección anterior, no todos los casos de uso se escriben en el formato completo durante la fase de inicio. El Modelo de Casos de Uso de esta fase para el caso de estudio podría detallarse como sigue:

<i>Completo</i>	<i>Informal</i>	<i>Breve</i>
Procesar Venta	Procesar Alquiler	Abrir Caja
Gestionar Devoluciones	Analizar Actividad de Ventas	Cerrar Caja
	Gestionar Seguridad	Gestionar Usuarios
	...	Poner en Marcha
		Suspender Operación
		Gestionar Tablas del Sistema
		...

6.18. Lecturas adicionales

La guía de casos de uso de mayor éxito, traducida a varios idiomas es *Writing Effective Use Cases* [Cockburn01]⁸. Por buenas razones se ha convertido en el libro de casos de uso más ampliamente leído y seguido y, por tanto, se recomienda como referencia fundamental. Este capítulo se basa, y es consistente, con su contenido. Sugerencia: no rechace el libro como consecuencia del uso de iconos para los diferentes niveles de casos de uso por parte de los autores, o el énfasis temprano en los niveles y la taxonomía de casos de uso. Los iconos son opcionales y no muy importantes. Y aunque el debate acerca de los niveles y objetivos podría parecer al principio que distrae la atención a aquellos nuevos en los casos de uso, los que han trabajado con ellos durante algún tiempo estiman que el nivel y alcance de los casos de uso son cuestiones prácticas claves, porque su incompreensión es una fuente típica de complicaciones en el modelado de casos de uso.

“Structuring Use Cases with Goals” [Cockburn97] es el artículo sobre casos de uso más citado, disponible on-line en www.usecases.org.

Use Cases: Requirements in Context [GK00] es otro texto útil. Destaca el importante punto de vista —como establece el título— de que los casos de uso no son únicamente

⁸ Nótese que Cockburn rima con *slow burn* (N. del T.: En una comunicación personal, el autor nos ha comentado que introdujo esta aclaración a petición de A. Cockburn para señalar que su apellido no se pronuncia como cock-burn y evitar chistes por la connotación sexual.)

otro artefacto de los requisitos, sino que constituyen el vehículo central que dirige el trabajo de los requisitos y la información.

Otra lectura que merece la pena destacar es *Applying Use Cases: A Practical Guide* [SW98], escrita por un profesor y experto en casos de uso que entiende y comunica cómo aplicar los casos de uso en un ciclo de vida iterativo.

6.19. Artefactos UP y contexto del proceso

Como se ilustra en la Figura 6.5 los casos de uso influyen en muchos artefactos UP.

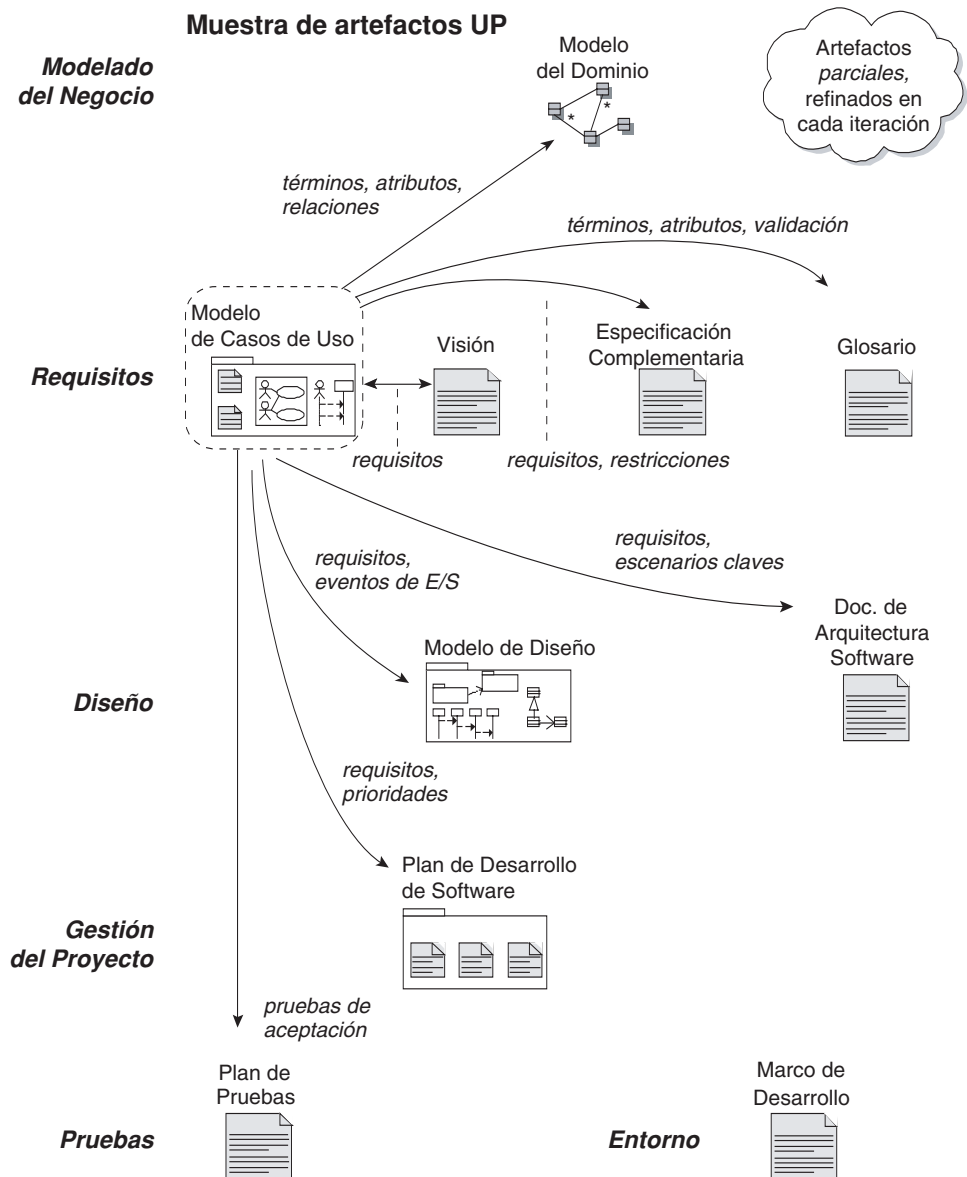


Figura 6.5. Muestra de la influencia entre los artefactos UP.

En el UP, el trabajo de los casos de uso es una actividad de la disciplina de requisitos que podría inicializarse durante un taller de requisitos. La Figura 6.6 presenta algunos consejos acerca del momento y el lugar para llevar a cabo este trabajo.

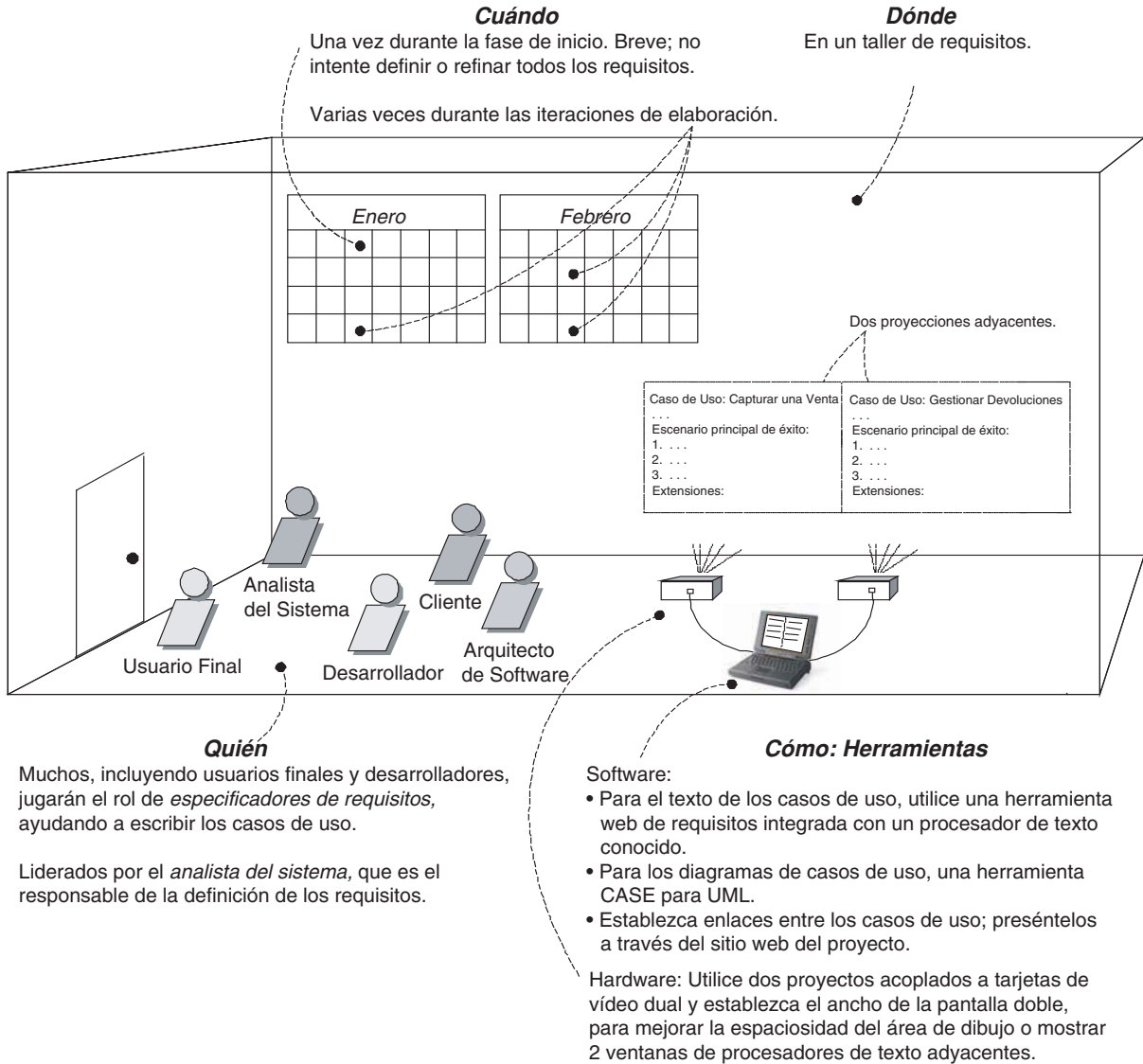


Figura 6.6. Proceso y establecimiento del contexto.

IDENTIFICACIÓN DE OTROS REQUISITOS

Cuando las ideas fallan, las palabras vienen muy bien.

Johann Wolfgang von Goethe

Objetivos

- Escribir una Especificación Complementaria, Glosario y Visión.
- Comparar y contrastar las características del sistema con los casos de uso.
- Relacionar la Visión con otros artefactos y con el desarrollo iterativo.
- Definir los atributos de calidad.

Introducción

No es suficiente escribir casos de uso. Existen otros tipos de requisitos que son necesarios identificar, como los relacionados con documentación, empaquetado, soporte, licencia, etcétera. Éstos se recogen en la **Especificación Complementaria**.

El **Glosario** almacena los términos y definiciones; puede también jugar el rol de diccionario de datos.

La **Visión** resume la “visión” del proyecto. Sirve para comunicar de manera concisa las grandes ideas acerca de por qué se propuso el proyecto, cuáles son los problemas, quiénes son las personas involucradas, qué necesitan, y cuál podría ser la apariencia de la solución propuesta.

Citando literalmente:

La Visión define la vista que tienen las personas involucradas del producto que se va a desarrollar, especificada en términos de las necesidades y características claves de dichas personas. Al contener un esquema de los principales requisitos previstos, proporciona la base contractual para los requisitos técnicos más detallados [RUP].

7.1. Ejemplos del PDV NuevaEra

El objetivo de los siguientes ejemplos no es presentar de manera exhaustiva la Visión, el Glosario y Especificación Complementaria, ya que algunas de las secciones —aunque útiles para el proyecto— no son relevantes para los objetivos de aprendizaje¹. La finalidad del libro son las principales técnicas en el diseño de objetos, análisis de requisitos con casos de uso, análisis orientado a objetos, no los problemas del PDV o las sentencias de Visión. Por tanto, sólo se hace referencia brevemente a algunas secciones, para establecer las conexiones entre el trabajo previo y futuro, destacar las cuestiones que merecen la pena, proporcionar una idea del contenido, y avanzar rápidamente.

7.2. Ejemplo NuevaEra: Especificación Complementaria (Parcial)

Especificación Complementaria

Historia de revisiones

<i>Versión</i>	<i>Fecha</i>	<i>Descripción</i>	<i>Autor</i>
Borrador de Inicio	10 Enero, 2031	Primer borrador. Para refinarse principalmente durante la elaboración.	Craig Larman

Introducción

Este documento es el repositorio de todos los requisitos del PDV NuevaEra que no se capturan en los casos de uso.

Funcionalidad

(Funcionalidad común entre muchos casos de uso)

Registro y gestión de errores

Registrar todos los errores en almacenamiento persistente.

¹ El crecimiento gradual del alcance no es sólo un problema de los requisitos, sino de la *escritura* sobre requisitos.

Reglas de negocio conectables

En varios puntos de los escenarios de varios casos de uso (pendientes de ser definidos) soportar la capacidad de adaptar la funcionalidad del sistema con un conjunto arbitrario de reglas que se ejecutan en ese punto o evento.

Seguridad

Todo uso requiere la autenticación de los usuarios.

Facilidad de uso***Factores humanos***

El cliente será capaz de ver la información en un gran monitor del PDV. Por tanto:

- Se debe ver el texto fácilmente a una distancia de 1 metro.
- Evitar colores asociados con formas comunes de daltonismo.

Velocidad, comodidad y procesamiento libre de errores, son lo más importante en el procesamiento de ventas, ya que el comprador desea irse rápidamente, o perciben la experiencia de compra (y al vendedor) como menos positiva.

El cajero está mirando a menudo al cliente o los artículos, no a la pantalla del ordenador. Por tanto, se deben comunicar las señales y avisos con sonidos, en lugar de sólo mediante gráficos.

Fiabilidad***Capacidad de recuperación***

Si se produce algún fallo al usar un servicio externo (autorización de pago, sistema de contabilidad,...) intentar solucionarlo con una solución local (ej. almacenar y remitir) para, no obstante, completar una venta. Se necesita mucho más análisis aquí...

Rendimiento

Como se mencionó en los factores humanos, los compradores quieren completar el proceso de ventas *muy* rápido. Un cuello de botella potencial es la autorización de pagos externa. El objetivo es conseguir la autorización en menos de 1 minuto, el 90% de las veces.

Soporte***Adaptabilidad***

Los diferentes clientes del PDV NuevaEra tienen necesidades de procesamiento y reglas de negocio únicas en el procesamiento de una venta. Por tanto, en varios puntos definidos en el escenario (por ejemplo, cuando se inicia una nueva venta, cuando se añade una nueva línea de venta) se habilitarán reglas de negocio conectables.

Facilidad para cambiar la configuración

Cientes diferentes desean que varíe la configuración de la red para sus sistemas PDV, como clientes gruesos *versus* delgados, en dos capas *versus* N-capas, etcétera. Además, desean poder cambiar estas configuraciones, para reflejar sus cambios en el negocio y necesidades de rendimiento. Por tanto, el sistema será configurable para reflejar estas necesidades. Se necesita mucho más análisis en este área para descubrir las áreas y grado de flexibilidad, y el esfuerzo para conseguirla.

Restricciones de Implementación

La dirección de NuevaEra insiste en una solución utilizando las tecnologías Java, previendo que esto mejorará a largo plazo la portabilidad y soporte, además de facilitar el desarrollo.

Componentes adquiridos

- El sistema de cálculo de impuestos. Debe soportar sistemas de cálculo conectables de diferentes países.

Componentes de libre distribución

En general, recomendamos maximizar el uso de componentes Java de libre distribución en este proyecto.

Aunque es prematuro diseñar y elegir los componentes de manera definitiva, sugerimos los siguientes como candidatos posibles:

- Framework para registros JLog
- ...

Interfaces

Interfaces y hardware destacables

- Monitor con pantalla táctil (detectado por el sistema operativo como monitor corriente, y los movimientos de contacto como eventos del ratón)
- Escáner láser de código de barras (normalmente unido a un teclado especial, y el software considera las entradas escaneadas como entradas del teclado)
- Impresora de recibos
- Lector de tarjetas de crédito/débito
- Lector de firmas (pero no en la primera versión)

Interfaces software

Para la mayoría de los sistema de colaboración externos (calculador de impuestos, contabilidad, inventario, ...) necesitamos ser capaces de conectar diversos sistemas y, por tanto, diversas interfaces.

Reglas de dominio (negocio)

<i>ID</i>	<i>Regla</i>	<i>Grado de variación</i>	<i>Fuente</i>
REGLA1	Se requiere la firma para pagos a crédito. años la mayoría de los clientes	Se continuará solicitando la "firma" de los compradores, aunque en 2 solicitarán la firma en un aparato de captura digital, y en 5 años esperamos que se demande la nueva "firma" digital única soportada por la ley americana.	La política de prácticamente todas las compañías de autorización de crédito.
REGLA2	Reglas sobre los impuestos. Hay que añadir un impuesto	Alto. La ley sobre impuestos cambia anualmente, en todos los niveles de gobierno.	ley

continúa

<i>ID</i>	<i>Regla</i>	<i>Grado de variación</i>	<i>Fuente</i>
	a las ventas. Ver los estatutos del gobierno para los detalles actuales.		
REGLA3	Las devoluciones de los pagos a crédito sólo pueden efectuarse como crédito en las cuentas de crédito de los compradores, no en efectivo.	Bajo.	Política de la compañía sobre la autorización de crédito.
REGLA4	Reglas de descuento al comprador. Ejemplo: Empleado: 20% Clientes preferentes: 10% Antiguos: 15%	Alto. Cada tienda utiliza reglas diferentes.	Política de la tienda.
REGLA5	Reglas de descuento de venta (nivel de transacción) Se aplica al total antes de los impuestos. Por ejemplo: 10% de descuento si el total es mayor de 100 €. 5% de descuento los lunes. 10% de descuento en las ventas de hoy entre las 10am y las 3pm. 50% de descuento en Tofu hoy desde las 9am hasta las 10am.	Alto. Cada tienda utiliza reglas distintas, y pueden cambiar diariamente o cada hora.	Política de la tienda.
REGLA6	Reglas de descuento de artículo (nivel de línea de venta). Ejemplo: 10% de descuento en tractores esta semana. Comprando 2 hamburguesas vegetales llévase 1 gratis.	Alto. Cada tienda utiliza reglas distintas, y pueden cambiar diariamente o cada hora.	Política de la tienda.

Cuestiones legales

Recomendamos algunos componentes de libre distribución si sus restricciones de licencia se pueden resolver para permitir la reventa de artículos que incluyen software de libre distribución.

Todas las reglas de impuestos se tienen que aplicar, por ley, durante las ventas. Nótese que pueden cambiar con frecuencia.

Información en dominios de interés

Fijación de precios

Además de las reglas de fijación de precios que se describen en la sección de reglas del dominio, observe que los artículos tienen un *precio original*, y opcionalmente, un *precio rebajado*. El precio de los artículos (antes de los descuentos adicionales) es el precio rebajado, si existe. Las organizaciones mantienen el precio original incluso si hay un precio rebajado, por razones de contabilidad e impuestos.

Gestión de pagos a crédito y débito

Cuando el servicio de autorización de pagos aprueba un pago electrónico, de crédito o débito, son los responsables del pago al vendedor, no el comprador. En consecuencia, por cada pago, el vendedor necesita registrar la suma de dinero que le deben en las cuentas, desde el servicio de autorización. Normalmente, por las noches, el servicio de autorización llevará a cabo una transferencia electrónica de fondos a la cuenta de los vendedores por la suma de dinero total del día, menos un (pequeño) cargo por transacción que cobra el servicio.

Impuesto de ventas

Los cálculos de los impuestos de las ventas pueden ser muy complejos, y pueden cambiar regularmente como respuesta a la legislación en todos los niveles de gobierno. Por tanto, es aconsejable delegar el cálculo a software de terceras partes (de los que hay varios disponibles). Los impuestos se pueden deber a la ciudad, al gobierno regional y agencias nacionales. Algunos artículos podrían estar exentos de impuestos sin requisitos, o exentos dependiendo del comprador o el receptor objetivo (por ejemplo, un granjero o un niño).

Identificadores de artículos: UPCs, EANs, SKUs, código de barras y lectores de códigos de barras

El PDV NuevaEra necesita dar soporte a varios esquemas de identificador de artículos. UPCs (Códigos de Producto Universales, *Universal Product Codes*), EANs (Numeración de Artículos Europea, *European Article Numbering*) y SKUs (Unidades de Mantenimiento de Stock, *Stock Keeping Units*) son tres tipos comunes de sistemas de identificación para los artículos que se venden. El Número de Artículo Japonés (JANs, *Japanese Article Number*) es un tipo de la versión EAN.

SKUs son identificadores completamente arbitrarios definidos por el vendedor.

Sin embargo, los UPCs y EANs tienen un componente estándar y normativo. Diríjase a www.adams1.com/pub/russadam/upccode.html para obtener una buena visión general. También puede ver www.uc-council.org y www.ean-int.org.

7.3. Comentario: Especificación Complementaria

La **Especificación Complementaria** captura otros requisitos, información y restricciones que no se recogen fácilmente en los casos de uso o el Glosario, que comprende los atributos o requisitos de calidad “URPS+” de todo el sistema. Fíjese que los requisitos específicos de un caso de uso pueden (y probablemente deben) escribirse en primer lugar con el caso de uso, en una sección *Requisitos Especiales*, pero algunos prefieren también reunirlos en la Especificación Complementaria. Los elementos de la Especificación Complementaria podrían comprender:

- Requisitos FURPS+ —funcionalidad, facilidad de uso, fiabilidad, rendimiento y soporte—.
- Informes.
- Restricciones de software y hardware (sistemas operativos y de red...).
- Restricciones de desarrollo (por ejemplo, herramientas de proceso y desarrollo).
- Otras restricciones de diseño e implementación.

- Cuestiones de internacionalización (unidades, idiomas...).
- Documentación (usuario, instalación, administración) y ayuda.
- Licencia y otras cuestiones legales.
- Empaquetado.
- Estándares (técnico, seguridad, calidad).
- Cuestiones del entorno físico (por ejemplo, calor o vibración).
- Cuestiones operacionales (por ejemplo, ¿cómo se gestionan los errores o con qué frecuencia se hacen las copias de seguridad?).
- Reglas del dominio o negocio.
- Información en los dominios de interés (por ejemplo, ¿cuál es el ciclo completo de gestión de pagos a crédito?).

Las **restricciones** no son comportamientos, sino otro tipo de restricciones del diseño o el proyecto. También son requisitos, pero se denominan comúnmente “restricciones” para remarcar su influencia *restrictiva*. Por ejemplo:

Debe utilizar Oracle (tenemos un contrato de licencia con ellos)

Debe funcionar bajo Linux (disminuirá el coste)

Sugerencia

Las decisiones y restricciones de diseño tempranas (“elaboración prematura”) son casi siempre una mala idea, así que merece la pena desconfiar y cuestionarlas, especialmente durante la fase de inicio, cuando se ha analizado muy poco con cuidado. Algunas restricciones se imponen por causas inevitables, como restricciones legales, o una interfaz de un sistema externo existente al que se debe invocar.

Atributos de calidad

Algunos requisitos se denominan **atributos de calidad** [BCK98] (o “-ilities”) de un sistema. Éstos incluyen facilidad de uso (*usability*), fiabilidad (*reliability*), etcétera. Nótese que se refieren a cualidades del sistema, no a que estos requisitos sean necesariamente de calidad alta (la palabra está sobrecargada en inglés). Por ejemplo, la cualidad de soporte (*supportability*) podría elegirse deliberadamente que fuese baja si el producto no se pretende que sirva a largo plazo.

Hay de dos tipos:

1. Observables en la ejecución (funcionalidad, facilidad de uso, fiabilidad, rendimiento, etc.).
2. No observables en ejecución (soporte, pruebas, etc.).

La funcionalidad se especifica en los casos de uso, como otros atributos de calidad relacionados con casos de uso específicos (por ejemplo, las cualidades de rendimiento en el caso de uso *Procesar Venta*).

Otros atributos de calidad FURPS+ del sistema se describen en la Especificación Complementaria.

Aunque la funcionalidad es un atributo de calidad válido, en su uso común, el término “atributo de calidad” casi siempre se refiere a “cualidades del sistema que no sean la funcionalidad”. En este libro, el término se utiliza de este modo. Esto no es exactamente lo mismo que los requisitos no funcionales, que es un término más amplio que incluye *todo* excepto la funcionalidad (por ejemplo, empaquetado y licencia).

Cuando nos ponemos nuestro “gorro de arquitecto”, los atributos de calidad de todo el sistema (y por tanto, la Especificación Complementaria donde se recogen) son especialmente interesantes porque —como veremos en el Capítulo 32— el análisis y el diseño de la arquitectura tienen que ver en gran medida con la identificación y resolución de los atributos de calidad en el contexto de los requisitos funcionales. Por ejemplo, suponga que uno de los atributos de calidad es que el sistema NuevaEra debe ser bastante tolerante a fallos cuando fallen los servicios remotos. Desde el punto de vista de la arquitectura, esto tendrá una influencia global en las decisiones de diseño a gran escala.

Entre los atributos de calidad existen interdependencias, lo que implica compromisos. Un ejemplo sencillo en el PDV podría ser, “muy fiable (tolerante a fallos)” y “fácil de probar” son opuestos, puesto que hay muchas formas sutiles de que pueda fallar un sistema distribuido.

Reglas del dominio (negocio)

Las reglas del dominio [Ross97, GK00] dictan el modo en el que podría operar un dominio o negocio. No son requisitos de ninguna aplicación, aunque, a menudo, los requisitos de una aplicación se ven afectados por las reglas del dominio. Las políticas de la compañía, leyes físicas y leyes gubernamentales, son reglas de dominio típicas.

Se denominan comúnmente **reglas del negocio**, que son el tipo más común, pero ese término está limitado, ya que existen aplicaciones software que no son de gestión de un negocio, como simulación del clima o logística militar. Una simulación del clima incluye “reglas del dominio”, relacionadas con las leyes y relaciones físicas, que afectan a los requisitos de la aplicación.

Con frecuencia, resulta útil identificar y registrar aquellas reglas del dominio que afectan a los requisitos, normalmente materializados en los casos de uso, porque pueden clarificar el contenido de un caso de uso ambiguo o incompleto. Por ejemplo, en el PDV NuevaEra, si alguien pregunta si en el caso de uso *Procesar Venta* debería escribirse una alternativa a los pagos a crédito que no necesite la captura de la firma, existe una regla del negocio (REGLA1) que aclara que no se permitirá por ninguna compañía de autorización de crédito.

Advertencia

Las reglas no son requisitos de la aplicación. No registre las características del sistema como reglas. Las reglas describen las restricciones y comportamientos del modo de trabajar del dominio, no de la aplicación.

Información en los dominios de interés

A menudo, a los expertos del dominio que se está estudiando, les resulta útil escribir (o proporcionar URLs con) explicaciones de dominios relacionados con el nuevo sistema software (ventas y contabilidad, la geofísica de los flujos subterráneos de petróleo/agua/gas,...), para presentar el contexto y ayudar a la comprensión por parte del equipo de desarrollo. Podría contener referencias importantes a expertos o a literatura, fórmulas, leyes u otras referencias. Por ejemplo, los misterios de los esquemas de codificación UPC y EAN, y la interpretación del código de barras, los deben entender, en cierta medida, el equipo de NuevaEra.

7.4. Ejemplo NuevaEra: Visión (Parcial)

Visión

Historia de revisiones

<i>Versión</i>	<i>Fecha</i>	<i>Descripción</i>	<i>Autor</i>
Borrador de Inicio	10 Enero, 2031	Primer borrador. Para refinarse principalmente durante la elaboración.	Craig Larman

El análisis del ejemplo es ilustrativo, pero ficticio

Introducción

Prevedemos una aplicación de punto de venta (PDV) tolerante a fallos de próxima generación, PDV NuevaEra, con flexibilidad para poder soportar variación en las reglas del negocio del cliente, múltiples mecanismos de terminal e interfaz de usuario, y la integración con múltiples sistemas de terceras partes.

Orientación

Oportunidad del negocio

Los productos PDV existentes no son adaptables al negocio del cliente, en términos de permitir variar las reglas de negocio y los diseños de la red (por ejemplo cliente delgado o no, arquitectura en 2, 3 ó 4 capas). Además, no permiten su extensión de manera adecuada cuando se incrementan los terminales y crece el negocio. Y ninguno permite trabajar en línea o desconectados, adaptándose dinámicamente dependiendo de los fallos. Ninguno se integra fácilmente con muchos sistemas de terceras partes. Ninguno admite nuevas tecnologías como los PDAs móviles. El mercado se siente insatisfecho debido a este estado inflexible de cosas, y demandan un PDV que rectifique esta situación.

Enunciado del problema

Los sistemas PDV tradicionales son inflexibles, intolerantes a fallos, y difíciles de integrar con sistemas de terceras partes. Esto da lugar a problemas en el oportuno procesamiento de las ventas, en el establecimiento de procesos mejorados que no concuerdan con el software, y con los datos de contabilidad e inventario precisos y oportunos para dar soporte a la planificación y medidas, entre otras cuestiones. Esto afecta a los cajeros, encargados del almacén, administradores del sistema y a la gestión empresarial.

Enunciado de la posición en el mercado del producto

- Resumen conciso de a quién está dirigido el producto, las características relevantes, y qué lo diferencia de la competencia.

Alternativas y competencia...

Entender
quiénes son los
participantes y
sus problemas

Descripción del personal involucrado**Demografía de mercado...****Resumen del personal involucrado (No usuarios)...****Resumen de Usuarios...****Objetivos de alto nivel y problemas claves del personal involucrado**

El taller de requisitos de un día con los expertos en la materia que se está estudiando y otras personas involucradas, y encuestas a varios distribuidores, nos llevaron a los siguientes objetivos y problemas claves:

Reunir las
entradas de la
Lista Actor-
Objetivos y la
sección de los
casos de uso
de Intereses
del personal
involucrado

Objetivo de alto nivel	Prioridad	Problemas e inquietudes	Soluciones actuales
Procesamiento de ventas rápido, robusto e integrado.	Alta	Se reduce la velocidad cuando se incrementa la carga. Pérdida de la capacidad de procesamiento de las ventas si los componentes fallan. Carencia de información actualizada y precisa de la contabilidad y otros sistemas debido a que no está integrado con los sistemas de contabilidad, inventario y RRHH. Da lugar a dificultades en las medidas y la planificación. Imposibilidad de adaptar las reglas de negocio a requisitos del negocio únicos. Dificultad al añadir nuevos tipos de terminales o interfaces de usuario (por ejemplo, PDAs móviles).	Los productos PDV existentes permiten el procesamiento básico de ventas, pero no abordan estos problemas.
...	...	...	...

Objetivos de nivel de usuario

Los usuarios (y los sistemas externos) necesitan un sistema para satisfacer sus objetivos:

- *Cajero*: procesar las ventas, gestionar las devoluciones, abrir y cerrar caja.
- *Administrador del sistema*: gestionar los usuarios, gestionar la seguridad, gestionar las tablas del sistema.
- *Director*: poner en marcha, suspender operación.
- *Sistema de actividad de ventas*: analizar los datos de las ventas.
- ...

Podría ser la
Lista Actor-
Objetivo creada
durante el
modelado de
casos de uso o
un resumen
más conciso

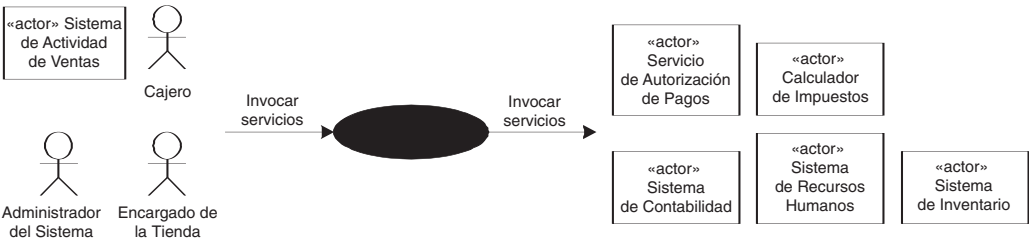
Entorno de usuario...

Visión general del producto

Perspectiva del producto

El PDV NuevaEra residirá, normalmente, en tiendas; si se utilizan terminales móviles se encontrarán muy próximos a la red de la tienda, en el interior o en el exterior. Proporcionará servicios al usuario, y colaborará con otros sistemas, como se indica en la Figura Visión-1.

Resumido a partir del diagrama de casos de uso



Los diagramas aparecen con diversos detalles, pero todos muestran los actores externos importantes para el sistema

Figura Visión-1. Diagrama de contexto del sistema PDV NuevaEra.

Resumen de los beneficios

Característica soportada	Beneficio del personal involucrado
Funcionalmente, el sistema proporcionará todos los servicios típicos que requiere la organización de las ventas, incluyendo la entrada de las ventas, autorización de pagos, gestión de devoluciones, etcétera.	Servicios de punto de venta rápidos y automáticos.
Detección de fallos automática, cambiando a procesamiento local sin conexión cuando los servicios no estén disponibles.	Procesamiento de ventas continuado cuando fallan los componentes externos.
Reglas de negocio “conectables” en varios puntos del escenario durante el procesamiento de las ventas.	Configuración flexible de la lógica del negocio.
Transacciones en tiempo real con sistemas de terceras partes, haciendo uso de protocolos estándares industriales.	Información de ventas, contabilidad e inventario oportuna y precisa, para abordar las mediciones y la planificación.
...	...

Similar a la lista Actor-Objetivo, esta tabla relaciona objetivos, beneficios y soluciones, pero a un nivel más alto, no únicamente relacionado con los casos de uso

Resume el valor y características diferenciadoras del producto

Suposiciones y dependencias...

Coste y fijación del precio...

Licencia e instalación...

Resumen de las características del sistema

- Entrada de ventas.
- Autorización de pagos (crédito, débito, cheque).

Como se presenta abajo, las características del sistema constituyen un formato conciso para resumir la funcionalidad

- Administración del sistema de usuarios, seguridad, código y tablas de constantes, etcétera.
- Procesamiento automático de ventas sin conexión cuando fallen los componentes externos.
- Transacciones en tiempo real, basadas en estándares industriales, con sistemas de terceras partes, que incluye los servicios de inventario, contabilidad, recursos humanos, cálculo de impuestos, y autorización de pagos.
- Definición y ejecución de reglas del negocio “conectables” en puntos comunes y fijos de los escenarios de procesamiento.
- ...

Otros requisitos y restricciones

Abarca las restricciones de diseño, facilidad de uso, fiabilidad, rendimiento, soporte, empaquetado, documentación, etcétera: Diríjase a la Especificación Complementaria y a los casos de uso.

7.5. Comentario: Visión

¿Estamos solucionando el mismo problema? ¿El problema correcto?

El enunciado del problema

Durante el trabajo de requisitos en la fase de inicio, hay que colaborar para definir un enunciado del problema conciso; reducirá la posibilidad de que el personal involucrado esté intentando solucionar problemas ligeramente diferentes, y normalmente, se crea rápido. Algunas veces, este trabajo revela diferencias de opinión fundamentales en lo que las partes están tratando de conseguir.

Una alternativa al simple texto es el formato de tabla propuesto en la plantilla del RUP para el enunciado del problema:

El problema de	...
afecta	...
el impacto de lo cual es	...
una solución de éxito sería	...

Los objetivos de alto nivel y problemas claves del personal involucrado

Esta tabla resume los objetivos y problemas a un nivel más alto que los casos de uso de nivel de tarea, y muestra importantes objetivos no funcionales y de calidad que podrían pertenecer a un caso de uso o abarcar a muchos, tales como:

- *Necesitamos procesamiento de ventas tolerante a fallos.*
- *Necesitamos la capacidad de adaptar las reglas del negocio.*

¿Cuáles son los problemas y objetivos esenciales?

Es típico que el personal involucrado exprese sus objetivos en base a soluciones que imaginan, como “Necesitamos un programador a tiempo completo que adapte las reglas del negocio cuando las cambiemos”. Las soluciones algunas veces son inteligentes, porque comprenden bien el dominio del problema y las opciones. Pero, algunas veces, el personal involucrado llega a soluciones que no son las más apropiadas o que no tratan el problema subyacente más importante y esencial.

Por tanto, el analista del sistema, necesita estudiar la cadena de problemas y objetivos —como se presentó en el capítulo anterior sobre los objetivos y casos de uso— para aprender los problemas subyacentes, y su importancia e impacto relativos, y de este modo priorizar y solucionar las cuestiones más relevantes con una solución ingeniosa.

Métodos para facilitar la idea del grupo

Aunque queda fuera del alcance de esta discusión, especialmente durante actividades tales como definición de problemas de alto nivel e identificación de objetivos, tiene lugar el trabajo de investigación creativo en grupo. Presentamos algunas técnicas útiles de ayuda al grupo para el descubrimiento de los problemas y objetivos esenciales y para ayudar a la generación de ideas y su priorización: mapas mentales, diagramas causa-efecto (*fish-bone*), diagramas pareto, “tormenta de ideas” (*brainstorming*), multivotación, votación por puntos (*dot voting*), técnica de grupo nominal, “escribir ideas en silencio” (*brainwriting*) y diagrama de afinidad. Estúdielos en la web. Prefiero aplicar varios de ellos durante el mismo taller, para descubrir problemas típicos y requisitos desde distintos ángulos.

Características del sistema—requisitos funcionales

Los casos de uso no son necesariamente el único modo para expresar los requisitos funcionales, por los siguientes motivos:

- Son detallados. A menudo, el personal involucrado quiere un resumen breve que identifique las funciones más destacables.
- ¿Y si listamos simplemente los nombres de los casos de uso (*Procesar Venta*, *Gestionar Devoluciones*,...) para resumir la funcionalidad? Primero, la lista podría ser todavía demasiado extensa. Además, los nombres pueden ocultar funcionalidad interesante que el personal involucrado realmente quiere conocer; esto es, el nivel de granularidad puede oscurecer funciones importantes. Por ejemplo, suponga que la descripción de la funcionalidad de la autorización automática de pago está contenida en el caso de uso *Procesar Venta*. Un lector de la lista de los nombres de los casos de uso no puede decir si el sistema realizará autorizaciones de pagos. Es más, uno podría desear agrupar un conjunto de casos de uso en una característica (por simplicidad), como *Administración del sistema para usuarios, seguridad, código y tablas de constantes, etcétera*.

- Algunas funcionalidades importantes, como es lógico, se expresan como sentencias cortas que no se corresponden de manera conveniente con los nombres de los casos de uso u objetivos de nivel de Procesos del Negocio Elementales (EBP). Podrían abarcar o ser ortogonales a los casos de uso. Por ejemplo, durante el primer taller de requisitos de NuevaEra, algunos podrían decir “el sistema debería ser capaz de llevar a cabo transacciones con sistemas de terceras partes de contabilidad, inventario y cálculo de impuestos”. Esta sentencia sobre la funcionalidad no representa ningún caso de uso en particular, pero es una manera cómoda y concisa para expresar, recoger y comunicar las características.
- Una variación más drástica del último punto es la siguiente. Algunas aplicaciones requieren ante todo una descripción de la funcionalidad como características; los casos de uso no se ajustan de manera natural. Esto es común, por ejemplo, en productos *middleware*, como los servidores de aplicaciones —los casos de uso no están motivados realmente—. Suponga que el equipo está considerando la próxima versión. Durante un debate de los requisitos, la gente (como los de marketing) dirá: “La siguiente versión necesita soportar *bean* entidad EJB2.0”. Los requisitos se conciben inicialmente en términos de una lista de características, no de casos de uso.

Por tanto, un modo alternativo, complementario para expresar las funciones del sistema, es mediante las **características**, o más concretamente en este contexto, **características del sistema**, que son sentencias concisas, de alto nivel, que resumen las funciones del sistema. Más formalmente, en el UP, un **característica del sistema** es “un servicio observable externamente proporcionado por el sistema que cumple directamente una necesidad del personal involucrado” [Kruchten00].

Las características son cosas que un sistema puede *hacer*. Deberían pasar este test lingüístico:
El sistema deberá hacer <característicaX>.

Por ejemplo:

El sistema deberá hacer la autorización del pago.

Recordemos que la Visión podría utilizarse como contrato formal o informal entre los desarrolladores y la empresa. Las características del sistema constituyen un mecanismo para resumir, en este contrato, lo que el sistema *hará*. Esto es complementario a los casos de uso, ya que las características son concisas.

Las características tienen que contrastarse con varios tipos de requisitos no funcionales y restricciones, por ejemplo: “*El sistema debe ejecutarse bajo Linux, debe tener disponibilidad de 24/7 y debe tener una interfaz con pantalla táctil*”. Nótese que falla el test lingüístico.

A veces, la propiedad de que sea “un servicio observable externamente...” es difícil de resolver. Por ejemplo, ¿debería ser lo siguiente una característica del sistema?:

El sistema hará las transacciones con sistemas de terceras partes de contabilidad, inventario, recursos humanos y cálculo de impuestos.

Se trata de un tipo de comportamiento, y probablemente de interés para el personal involucrado, pero la colaboración en sí misma podría no ser visible externamente, dependiendo del margen de tiempo, lo cerca que se esté y de dónde se mire. Téngalo en cuenta —rara vez merece la pena que se preocupe por cuestiones de clasificación de poca granularidad—.

Por último, podemos observar que la mayoría de las características del sistema se expresarán de manera detallada en el texto de los casos de uso.

Notación y organización

Ante todo, son importantes las descripciones breves de alto nivel. Uno debería ser capaz de leer la lista de características del sistema rápidamente.

No es necesario incluir la frase canónica “El sistema deberá hacer...” o una frase variante, aunque es lo normal.

A continuación, presentamos un ejemplo de características de alto nivel, para un proyecto multi-sistema amplio, del cual el PDV es sólo un elemento:

Las características principales incluyen:

- *Servicios PDV.*
- *Gestión de inventario.*
- *Compras basadas en la web.*
- ...

Es habitual organizar una jerarquía de dos niveles de características del sistema. Pero en el documento de Visión, más de dos niveles nos llevaría a un detalle excesivo; el sentido de las características del sistema en la Visión es resumir la funcionalidad, no descomponerla en una larga lista de elementos de grano fino. Un ejemplo razonable en cuanto a detalle:

Las características principales incluyen:

- *Servicios PDV:*
 - *Capturar ventas.*
 - *Autorización de pago.*
 - ...
- *Gestión de inventario:*
 - *Reordenación automática.*
 - ...

Algunas veces, estas características de segundo nivel son básicamente equivalentes a los nombres de los casos de uso (u objetivos de nivel de usuario), pero no es imprescindible; las características son un modo alternativo para resumir la funcionalidad. No obstante, la mayoría de las características del sistema se expresarán de manera detallada en el texto de los casos de uso.

¿Cuántas características del sistema debería contener la Visión?

Sugerencia

Es deseable un documento de Visión con menos de 50 características. Si hay más, considere la posibilidad de agrupar y abstraer las características.

Otros requisitos en la Visión

En la Visión, las características del sistema resumen brevemente los requisitos funcionales, que se expresan en detalle en los casos de uso. De igual modo, la Visión *puede* resumir otros requisitos (por ejemplo, fiabilidad y facilidad de uso) que se detallan en la sección de *Requisitos Especiales* de los casos de uso, y en la Especificación Complementaria (SS, *Supplementary Specification*). Sin embargo, hay peligro de duplicaciones inútiles. Por ejemplo, el producto RUP proporciona plantillas para la Visión y SS que contienen secciones idénticas o similares para otros requisitos como facilidad de uso, fiabilidad, rendimiento, etcétera. Tal duplicación es inevitablemente difícil de mantener. Además, el nivel de detalle de las secciones similares (por ejemplo, rendimiento) en la Visión y SS, necesita que sea bastante parecido para que tenga sentido; es decir, las descripciones de otros requisitos “detalladas” y “esenciales”, tienden a ser prácticamente iguales.

Sugerencia

Para otros requisitos, evite su duplicación incluyéndolo de forma idéntica o casi idéntica, tanto en la Visión como en la Especificación Complementaria (SS) —y en los casos de uso—. Más bien, recójalos sólo en la SS o los casos de uso (si son específicos de los casos de uso). En la Visión dirija al lector a las secciones de éstos donde puede encontrar estos otros requisitos.

Visión, características o casos de uso—¿qué va primero?

No es útil ser estricto en el orden de algunos artefactos. Durante la colaboración para crear diferentes artefactos de requisitos, se produce una sinergia en la que el trabajo con uno influye y ayuda a clarificar los otros. Sin embargo, la secuencia recomendada es:

1. Escribir un primer borrador breve de la Visión.
2. Identificar los objetivos de usuario y los casos de uso de apoyo.
3. Escribir algunos casos de uso y comenzar la Especificación Complementaria.
4. Refinar la Visión, resumiendo la información a partir de éstos.

7.6. Ejemplo NuevaEra: un Glosario (Parcial)

Glosario

Historia de revisiones

Versión	Fecha	Descripción	Autor
Borrador de Inicio	10 Enero, 2031	Primer borrador. Para refinarse principalmente durante la elaboración.	Craig Larman

Definiciones

<i>Término</i>	<i>Definición e Información</i>	<i>Alias</i>
artículo	Un artículo o servicio en venta.	
autorización de pago	Validación llevada a cabo por un servicio externo de autorización de pago, que hará o garantizará el pago al vendedor.	
solicitud de autorización de pago	Un compuesto de elementos enviados electrónicamente a un servicio de autorización, normalmente como un array de caracteres. Los elementos comprenden: ID de la tienda, número de cuenta del cliente, cantidad y fecha.	
UPC	Código de 12 dígitos que identifica un artículo. Normalmente se representa mediante un código de barras en los artículos. Diríjase a http://www.uc-council.org para ver más detalles.	Código de Producto Universal
...	...	...

7.7. Comentario: Glosario (Diccionario de Datos)

En su forma más simple, el **Glosario** es una lista de los términos relevantes y sus definiciones. Es sorprendentemente habitual que un término, frecuentemente técnico o propio del dominio, se utilice de forma ligeramente distinta por diferentes personas involucradas; esto tiene que resolverse para reducir los problemas de comunicación y los requisitos ambiguos.

Sugerencia

Comience el Glosario cuanto antes. Recuerdo una experiencia trabajando con expertos en simulación, en la que descubrimos que la aparentemente inocua, pero importante, palabra “celda” tenía significados diversos y escurridizos entre los miembros del grupo.

El objetivo no es recoger todos los posibles términos, sino aquellos que no están claros, son ambiguos o que requieren algún tipo de elaboración relevante, como el formato de la información o las reglas de validación.

El Glosario como diccionario de datos

En el UP, el Glosario también juega el rol de **diccionario de datos**, un documento que recoge los datos sobre los datos, es decir, **metadatos**. Durante la fase de inicio, el glosario debe ser un documento sencillo de términos y descripciones. Durante la elaboración, podría ampliarse a un diccionario de datos.

Los atributos de los términos podrían contener:

- Alias.
- Descripción.
- Formato (tipo, longitud, unidad).
- Relaciones con otros elementos.

- Rango de valores.
- Reglas de validación.

Observe que el rango de valores y las reglas de validación del Glosario constituyen requisitos con implicaciones en el comportamiento del sistema.

Unidades

Como subraya Martin Fowler en *Analysis Patterns* [Fowler96], se deben tener en cuenta las unidades (moneda, medidas,...), especialmente en esta era de aplicaciones software internacionalizadas. Por ejemplo, en el sistema NuevaEra, que, con un poco de suerte, se venderá a muchos clientes en diferentes países, el *precio* no puede ser sólo un simple número. Debe estar en una unidad de *Dinero* o *Moneda* que permita la variación de monedas.

Términos compuestos

El Glosario no está destinado sólo a términos atómicos como el “precio del artículo”. Puede y debe incluir términos compuestos, como “venta” (que incluye otros elementos, tales como la fecha y la ubicación), y alias utilizados para describir una colección de datos que se transmiten entre los actores en los casos de uso. Por ejemplo, en el caso de uso *Procesar Venta*, considere la siguiente sentencia:

El Sistema envía una *solicitud de autorización de pago* a un Servicio externo de Autorización de Pago, y solicita la aprobación del pago.

La “solicitud de autorización del pago” es un alias para un agregado de datos, que tiene que explicarse en el Glosario.

7.8. Especificaciones fiables: ¿Un Oxímoron²?

Los requisitos escritos pueden dar la impresión de que se han entendido los requisitos reales y están bien definidos, y pueden utilizarse (pronto) para estimar y planificar el proyecto de manera fiable. Esta impresión es más intensa en los desarrolladores no programadores; los programadores conocen a partir de experiencias dolorosas lo poco fiable que es. Esto justifica en parte de la motivación de la cita de Goethe del comienzo del capítulo.

Lo que realmente importa es construir software que pase las pruebas de aceptación definidas por los usuarios y el resto de personas implicadas en el proyecto, y que satisfaga sus verdaderos objetivos (que, a menudo, no se descubren hasta que están evaluando o trabajando con el software).

² *N. del T.*: Figura en la que el adjetivo que califica un nombre contradice la esencia del nombre que modifica, como en “nieve ardiente”.

Es importante escribir como ejercicio la Visión y Especificación Complementaria para poner en claro una primera aproximación de lo que se quiere, la motivación del producto y como repositorio para las grandes ideas. Pero no son —ni ningún artefacto de requisitos— una especificación fiable. Sólo escribiendo código, probándolo, obteniendo retroalimentación, manteniendo una estrecha colaboración con los usuarios y los clientes y adaptando, se da realmente en el blanco.

Esto no es una llamada a abandonar el análisis y la reflexión e ir directamente a escribir código, sino una sugerencia para tratar los requisitos escritos de una manera ligera, y continuamente involucrar a los usuarios —de hecho, debía ser diariamente—.

7.9. Artefactos disponibles en el sitio web del proyecto

Puesto que esto es un libro, estos ejemplos y los anteriores casos de uso tienen un aspecto estático y, quizás orientado al papel. Sin embargo, deberían ser artefactos digitales recogidos sólo on-line a través del sitio web del proyecto. Y, en lugar de ser documentos estáticos planos, podrían contener hipervínculos, o guardarse en herramientas distintas a un procesador de texto u hoja de cálculo. Por ejemplo, el Glosario podría almacenarse en una tabla de una base de datos.

7.10. ¿Poco UML durante la fase de inicio?

El objetivo de la fase de inicio es recopilar sólo la información suficiente para establecer una visión común, decidir si es viable avanzar y si merece la pena una investigación seria del proyecto en la fase de elaboración. Como tal, a menudo, no son necesarios más diagramas que los simples diagramas de casos de uso UML. Durante la fase de inicio se hace hincapié en entender el alcance básico y el 10% de los requisitos, expresados textualmente. En la práctica y, por tanto, en esta presentación, la mayoría de los diagramas UML se realizarán en la siguiente fase —elaboración—.

7.11. Otros artefactos de requisitos en el UP

Como en el anterior capítulo de casos de uso, la Tabla 7.1 resume una muestra de los artefactos y su evolución temporal. Todos los artefactos de requisitos se comienzan en la fase de inicio, se trabaja sobre ellos sobre todo durante la elaboración.

Inicio

No debería ser el caso que estos artefactos de requisitos se terminen en la fase de inicio. De hecho, apenas se habrán empezado.

El personal involucrado necesita decidir si merece la pena llevar a cabo una investigación seria en el proyecto; esta investigación tiene lugar durante la elaboración, no la fase de inicio. Durante la fase de inicio, la Visión resume la idea del proyecto de manera que ayude, a los que tiene que tomar la decisión, a determinar si merece la pena continuar, y dónde comenzar.

Tabla 7.1. Muestra de los artefactos UP y evolución temporal. c – comenzar; r – refinar.

<i>Disciplina</i>	<i>Artefacto</i> <i>Iteración →</i>	<i>Inicio</i> <i>I1</i>	<i>Elab.</i> <i>E1...En</i>	<i>Const.</i> <i>C1...Cn</i>	<i>Trans.</i> <i>T1...T2</i>
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c		
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Puesto que la mayoría del trabajo de requisitos ocurre durante la elaboración, la Especificación Complementaria debería desarrollarse ligeramente durante la fase de inicio, resaltando los atributos de calidad relevantes (por ejemplo, el PDV NuevaEra debe recuperarse cuando fallen los servicios externos) que muestren mayores riesgos y retos.

La información de entrada a estos artefactos podría generarse durante un taller de requisitos de la fase de inicio, tanto mediante la consideración explícita de estas cuestiones, como indirectamente, por medio del análisis de casos de uso. En el taller no se escribirá un borrador legible de los artefactos, sino que lo elaborará después el analista del sistema.

Elaboración

A través de las iteraciones de la elaboración, se refina la “visión” y la Visión, en base a la retroalimentación a partir de la construcción incremental de partes del sistema, la adaptación, y varios talleres de requisitos a lo largo de varias iteraciones de desarrollo.

Mediante la investigación de requisitos continuada y el desarrollo iterativo, los otros requisitos llegarán a verse más claros y se recogerán en la SS. Los atributos de calidad (por ejemplo, fiabilidad) identificados en la SS, serán claves para dar forma al núcleo de la arquitectura que se diseña y programa durante la elaboración. También pueden ser factores de riesgo claves que influyen en lo que se va a trabajar en las primeras iteraciones. Por ejemplo, el requisito de calidad del PDV NuevaEra de recuperación en el lado del cliente si fallan los componentes externos, se explorará durante la elaboración.

La mayoría de los términos se descubren y elaboran en el Glosario durante esta fase.

Llegando al final de la elaboración, es viable contar con casos de uso, una Especificación Complementaria y un documento de Visión que, de manera razonable, reflejan las características estabilizadas más importantes y otros requisitos que se deben completar para entregar el producto. Sin embargo, la Especificación Complementaria y la

Visión no se congelan o “se dan por concluidas” como especificaciones fijas; la adaptación —no la rigidez— es un valor central del desarrollo iterativo y el UP.

Aclaremos el comentario “dar por concluido y congelar”: es perfectamente razonable —al final de la elaboración— llegar a un acuerdo con el personal involucrado sobre lo que se hará en lo que queda del proyecto, y comprometerse (quizás contractualmente) considerando los requisitos y la planificación. En algunos puntos (el final de la elaboración, en el UP), necesitamos una idea fiable sobre “qué, cuánto y cómo”. En este sentido, es normal, y se espera, un acuerdo formal sobre los requisitos. También es necesario contar con un proceso de control de cambios (una de las mejores prácticas explícitas del UP) de forma que se tienen en cuenta y se aprueban, formalmente, los cambios en los requisitos, en lugar de cambios caóticos e incontrolados.

Más bien, el comentario “dar por concluido y congelar” implica varias ideas:

- En el desarrollo iterativo y el UP, se entiende que no importa cuanta atención merecida se presta a la especificación de requisitos, algún cambio es inevitable, y se debería aceptar. Este cambio podría ser una mejora oportunista que incorpora una novedad importante en el sistema que confiere a sus propietarios ventaja competitiva, o un cambio debido a una visión mejorada.
- En el desarrollo iterativo, es un valor central contar con un compromiso continuo por parte del personal involucrado para evaluar, proporcionar retroalimentación, y dirigir el proyecto como realmente quieran. No beneficia al personal involucrado el “lavarse las manos” de un compromiso solícito, dando por concluido un conjunto de requisitos congelados y esperando el producto final, porque raramente obtendrán lo que realmente necesitan.

Construcción

En la construcción, los requisitos importantes —tanto funcionales como de otro tipo— deberían estar estabilizados —no terminados—, aunque dispuestos a cambios menores. Por tanto, no es probable que la SS y la Visión experimenten muchos cambios en esta fase.

7.12. Lecturas adicionales

Los documentos como el de Visión y Especificaciones Complementarias no son nuevos. Se utilizan en muchos proyectos y se describen en muchos libros de requisitos. Muchos de estos libros asumen de manera implícita la actitud del proceso en cascada, en el que el objetivo es obtener los requisitos detallados y correctos al principio, y llegar a un compromiso, antes de pasar al diseño y la implementación. En este sentido, la descripción tradicional que presentan no es útil aunque, por otra parte, proporcionan buenos consejos sobre posibles secciones y sus contenidos.

La mayoría de los libros sobre arquitectura software incluyen una discusión sobre el análisis de requisitos para los requisitos de calidad de la aplicación, puesto que estos requisitos de calidad tienden a influenciar fuertemente el diseño de la arquitectura. Un ejemplo es *Software Architecture in Practice* [BCK98].

Las reglas de negocio se tratan de manera exhaustiva en *The Business Rule Book* [Ross97]. El libro presenta una teoría amplia, profunda y muy bien desarrollada de las reglas del negocio, pero el método no se conecta de manera adecuada con otras técnicas de requisitos modernas, como los casos de uso o el desarrollo iterativo.

7.13. Artefactos UP y contexto del proceso

La Figura 7.1 muestra la influencia entre los artefactos, destacando los documentos de Visión, Especificación Complementaria y el Glosario.

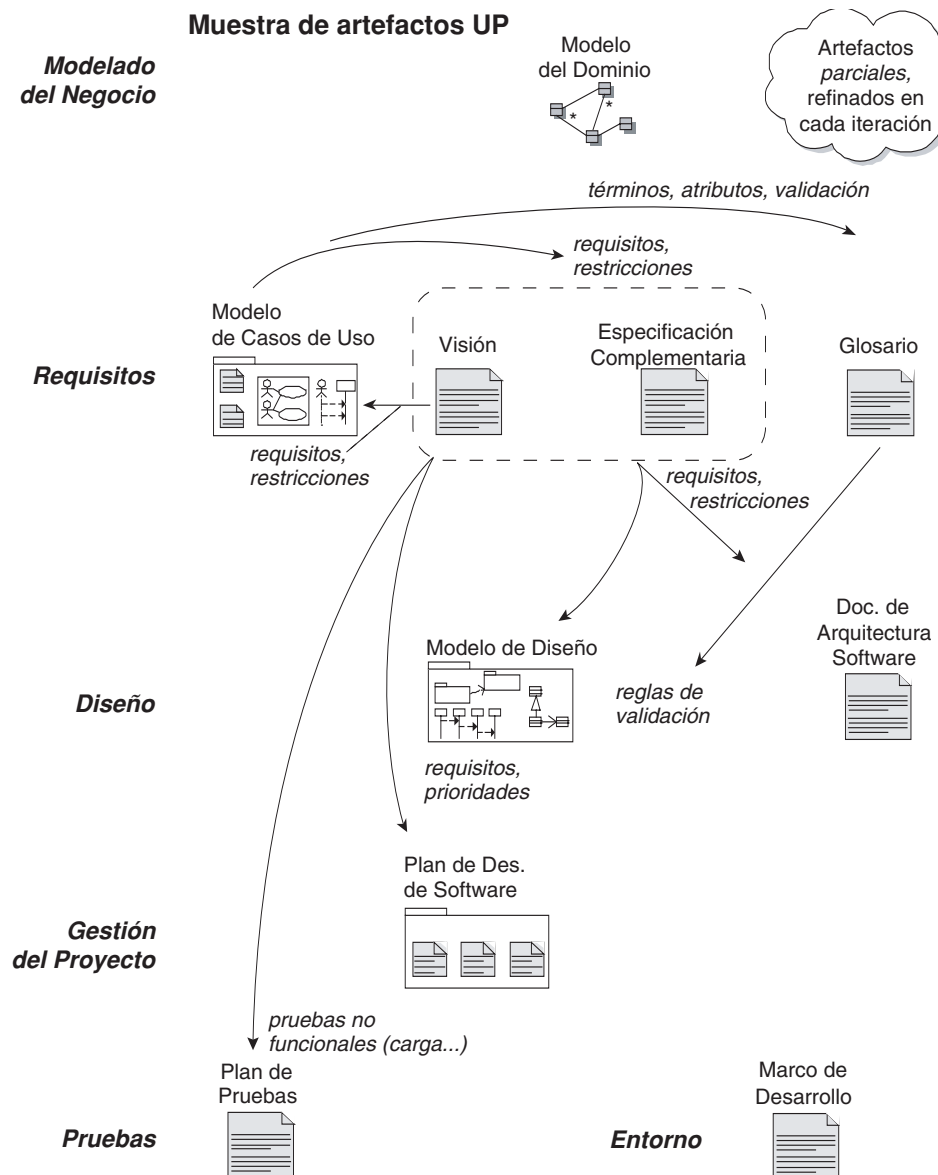


Figura 7.1. Muestra de la influencia entre los artefactos UP.

En el UP, el trabajo de Visión y Especificación Complementaria es una actividad de la disciplina de requisitos que podría iniciarse durante un taller de requisitos, junto con el análisis de casos de uso. La Figura 7.2 proporciona algunos consejos sobre el momento y el lugar para llevar a cabo este trabajo.

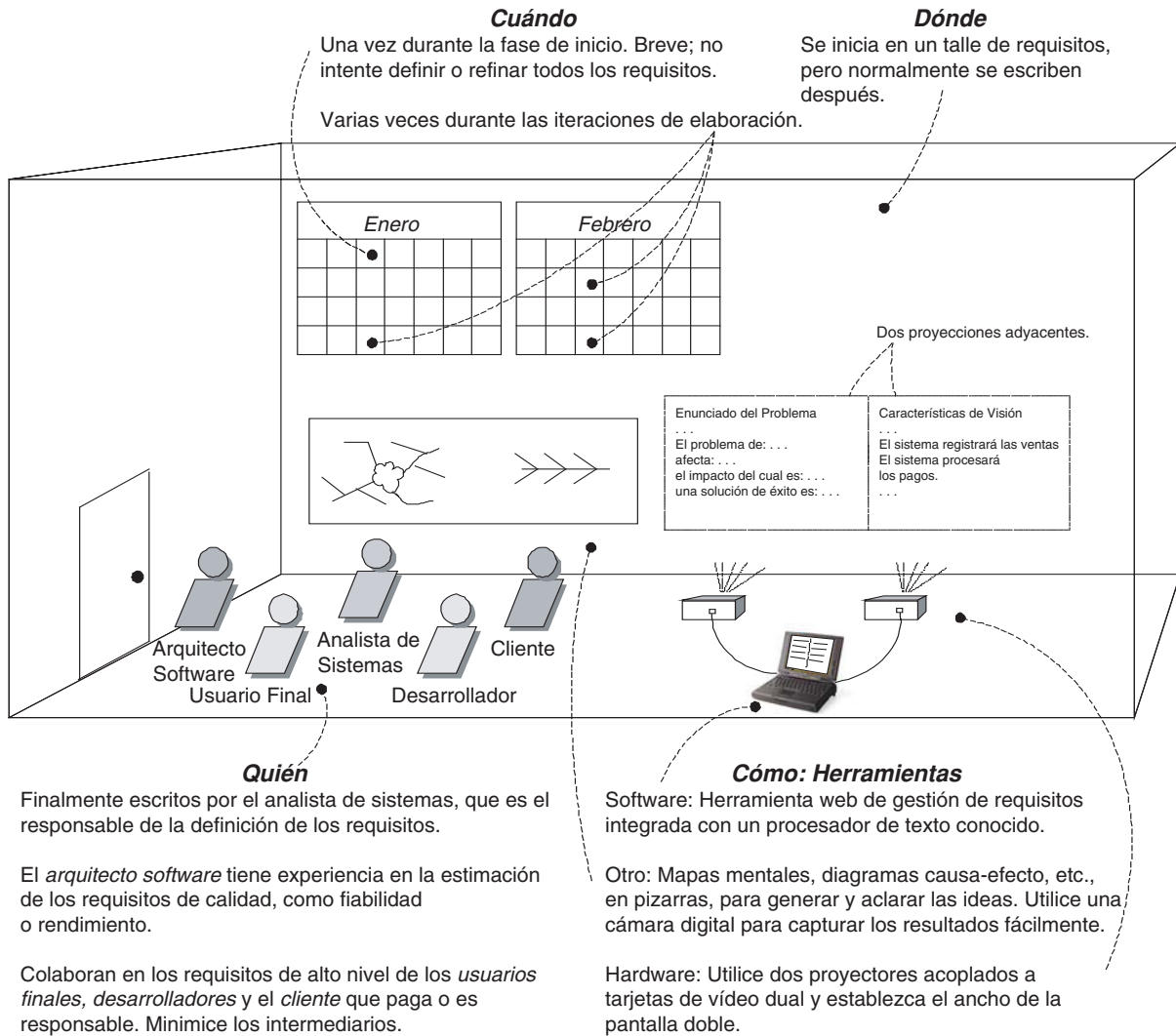


Figura 7.2. Proceso y establecimiento del contexto.

Capítulo 8

DEL INICIO A LA ELABORACIÓN

Lo duro y rígido se rompe. Lo flexible prevalece.

Tao Te Ching

Objetivos

- Definir la etapa de elaboración.
- Motivar los siguientes capítulos de esta sección.

Introducción

La elaboración es la serie inicial de iteraciones durante la que:

- se descubren y estabilizan la mayoría de los requisitos
- se reducen o eliminan los riesgos importantes
- se implementan y prueban los elementos básicos de la arquitectura

Rara vez, la arquitectura no es un riesgo —por ejemplo, si se construye un sitio web como el que el equipo ya ha elaborado con éxito, con las mismas herramientas y los mismos requisitos— en cuyo caso, no tiene que considerarse en estas primeras iteraciones. En este caso, podrían implementarse características o casos de uso críticos, pero no significativos desde el punto de vista de la arquitectura.

Es en esta fase donde el libro se centra en una introducción al A/DOO, aplicando UML, patrones y arquitectura.

8.1. Punto de control: ¿qué sucedió en el inicio?

La etapa de inicio del proyecto de PDV NuevaEra puede durar sólo una semana. Los artefactos creados deberían ser breves e incompletos, la etapa rápida, y la investigación ligera.

No es la fase de requisitos del proyecto, sino una etapa breve para determinar la viabilidad, riesgo y alcance básicos, y decidir si merece la pena más investigación seria en el proyecto, que tendrá lugar en la elaboración. No se han cubierto todas las actividades que podrían ocurrir, de manera razonable, en la fase de inicio; este estudio resalta los artefactos orientados a los requisitos. Algunas de las actividades y artefactos posibles en la fase de inicio comprenden:

- Un breve taller de requisitos.
- La mayoría de los actores, objetivos y casos de usos, con los nombres.
- La mayoría de los casos de uso escritos en formato breve; del 10 al 20% de los casos de uso se escriben en detalle en formato completo para mejorar la comprensión del alcance y la complejidad.
- Identificación de la mayoría de los requisitos de calidad influyentes y de riesgo.
- Escritura de la primera versión de la Visión y Especificación Complementaria.
- Lista de riesgos.
 - Por ejemplo, el director en realidad quiere una demostración en la feria comercial POSWorld en Hamburgo, dentro de 18 meses. Pero el esfuerzo para el desarrollo de la demo no puede ser ni siquiera estimado a grandes rasgos hasta que no se haga un estudio más profundo.
- Prototipos de pruebas de conceptos técnicos y otros estudios para explorar la viabilidad técnica de los requisitos especiales (“¿Funcionan los Java Swing de manera adecuada en pantallas táctiles?”).
- Prototipos orientados a la interfaz de usuario para clarificar la visión de los requisitos funcionales.
- Recomendaciones sobre qué componentes comprar/construir/reutilizar, que se refinarán en la elaboración.
 - Por ejemplo, una recomendación de compra de un paquete de cálculo de impuestos.
- Arquitectura de alto nivel *candidata* y componentes propuestos.
 - No se trata de una descripción detallada de la arquitectura, y no se pretende que sea final o correcta. Más bien, se refiere a una breve especulación que se va a utilizar como punto de partida del estudio en la elaboración. Por ejemplo, “una aplicación Java del lado del cliente, sin un servidor de aplicaciones, Oracle para la base de datos,...”. En la elaboración, podría probarse que merece la pena, o descubrir que es una mala idea y rechazarla.
- Plan para la primera iteración.
- Lista de herramientas candidatas.

8.2. En la elaboración

La elaboración es la serie inicial de iteraciones durante las que el equipo lleva a cabo un estudio serio, implementa (programas y pruebas) el núcleo central de la arquitectura, aclara la mayoría de los requisitos, y aborda las cuestiones de alto riesgo. En el UP, el “riesgo” incluye valor del negocio. Por tanto, el trabajo inicial podría incluir la implementación de los escenarios que se consideran importantes, pero que no son especialmente arriesgados desde el punto de vista técnico.

La elaboración, a menudo, consta de entre dos y cuatro iteraciones; se recomienda que cada iteración dure entre dos y seis semanas, a menos que el tamaño del equipo sea muy grande. Se fija la duración de cada iteración, entendiendo que se fija la fecha de finalización; si es probable que el equipo no cumpla con la fecha, algunos requisitos se colocan en la lista de tareas futuras, de manera que la iteración pueda concluir a tiempo, con una versión estable y que se pueda probar.

La elaboración no se corresponde con una fase de diseño o una fase en la que se desarrollan completamente los modelos preparándolos para que se implementen en la etapa de construcción —que sería un ejemplo de superposición de las ideas del desarrollo en cascada sobre el desarrollo iterativo y el UP—.

Durante esta fase, uno no está creando prototipos desechables; sino que el código y el diseño son porciones del sistema final con calidad de producción. En algunas descripciones del UP, el término, que puede ser origen de malentendidos, “**prototipo de la arquitectura**” se utiliza para describir el sistema parcial. La intención no es que sea un prototipo en el sentido de un experimento desechable; en el UP, significa un subconjunto de producción del sistema final. Más comúnmente se denomina **arquitectura ejecutable o base de la arquitectura**.

La elaboración en una frase

Construir el núcleo central de la arquitectura, resolver los elementos de alto riesgo, definir la mayoría de los requisitos, y estimar la planificación y los recursos globales.

Algunas de las ideas claves y buenas prácticas, que se pondrán de manifiesto en la elaboración, incluyen:

- Llevar a cabo iteraciones breves, de duración fija, dirigidas por el riesgo.
- Comenzar a programar pronto.
- Diseñar, implementar y probar, de manera adaptable, las partes básicas y arriesgadas de la arquitectura.
- Probar desde el principio, a menudo y de manera realista.
- Adaptar en base a la retroalimentación procedente de las pruebas, usuarios y desarrolladores.
- Escribir la mayoría de los casos de uso y otros requisitos en detalle, a través de una serie de talleres, uno por cada iteración de la elaboración.

¿Qué es significativo desde el punto de vista de la arquitectura en la elaboración?

Las iteraciones iniciales construyen y prueban el núcleo central de la arquitectura. Para el proyecto de PDV NuevaEra —y de hecho, para la mayoría— esto incluirá:

- Empleo de diseño e implementación “ancho y superficial”; o, como lo ha llamado Grady Booch, “diseño en las fronteras”.
 - Es decir, identificación de los diferentes procesos, capas, paquetes, y subsistemas, y sus responsabilidades e interfaces de alto nivel. Implementarlos parcialmente con el objeto de conectarlos y clarificar las interfaces. Los módulos contendrían, en su mayor parte, código “*stubbed*” que permite probar las conexiones e interfaces.
- Refinamiento de las interfaces locales y remotas inter-módulos (incluyendo los detalles más sutiles acerca de los parámetros y valores de retorno).
 - Por ejemplo, la interfaz del objeto que actuará de intermediario (*wrapper*) en el acceso al sistema de contabilidad de terceras partes.
 - La primera versión de una interfaz rara vez es perfecta. La preocupación inicial por insistir en la prueba, “provocar fallos” y refinamiento de las interfaces, ayuda más tarde al trabajo en paralelo de varios equipos, confiando en interfaces estables.
- Integración de los componentes existentes.
 - Por ejemplo, un calculador de impuestos.
- Implementación de escenarios simplificados de caminos de éxito y fracaso que impulsan el diseño, la implementación y las pruebas, a través de muchos componentes importantes.
 - Por ejemplo, el escenario principal de éxito de *Procesar Venta*, utilizando el escenario de extensión para el pago a crédito.

Las pruebas en la fase de elaboración son importantes, para obtener retroalimentación, adaptar y probar que el núcleo es robusto. Las pruebas iniciales del proyecto NuevaEra incluirán:

- Pruebas de usabilidad de la interfaz de usuario para *Procesar Venta*.
- Pruebas de recuperación cuando falla el servicio remoto, como la autorización de crédito.
- Pruebas de carga alta a los servicios remotos, como la carga al sistema de cálculo de impuestos remoto.

8.3. Planificación de la siguiente iteración

La planificación y gestión del proyecto son importantes pero son temas amplios. En esta sección presentamos brevemente algunas ideas claves, y se proporcionará una introducción en el Capítulo 36.

Organice los requisitos y las iteraciones según el riesgo, grado de cobertura y naturaleza crítica.

- **Riesgo:** comprende tanto la complejidad técnica como otros factores, como incertidumbre del esfuerzo o facilidad de uso.
- **Cobertura:** implica que todas las partes importantes del sistema se tratan, al menos, en las primeras iteraciones —quizás una implementación “amplia y superficial” a través de muchos componentes—.
- **Naturaleza crítica:** se refiere a las funciones de alto valor para el negocio.

Estos criterios se utilizan para priorizar el trabajo a través de las iteraciones. Los casos de uso, o los escenarios de los casos de uso, se clasifican para la implementación —las primeras iteraciones implementan los escenarios clasificados de rango alto—. Además, algunos requisitos se enuncian como características de alto nivel no relacionadas con ningún caso de uso en particular, como el servicio de registro. Éstas también se clasifican.

La clasificación se realiza antes de la Iteración 1, pero después otra vez antes de la Iteración 2, etcétera, cuando nuevos requisitos y nuevas interpretaciones influyan en el orden. Es decir, el plan es adaptable, en lugar de fijado de un modo especulativo al comienzo del proyecto.

Normalmente, surgirá una agrupación difusa de requisitos basada en algunas técnicas de clasificación de colaboración de grupos pequeños. Por ejemplo:

<i>Rango</i>	<i>Requisito (Caso de Uso o Característica)</i>	<i>Comentario</i>
Alto	Procesar Venta Registro (Logging) ...	Puntuación alta en todos los criterios de clasificación. Extendido. Difícil de añadir más tarde. ...
Medio	Mantener Usuarios ...	Afecta al subdominio de seguridad ...
Bajo	...	...

Basándonos en esta clasificación, vemos que algunos escenarios claves del caso de uso *Procesar Venta*, significativos desde el punto de vista de la arquitectura, deberían abordarse en las primeras iteraciones. Esta lista no es exhaustiva; también se abordarán otros requisitos. Además, en cada iteración se trabajará en un caso de uso, implícito o explícito, *Poner en Marcha*, para satisfacer sus necesidades de inicialización.

En términos de los artefactos del UP, unos pocos comentarios sobre esta información de planificación:

- Los requisitos elegidos para la siguiente iteración se listan brevemente en un **Plan de Iteración**. No se trata de un plan de todas las iteraciones, sólo un plan de la siguiente.
- Si la breve descripción del Plan de Iteración es insuficiente, se podría escribir con más detalle una tarea o requisitos en una **Solicitud de Cambio**, y dársela al equipo responsable.
- La clasificación de requisitos se recoge en el **Plan de Desarrollo de Software**.

8.4. Requisitos y énfasis de la iteración 1: habilidades de A/DOO fundamentales

En este caso de estudio, la Iteración 1 de la fase de elaboración hace hincapié en una gama de habilidades de A/DOO fundamentales y comunes, utilizadas en la construcción de sistemas de objetos, como la asignación de las responsabilidades a los objetos. Por supuesto, se necesitan muchas otras habilidades y etapas —como el diseño de bases de datos, ingeniería de usabilidad y diseño de interfaces de usuario— para construir software, pero quedan fuera del alcance de esta introducción al A/DOO y al UP.

Requisitos de la iteración 1

Los requisitos para la primera iteración de la aplicación PDV NuevaEra son los siguientes:

- Implementación de un escenario clave básico del caso de uso *Procesar Venta*: entrada de artículos y recepción del pago en efectivo.
- Implementación de un caso de uso *Poner en Marcha*, necesario para dar soporte a las necesidades de inicialización de la iteración.
- No se maneja nada especial o complejo, sólo un simple escenario que acaba con éxito, y el diseño e implementación que lo soporte.
- No hay colaboración con servicios externos, como el sistema de cálculo de impuestos o base de datos de artículos.
- No se aplican reglas complejas para fijar los precios.

Aunque no se cubre, también se abordaría el diseño e implementación de las interfaces de usuario de soporte.

Las siguientes iteraciones extenderán esta base.

Desarrollo incremental para el mismo caso de uso a través de las iteraciones

Nótese que no todos los requisitos del caso de uso *Procesar Venta* se han tratado en la iteración 1. Es normal continuar con el trabajo de varios escenarios o características del mismo caso de uso en varias iteraciones y gradualmente ampliar el sistema para, al final, tratar toda la funcionalidad requerida (ver Figura 8.1). Por otro lado, los casos de uso simples y cortos, podrían completarse en una iteración.

8.5. ¿Qué artefactos podrían crearse en la elaboración?

La Tabla 8.1 lista artefactos de *muestra* que podrían crearse en la elaboración, e indica qué cuestión abordan. Los capítulos siguientes presentarán algunos de ellos con mayor

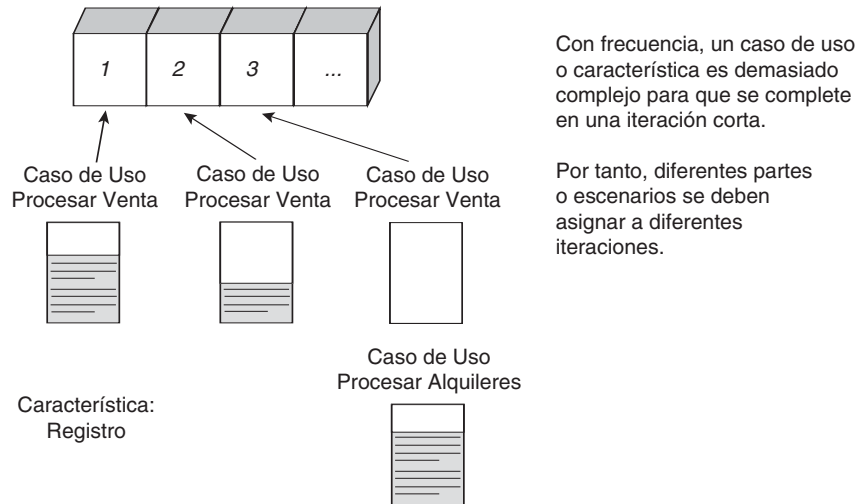


Figura 8.1. La implementación del caso de uso puede extenderse a lo largo de varias iteraciones.

Tabla 8.1. Muestra de Artefactos de elaboración, excluyendo aquellos que se crearon en la fase de inicio.

<i>Artefacto</i>	<i>Comentario</i>
Modelo del Dominio	Es una visualización de los conceptos del dominio; es similar al modelo de información estático de las entidades del dominio.
Modelo de Diseño	Es el conjunto de diagramas que describen el diseño lógico. Comprende los diagramas de clases software, diagramas de interacción, diagramas de paquetes, etcétera.
Documento de la Arquitectura Software	Una ayuda de aprendizaje que resume las cuestiones claves de la arquitectura y cómo se resuelven en el diseño. Es un resumen de las ideas destacadas del diseño y su motivación en el sistema.
Modelo de Datos	Incluye los esquemas de bases de datos, y las estrategias de transformación entre representaciones de objetos y no objetuales.
Modelo de Pruebas	Una descripción de lo que se probará y cómo.
Modelo de Implementación	Se corresponde con la implementación real —el código fuente, ejecutables, base de datos, etcétera—.
Guiones de Caso de Uso, Prototipos UI	Descripción de la interfaz de usuario, caminos de navegación, modelos de facilidad de uso, etcétera.

detalle, en especial el Modelo del Dominio y el Modelo de Diseño. Por brevedad, la tabla excluye los artefactos que pueden crearse en la fase de inicio (y se listaron en el Capítulo 4); presenta los artefactos que es más probable que se creen en la elaboración. Nótese que no se completarán en una iteración; sino que se refinarán a lo largo de una serie de iteraciones.

8.6. No se entendió la elaboración cuando...

- La duración es superior a “unos pocos” meses en la mayoría de los proyectos.
- Sólo comprende una iteración (con raras excepciones para los problemas bien entendidos).
- La mayoría de los requisitos se definieron antes de la elaboración.
- Los elementos arriesgados y el núcleo de la arquitectura no se abordan.
- El resultado no es una arquitectura *ejecutable*; no hay programación de código de producción.
- Se considera fundamentalmente una fase de requisitos, que precede a una fase de implementación en la construcción.
- Se intenta llevar a cabo un diseño completo y cuidadoso antes de la programación.
- Existe una retroalimentación y adaptación mínima; los usuarios no se involucran continuamente en la evaluación y retroalimentación.
- No se llevan a cabo pruebas realistas en las primeras etapas.
- La arquitectura se termina de forma especulativa antes de la programación.
- Se considera una etapa para realizar la programación de pruebas de conceptos, en lugar de programar la arquitectura ejecutable básica de producción.
- No se realizan varios talleres de requisitos breves que adaptan y refinan los requisitos en base a la retroalimentación de las iteraciones anteriores y la actual.

Si un proyecto presenta estos síntomas, no se ha entendido la fase de elaboración.

Parte 3

ELABORACIÓN EN LA ITERACIÓN 1

MODELO DE CASOS DE USO: REPRESENTACIÓN DE LOS DIAGRAMAS DE SECUENCIA DEL SISTEMA

En teoría, no hay diferencia entre teoría y práctica. Pero, en la práctica, la hay.

Jan L. A. van de Snepscheut

Objetivos

- Identificar los eventos de sistema.
- Crear diagramas de secuencia del sistema para los casos de uso.

Sigamos con la iteración 1

El proyecto PDV NuevaEra ha entrado en la primera iteración de desarrollo real. En la fase de inicio se ha realizado un trabajo ligero de requisitos para ayudar a decidir si merecía la pena más investigación seria en el proyecto. Se ha completado la planificación de la primera iteración, y se ha decidido abordar un escenario sencillo de éxito del caso de uso *Procesar Venta* para, únicamente, el pago en efectivo (sin colaboraciones remotas), con el objetivo de comenzar un diseño e implementación “amplio y superficial”, que incluya muchos de los elementos importantes del nuevo sistema. En la primera iteración, tienen lugar muchas tareas relacionadas con el establecimiento del entorno (herramientas, personal, procesos e instalación) que no serán consideradas.

Más bien, volvemos nuestra atención a los casos de usos y al análisis del modelado del dominio. Antes de empezar el trabajo de diseño de la iteración 1, resultará útil realizar un estudio adicional del dominio del problema. Parte de este estudio comprende la

aclaración de los eventos del sistema de entrada y salida relacionados con nuestro sistema, que puede representarse en diagramas de secuencia UML.

Introducción

Un diagrama de secuencia del sistema es un artefacto creado de manera rápida y fácil, que muestra los eventos de entrada y salida relacionados con el sistema que se está estudiando. UML incluye la notación de los diagramas de secuencia para representar los eventos que parten de los actores externos hacia el sistema.

9.1. Comportamiento del sistema

Antes de continuar con el diseño lógico de cómo funcionará la aplicación software, es conveniente estudiar y definir su comportamiento como una “caja negra”. El **comportamiento del sistema** es una descripción de *qué* hace el sistema, sin explicar cómo lo hace. Una parte de esa descripción es un diagrama de secuencia del sistema. Otras partes comprenden los casos de uso y los contratos del sistema (que se presentarán después).

9.2. Diagramas de secuencia del sistema

Los casos de uso describen cómo interactúan los actores externos con el sistema software que estamos interesados en crear. Durante esta interacción, un actor genera eventos sobre un sistema, normalmente solicitando alguna operación como respuesta. Por ejemplo, cuando un cajero inserta el ID de un artículo está solicitando al sistema PDV que registre la venta de ese artículo. Ese evento de solicitud inicia una operación sobre el sistema.

Es deseable aislar e ilustrar las operaciones que un actor externo solicita a un sistema, porque constituyen una parte importante de la comprensión del comportamiento del sistema. UML incluye los **diagramas de secuencia** como notación que puede representar las interacciones de los actores y las operaciones que inician.

Un **diagrama de secuencia del sistema** (DSS) es un dibujo que muestra, para un escenario específico de un caso de uso, los eventos que generan los actores externos, el orden y los eventos entre los sistemas. Todos los sistemas se tratan como cajas negras; los diagramas destacan los eventos que cruzan los límites del sistema desde los actores a los sistemas.

Debería hacerse un DSS para el escenario principal de éxito del caso de uso, y los escenarios alternativos complejos o frecuentes.

UML no define nada denominado diagrama de secuencia “del sistema”, sino simplemente diagrama de secuencia. La calificación se utiliza para subrayar su aplicación para representar sistemas como cajas negras. Posteriormente, se utilizarán los diagramas de secuencia en otro contexto —para ilustrar el diseño de la interacción entre objetos software para completar un trabajo.

9.3. Ejemplo de un DSS

Un DSS muestra, para un curso de eventos específico en un caso de uso, los actores externos que interaccionan directamente con el sistema, el sistema (como una caja negra) y los eventos del sistema que genera el actor (ver Figura 9.1). El tiempo avanza hacia abajo, y la ordenación de los eventos debería seguir su orden en el caso de uso.

Los eventos del sistema podrían contener parámetros.

Este ejemplo muestra el escenario principal de éxito del caso de uso *Procesar Venta*. Se indica que el cajero genera los eventos del sistema *crearNuevaVenta*, *introducirArticulo*, *finalizarVenta*, y *realizarPago*.

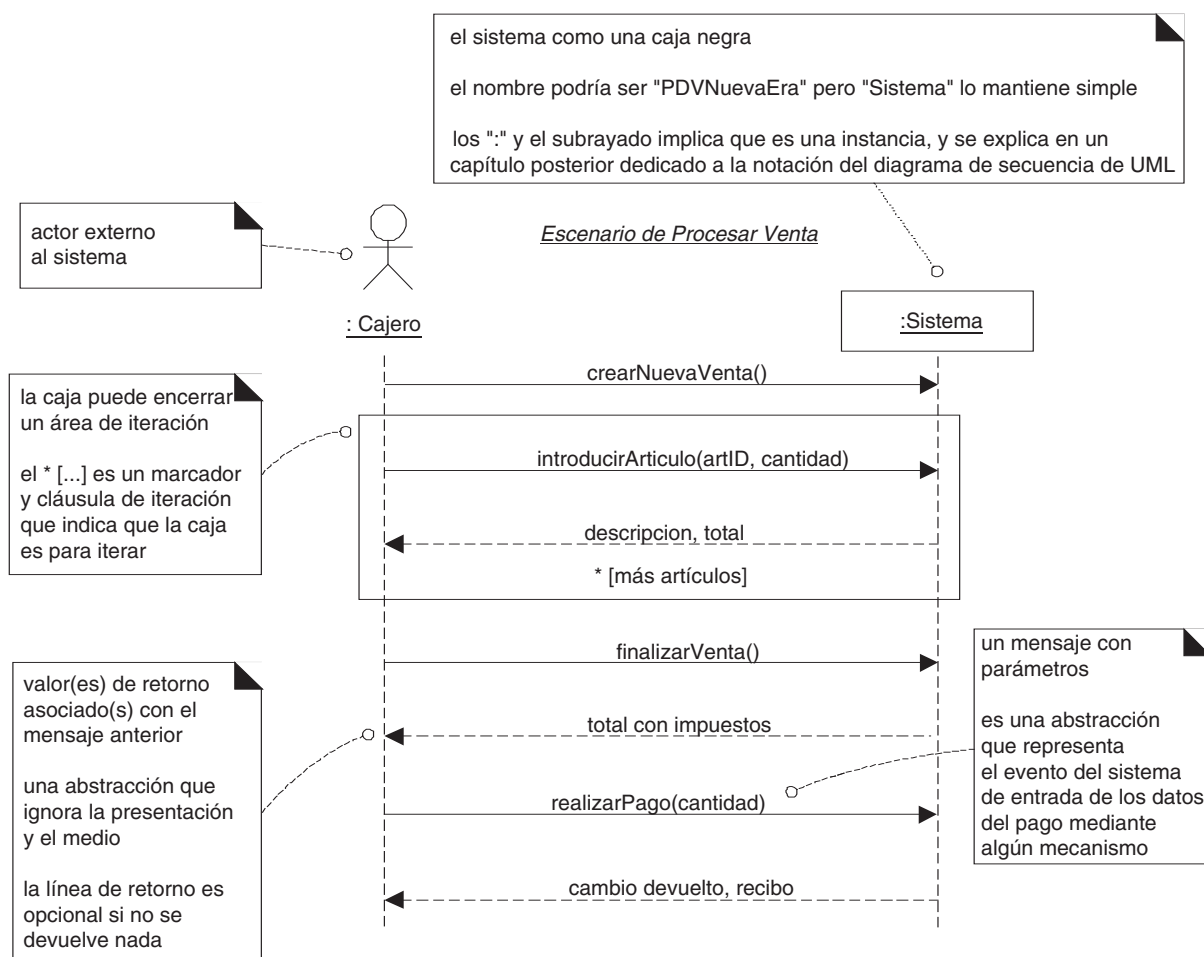


Figura 9.1. DSS para un escenario de *Procesar Venta*.

9.4. DSS entre sistemas

Los DSS también pueden utilizarse para ilustrar las colaboraciones entre sistemas, como entre el PDV NuevaEra y el sistema externo que autoriza pagos a crédito. Sin em-

bargo, se pospone hasta una iteración posterior en el caso de estudio, puesto que esta iteración no incluye las colaboraciones con sistemas remotos.

9.5. DSS y los casos de uso

Un DSS muestra los eventos del sistema para un escenario de un caso de uso, por tanto, se genera para el estudio de un caso de uso (ver Figura 9.2).

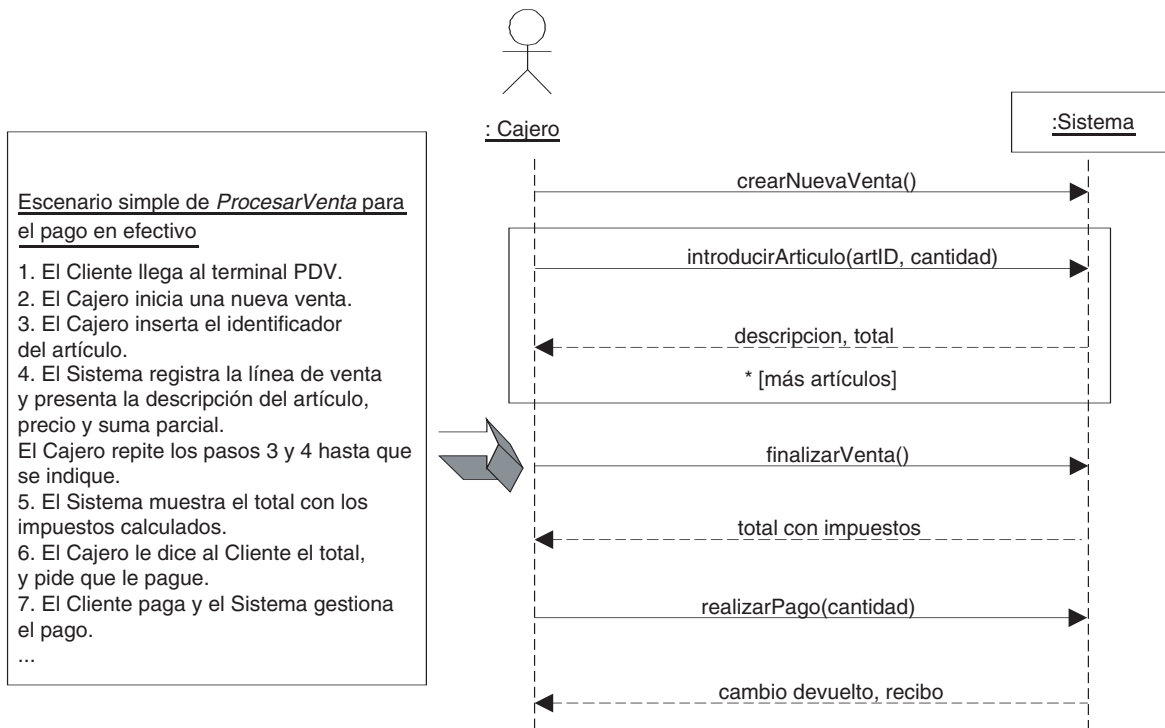


Figura 9.2. Los DSS se derivan de los casos de uso.

9.6. Eventos del sistema y los límites del sistema

Para identificar los eventos del sistema, es necesario tener claros los límites del sistema, como se presentó en el capítulo anterior sobre los casos de uso. Por lo que toca al desarrollo de software, el límite del sistema normalmente se elige para que sea el propio sistema software (y posiblemente hardware); en este contexto, un evento del sistema es un evento externo que lanza un estímulo directamente al software (ver Figura 9.3).

Consideremos el caso de uso *Procesar Venta* para identificar los eventos del sistema. Primero, debemos determinar los actores que interactúan directamente con el sistema software. El cliente interactúa con el cajero, pero para este escenario simple de pago en efectivo, no interactúa directamente con el sistema PDV —sólo lo hace el cajero—. Por tanto, el cliente no es un generador de eventos del sistema; sólo lo es el cajero.

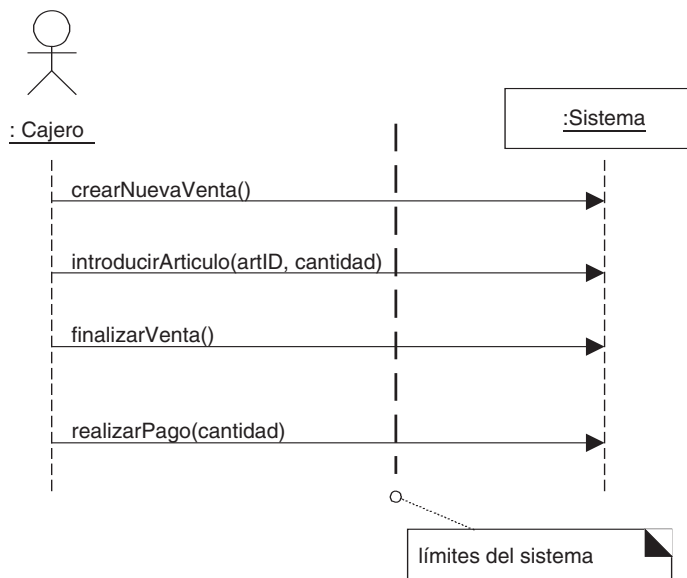


Figura 9.3. Definición de los límites del sistema.

9.7. Asignación de nombres a los eventos y operaciones

Los eventos del sistema (y sus operaciones del sistema asociadas) deberían expresarse al nivel de intenciones en lugar de en términos del medio de entrada físico o a nivel de elementos de la interfaz de usuario.

También se mejora la claridad, el comenzar el nombre de un evento del sistema con un verbo (añadir..., insertar..., finalizar..., crear...), como en la Figura 9.4, puesto que resalta la orientación de orden de estos eventos.

Así, “introducirArticulo” es mejor que “escanear” (esto es, escanear con láser) porque captura la intención de la operación, al mismo tiempo que permanece abstracta y sin compromiso respecto a las elecciones de diseño sobre qué interfaz utilizar para capturar el evento del sistema.

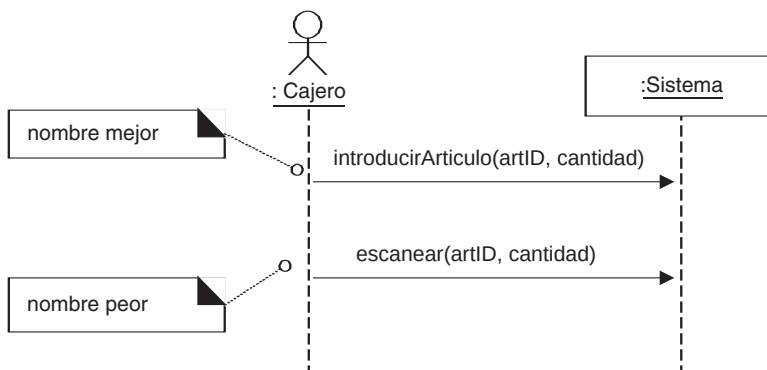


Figura 9.4. Elección de los nombres de los eventos y las operaciones a un nivel abstracto.

9.8. Mostrar el texto del caso de uso

A veces, es deseable mostrar al menos fragmentos del texto del caso de uso del escenario, con el fin de aclarar o enriquecer las dos vistas (ver Figura 9.5). El texto proporciona los detalles y el contexto; el diagrama resume visualmente la interacción.

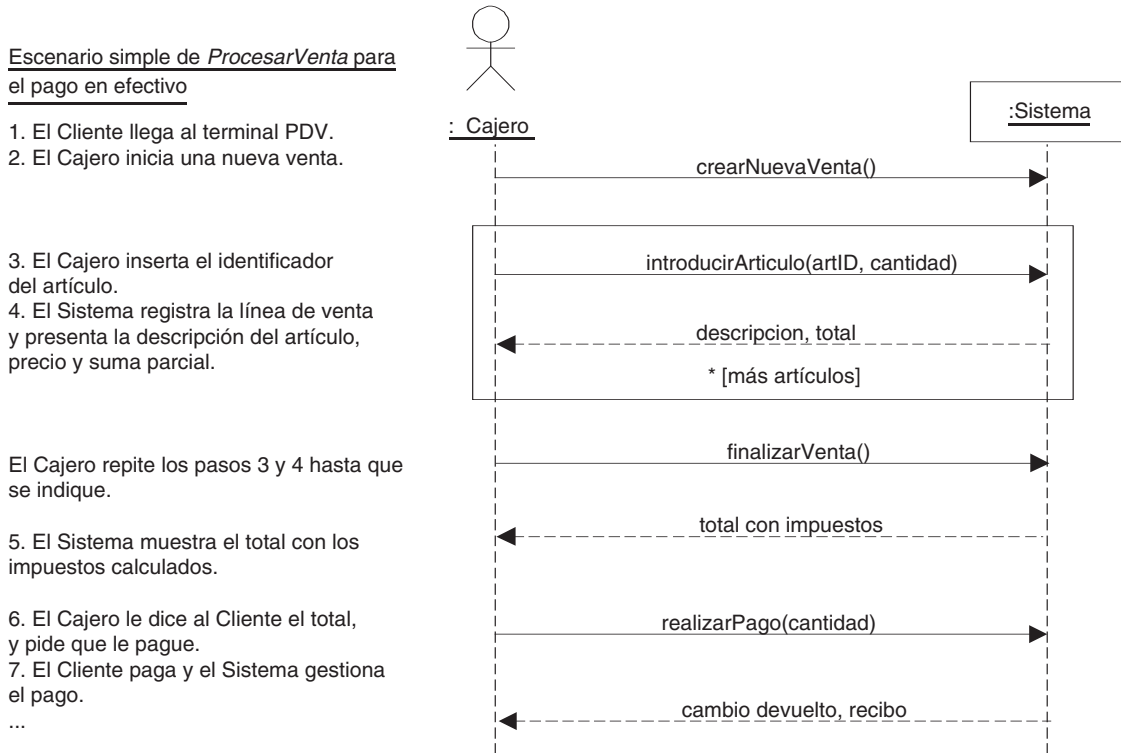


Figura 9.5. DSS con texto del caso de uso..

9.9. Los DSS y el Glosario

Los términos representados en los DSS (operaciones, parámetros, valores de retorno) son concisos. Éstos podrían necesitar una explicación más adecuada de manera que, durante el trabajo de diseño, esté claro qué es lo que entra y lo que sale. Si no se explicó en los casos de uso, podría utilizarse el Glosario.

Sin embargo, como siempre que discutimos la creación de artefactos distintos al código (lo esencial del proyecto), hay que ser desconfiado. Con los datos del Glosario debería hacerse algún uso o decisión realmente significativa, de lo contrario es un trabajo innecesario de poco valor.

9.10. DSS en el UP

Los DSSs forman parte del Modelo de Casos de Uso —una visualización de las interacciones implicadas en los casos de uso—. Los DSSs no se mencionaron explícitamente en la

descripción original del UP, aunque los creadores del UP eran conscientes y entendían la utilidad de tales diagramas. Los DSS son un ejemplo de los muchos posibles artefactos o actividades de análisis y diseño de utilidad que los documentos UP o el RUP no mencionan.

Fases

Inicio: Los DSS no se incentivan normalmente en la fase de inicio.

Elaboración: La mayoría de los DSS se crean durante la elaboración, cuando es útil identificar los detalles de los eventos del sistema para poner en claro cuáles son las operaciones que se deben diseñar que gestione el sistema, escribir los contratos de las operaciones del sistema (se presentarán en el Capítulo 13), y posiblemente, dar soporte a la estimación (por ejemplo, macroestimación con puntos de función no ajustados y COCOMO II).

Nótese que no es necesario crear DSS para todos los escenarios de todos los casos de uso —al menos no a la vez. Sino que, se crearán sólo para algunos escenarios seleccionados de la iteración actual.

Finalmente, sólo debería llevar unos pocos minutos o una media hora la creación de los DSSs.

Tabla 9.1. Muestra de los artefactos UP y evolución temporal. c – comenzar; r – refinar.

<i>Disciplina</i>	<i>Artefacto</i> <i>Iteración →</i>	<i>Inicio</i> <i>I1</i>	<i>Elab.</i> <i>E1...En</i>	<i>Const.</i> <i>C1...Cn</i>	<i>Trans.</i> <i>T1...T2</i>
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso (DSS)	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c		
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

9.11. Lecturas adicionales

Durante décadas, se ha utilizado ampliamente variaciones de los diagramas que ilustran los eventos de entrada/salida de un sistema considerado como una caja negra; por ejemplo, en telecomunicaciones como diagramas de flujo de llamadas. Especialmente, se hicieron populares en los métodos orientados a objetos a través de su uso en el método Fusion [Coleman+94], que proporciona un ejemplo detallado de la relación de los DSSs y las operaciones del sistema con otros artefactos del análisis y diseño.

9.12. Artefactos del UP

La Figura 9.6 presenta ejemplos de relaciones de los DSSs con otros artefactos.

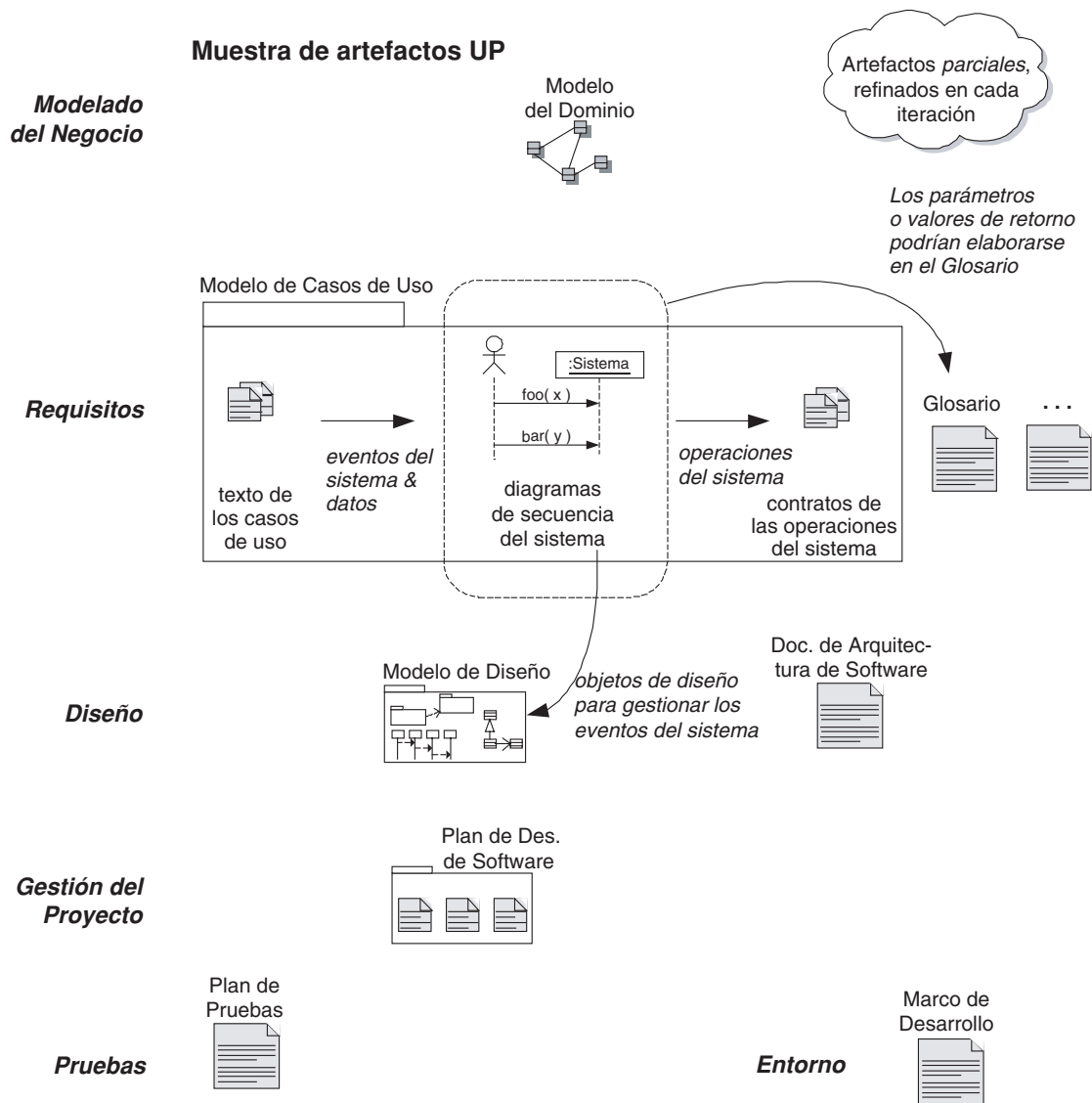


Figura 9.6. Muestra de la influencia de los artefactos UP.

MODELO DEL DOMINIO: VISUALIZACIÓN DE CONCEPTOS

Todo está muy bien en la práctica, pero nunca funcionará en la teoría.

Máxima anónima sobre la gestión

Objetivos

- Identificar las clases conceptuales relacionadas con los requisitos de la iteración actual.
- Crear un modelo del dominio inicial.
- Distinguir entre atributos correctos e incorrectos.
- Añadir las clases conceptuales de *especificación*, cuando sea oportuno.
- Comparar y contrastar las vistas conceptual y de implementación.

Introducción

Un modelo del dominio se utiliza con frecuencia como fuente de inspiración para el diseño de los objetos software, y será una entrada necesaria para varios de los siguientes artefactos que se presentan en este libro. Por tanto, es importante leer este capítulo si el lector no está familiarizado con el tema del modelado del dominio.

Un modelo del dominio muestra (a los modeladores) clases conceptuales significativas en un dominio del problema; es el artefacto más importante que se crea durante el análisis orientado a objetos¹. Este capítulo estudia técnicas introductorias a la creación de modelos del dominio. Los siguientes dos capítulos tratarán más extensamente las técnicas de modelado del dominio —añadiendo atributos y asociaciones—.

¹ Los casos de uso son un importante artefacto del análisis de requisitos, pero no son orientados a *objetos*. Ponen de relieve una vista de procesos del dominio.

La identificación de un conjunto rico de objetos o clases conceptuales es una parte esencial del análisis orientado a objetos, y bien merece la pena el esfuerzo en relación con los beneficios durante el trabajo de diseño e implementación.

La identificación de las clases conceptuales forma parte del estudio del dominio del problema. UML contiene notación, en forma de diagramas de clases, para representar los modelos del dominio.

Idea clave

Un modelo del dominio es una representación de las clases conceptuales del mundo real, no de componentes software. *No* se trata de un conjunto de diagramas que describen clases software, u objetos software con responsabilidades.

10.1. Modelos del dominio

La etapa orientada a *objetos* esencial del análisis o investigación es la descomposición de un dominio de interés en clases conceptuales individuales u objetos —las cosas de las que somos conscientes—. Un **modelo del dominio** es una representación *visual* de las clases conceptuales u objetos del mundo real en un dominio de interés [MO95, Fowler96]. También se les denomina **modelos conceptuales** (término utilizado en la primera edición de este libro), **modelo de objetos del dominio** y **modelos de objetos de análisis**².

El UP define un Modelo de Dominio³ como uno de los artefactos que podrían crearse en la disciplina del Modelado del Negocio.

Utilizando la notación UML, un modelo del dominio se representa con un conjunto de **diagramas de clases** en los que no se define ninguna operación. Pueden mostrar:

- Objetos del dominio o clases conceptuales.
- Asociaciones entre las clases conceptuales.
- Atributos de las clases conceptuales.

Por ejemplo, la Figura 10.1 muestra un modelo del dominio parcial. Ilustra que las clases conceptuales *Pago* y *Venta* son significativas en este dominio, que un *Pago* está relacionado con una *Venta* de un modo que es significativo hacer notar, y que una *Venta* tiene una fecha y una hora. Los detalles de la notación no son importantes en este momento.

Idea clave: Modelo del dominio—un diccionario visual de abstracciones

Por favor, reflexione un momento sobre la Figura 10.1. La figura visualiza y relaciona algunas palabras o clases conceptuales del dominio. También describe una *abstracción* de

² También están relacionados con los modelos conceptuales entidad-relación, que son capaces de mostrar vistas de los dominios puramente conceptuales, pero que han sido ampliamente re-interpretados como modelos de datos para el diseño de bases de datos. Los modelos del dominio no son modelos de datos.

³ Se utiliza Modelo del Dominio en mayúsculas cuando deseo resaltar que es un modelo oficial definido en el UP, frente al concepto bien conocido de “modelos de dominio”.

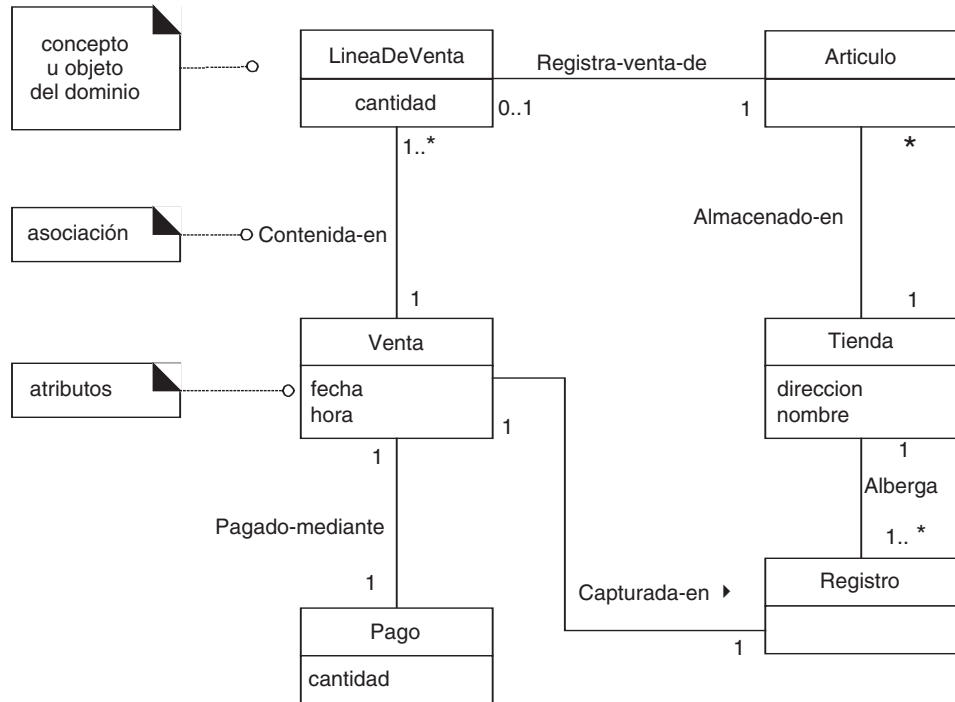


Figura 10.1. Modelo del dominio parcial —un diccionario visual—. El número de cada extremo de la línea indica la multiplicidad, que se describirá en un capítulo posterior.

las clases conceptuales, porque hay muchas cosas que uno podría comunicar sobre los registros, las ventas, etcétera. El modelo muestra una vista parcial, o abstracción, e ignora detalles sin interés (para el modelador).

La información que presenta (utilizando la notación UML) podría, de manera alternativa, haberse expresado en prosa, mediante sentencias en el Glosario o en algún otro sitio. Pero es fácil entender los distintos elementos y sus relaciones mediante este lenguaje visual, puesto que un porcentaje significativo del cerebro toma parte en el procesamiento visual —es una cualidad de los humanos—.

Por tanto, el modelo del dominio podría considerarse como un *diccionario visual* de las abstracciones relevantes, vocabulario del dominio e información del dominio.

Los modelos del dominio no son modelos de componentes software

Un modelo del dominio, como se muestra en la Figura 10.2, es una representación de las cosas del mundo real del dominio de interés, *no* de componentes software, como una clase Java o C++ (ver Figura 10.3), u objetos software con responsabilidades. Por tanto, los siguientes elementos no son adecuados en un modelo del dominio:

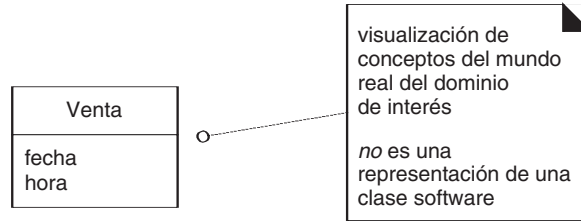


Figura 10.2. Un modelo del dominio muestra clases conceptuales del mundo real, no clases software.

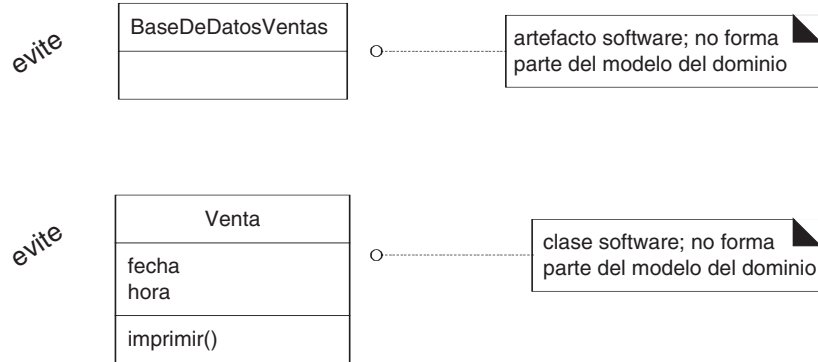


Figura 10.3. Un modelo del dominio no muestra artefactos software o clases.

- Artefactos software, como una ventana o una base de datos, a menos que el dominio que se esté modelando sea de conceptos software, como un modelo de interfaces de usuario gráficas.
- Responsabilidades o métodos⁴.

Clases conceptuales

El modelo del dominio muestra las clases conceptuales o vocabulario del dominio. Informalmente, una clase conceptual es una idea, cosa u objeto. Más formalmente, una clase conceptual podría considerarse en términos de su símbolo, intensión, y extensión [MO95] (ver Figura 10.4).

- **Símbolo:** palabras o imágenes que representan una clase conceptual.
- **Intensión:** la definición de una clase conceptual.
- **Extensión:** el conjunto de ejemplos a los que se aplica la clase conceptual.

Por ejemplo, considere la clase conceptual para el evento de una transacción de compra. Podría elegir nombrarla con el símbolo *Venta*. La intensión de una *Venta* podría es-

⁴ En el modelado de objetos, normalmente hablamos de responsabilidades relacionadas con los componentes software. Y los métodos son puramente conceptos software. Pero el modelo de dominio describe conceptos del mundo real, no componentes software. Es importante considerar las responsabilidades de los objetos durante el trabajo de *diseño*; sólo que no forma parte de este modelo. Un caso válido en el que se podrían mostrar las responsabilidades en un modelo de dominio es si incluye roles de trabajadores humanos (como el Cajero), y el modelador desea recoger las responsabilidades de estos trabajadores humanos.

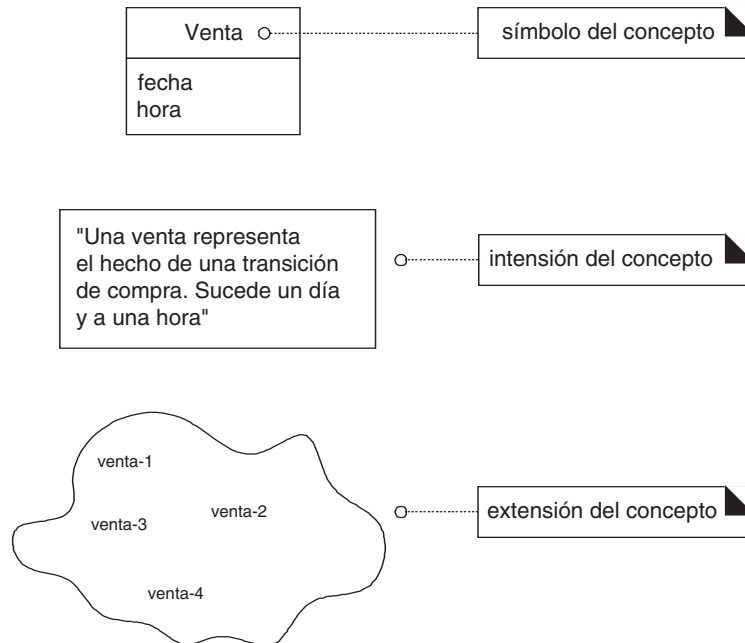


Figura 10.4. Una clase conceptual tiene un símbolo, una intención y una extensión.

tablecer que “representa el hecho de una transacción de una compra, y tiene una fecha y una hora”. La extensión de una *Venta* la forman todos los ejemplos de ventas; en otras palabras, el conjunto de todas las ventas.

Cuando creamos un modelo del dominio, normalmente, el símbolo y la vista intensional de la clase conceptual son los que tienen un mayor interés práctico.

Modelos y descomposición del dominio

Los problemas del software pueden ser complejos; la descomposición —divide y vencerás— es una estrategia común para tratar esta complejidad mediante la división del espacio del problema en unidades fáciles de comprender. En el **análisis estructurado**, la dimensión de la descomposición es por procesos o por *funciones*. Sin embargo, en el análisis orientado a objetos, la dimensión de la descomposición es fundamentalmente por cosas o entidades del dominio.

Una diferencia esencial entre el análisis orientado a objetos y el estructurado es: la división por clases conceptuales (objetos) en lugar de la división por funciones.

Por tanto, la principal tarea del análisis es identificar diferentes conceptos en el dominio del problema y documentar el resultado en un modelo del dominio.

Clases conceptuales en el dominio de ventas

Por ejemplo, en el dominio de ventas en una tienda del mundo real, existen las clases conceptuales de *Tienda*, *Registro* y *Venta*. Por tanto, nuestro modelo del dominio, mostrado en la Figura 10.5, podría incluir *Tienda*, *Registro* y *Venta*.

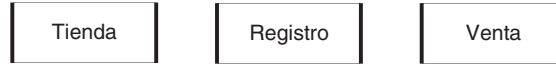


Figura 10.5. Modelo del dominio parcial en el dominio de la tienda.

10.2. Identificación de las clases conceptuales

Nuestro objetivo es crear un modelo del dominio de clases conceptuales interesantes o significativas del dominio de interés (ventas). En este caso, eso significa conceptos relacionados con el caso de uso *Procesar Venta*.

En el desarrollo iterativo, uno incrementalmente construye un modelo del dominio a lo largo de varias iteraciones en la fase de elaboración. En cada una, el modelo de dominio se limita a los escenarios anteriores y actual en estudio, en lugar de un modelo de “gran explosión”, que en las primeras etapas intenta capturar todas las posibles clases conceptuales y las relaciones. Por ejemplo, esta iteración está limitada al escenario de pago en efectivo de *Procesar Venta*; por tanto, se creará un modelo de dominio parcial únicamente para reflejar eso —no más—.

La tarea central es, por tanto, identificar las clases conceptuales relacionadas con el escenario que se está diseñando.

A continuación presentamos una guía útil para la identificación de las clases conceptuales:

Es mejor especificar en exceso un modelo del dominio con muchas clases conceptuales de grano fino que especificar por defecto.

No piense que un modelo de dominio es mejor si contiene pocas clases conceptuales; suele ser verdad justamente lo contrario.

Es normal obviar clases conceptuales durante la etapa de identificación inicial, y descubrirlas más tarde al considerar los atributos y asociaciones, o durante el trabajo de diseño. Cuando se encuentren, se pueden añadir al modelo del dominio.

No excluya una clase conceptual simplemente porque los requisitos no indican ninguna necesidad obvia para registrar información sobre ella (un criterio común en el modelado de datos para el diseño de bases de datos relacionales, pero no relevante en el modelado del dominio), o porque la clase conceptual no tiene atributos.

Es válido tener clases conceptuales sin atributos, o clases conceptuales con un rol puramente de comportamiento en el dominio, en lugar de un rol de información.

Estrategias para identificar clases conceptuales

En las siguientes secciones se presentan dos técnicas:

1. Utilización de una lista de categorías de clases conceptuales.
2. Identificación de frases nominales.

Otra excelente técnica para el modelado del dominio es el uso de **patrones de análisis**, que son modelos de dominios parciales existentes creados por expertos, utilizando libros publicados como *Analysis Patterns* [Fowler96] y *Data Model Patterns* [Hay96].

Utilización de una lista de categorías de clases conceptuales

Comience la creación de un modelo del dominio haciendo una lista de clases conceptuales candidatas. La Tabla 10.1 contiene muchas categorías habituales que, normalmente, merece la pena tener en cuenta, aunque no en ningún orden particular de importancia. Los ejemplos se han extraído del dominio de las tiendas y las reservas de vuelos.

Tabla 10.1. Lista de categorías de clases conceptuales.

<i>Categoría de clase conceptual</i>	<i>Ejemplos</i>
objetos tangibles o físicos	<i>Registro Avion</i>
especificaciones, diseños o descripciones de la cosas	<i>EspecificacionDelProducto DescripcionDelVuelo</i>
lugares	<i>Tienda</i>
transacciones	<i>Venta, Pago Reserva</i>
líneas de la transacción	<i>LineaDeVenta</i>
roles de la gente	<i>Cajero Piloto</i>
contenedores de otras cosas	<i>Tienda, Lata Avion</i>
cosas en un contenedor	<i>Articulo Pasajero</i>
otros sistemas informáticos o electromecánicos externos al sistema	<i>SistemaAutorizacionPagoCredito ControlDeTraficoAereo</i>
conceptos abstractos	<i>Ansia Acrofobia</i>
organizaciones	<i>DepartamentoDeVentas CompañiaAerea</i>
hechos	<i>Venta, Pago, Reunion Vuelo, Colision, Aterrizaje</i>

continúa

Tabla 10.1. Lista de categorías de clases conceptuales. (Continuación)

<i>Categoría de clase conceptual</i>	<i>Ejemplos</i>
procesos (normalmente <i>no</i> se representan como conceptos, pero podría ocurrir)	<i>VentaDeUnProducto</i> <i>ReservaUnAsiento</i>
reglas y políticas	<i>PoliticaDeReintegro</i> <i>PoliticaDeCancelación</i>
catálogos	<i>CatalogoDeProductos</i> <i>CatalogoDePiezas</i>
registros de finanzas, trabajo, contratos, cuestiones legales	<i>Recibo, LibroMayor, ContratoEmpleo</i> <i>RegistroMantenimiento</i>
instrumentos y servicios financieros	<i>LineaDeCredito</i> <i>Stock</i>
manuales, documentos, artículos de referencia, libros	<i>ListaDeCambiosDePreciosDiarios</i> <i>ManualReparaciones</i>

Descubrimiento de clases conceptuales mediante la identificación de frases nominales

Otra técnica útil (debido a su simplicidad) recomendada en [Abbot83] es el análisis lingüístico: identificar los nombres y frases nominales en las descripciones textuales de un dominio, y considerarlos como clases conceptuales o atributos candidatos.

Se debe tener cuidado con este método; no es posible realizar una correspondencia mecánica de nombres a clases, y las palabras en lenguaje natural son ambiguas.

En cualquier caso, es otra fuente de inspiración. Los casos de uso en formato completo constituyen una descripción excelente a partir de la cual extraer este análisis. Por ejemplo, se puede utilizar el escenario actual del caso de uso *Procesar Venta*.

- Escenario principal de éxito (o Flujo Básico):**
1. El **Cliente** llega a un **terminal PDV** con **mercancías** y/o **servicios** que comprar.
 2. El **Cajero** comienza una nueva **venta**.
 3. El **Cajero** introduce el **identificador del artículo**.
 4. El Sistema registra la **línea de la venta** y presenta la **descripción del artículo, precio** y **suma** parcial. El precio se calcula a partir de un conjunto de reglas de precios.
El Cajero repite los pasos 3-4 hasta que se indique.
 5. El Sistema presenta el total con los **impuestos** calculados.
 6. El Cajero le dice al Cliente el total y solicita el **pago**.
 7. El Cliente paga y el Sistema gestiona el pago.
 8. El Sistema registra la **venta** completa y envía la información de la venta y el pago al sistema de **Contabilidad** externo (para la contabilidad y las **comisiones**) y al sistema de **Inventario** (para actualizar el inventario).
 9. El Sistema presenta el **recibo**.
 10. El Cliente se va con el recibo y las mercancías (si es el caso).

Extensiones (o Flujos Alternativos):

...

7a. Pago en efectivo:

1. El Cajero introduce la **cantidad** de dinero **entregada** en efectivo.
2. El Sistema muestra la **cantidad** de dinero **a devolver** y abre el **cajón de caja**.
3. El Cajero deposita el dinero entregado y devuelve el cambio al Cliente.
4. El Sistema registra el pago en efectivo.

El modelo del dominio es una visualización de los conceptos del dominio y vocabulario relevantes. ¿Dónde se encuentran estos términos? En los casos de uso. Por tanto, constituyen una fuente rica a explorar mediante la identificación de frases nominales.

Algunas de las frases nominales son clases conceptuales candidatas, algunas podrían hacer referencia a clases conceptuales que se ignoran en esta iteración (por ejemplo, “Contabilidad” y “comisiones”), y algunas podrían ser atributos de las clases conceptuales. Por favor, diríjase a la siguiente sección y al capítulo sobre los atributos, para obtener los consejos que permiten diferenciar entre los dos.

Un punto débil de este enfoque es la imprecisión del lenguaje natural; frases nominales diferentes podrían representar la misma clase conceptual o atributo, entre otras ambigüedades. Sin embargo, se recomienda que se combine con la técnica la *Lista de Categorías de Clases Conceptuales*.

10.3. Clases conceptuales candidatas para el dominio de ventas

A partir del análisis de la Lista de Categorías de Clases Conceptuales y las frases nominales, se genera una lista de clases conceptuales candidatas del dominio. La lista está restringida a los requisitos y simplificaciones que se están estudiando actualmente —el escenario simplificado de *Procesar Venta*—.

<i>Registro</i>	<i>EspecificacionDelProducto</i>
<i>Articulo</i>	<i>LineaDeVenta</i>
<i>Tienda</i>	<i>Cajero</i>
<i>Venta</i>	<i>Cliente</i>
<i>Pago</i>	<i>Encargado</i>
<i>CatalogoDeProductos</i> ⁵	

No existe una lista “correcta”. Es una colección algo arbitraria de abstracciones y vocabulario del dominio que el modelador considera relevantes. En cualquier caso, siguiendo la estrategia de identificación, diferentes modeladores producirán listas similares.

⁵ *N. del T.*: Aunque siguiendo las convenciones para nombrar las clases no se utilizaría la preposición, en este caso al igual que en *EspecificacionDelProducto* y *LineaDeVenta*, se han utilizado para mejorar la legibilidad del texto.

Objetos de informes: ¿incluir el recibo en el modelo?

Un recibo es un informe de una venta y del pago, y una clase conceptual relativamente destacable del dominio, por tanto, ¿debería mostrarse en el modelo?

A continuación presentamos algunos factores a tener en cuenta:

- Un recibo es un informe de una venta. En general, no es útil mostrar un informe de otra información en un modelo del dominio puesto que toda esta información se deriva de otras fuentes; duplica información que se encuentra en otras partes. Ésta es una razón para excluirlo.
- Un recibo tiene un rol especial en término de las reglas del negocio: normalmente, confiere al portador del recibo, el derecho a devolver los artículos comprados. Ésta es una razón para mostrarlo en el modelo.

Puesto que la devolución de artículos no está siendo considerada en esta iteración, el *Recibo* será excluido. Durante la iteración que aborda el caso de uso *Gestionar Devoluciones*, estaría justificada su inclusión.

10.4. Guías para el modelado del negocio

Cómo hacer un modelo del dominio

Aplique los siguientes pasos para crear un modelo del dominio:

1. Liste las clases conceptuales candidatas, utilizando las técnicas de la Lista de Categorías de Clases Conceptuales y la identificación de frases nominales, relacionadas con los requisitos actuales en estudio.
2. Representelos en un modelo del dominio.
3. Añada las asociaciones necesarias para registrar las relaciones que hay que mantener en memoria (se discutirá en un capítulo siguiente).
4. Añada los atributos necesarios para satisfacer los requisitos de información (se discutirá en un capítulo siguiente).

Un método útil auxiliar es aprender y copiar patrones de análisis, que se discutirán en un capítulo posterior.

Nombrar y modelar cosas: el cartógrafo

La estrategia del cartógrafo se aplica tanto a los mapas como a los modelos del dominio.

Haga un modelo del dominio con el espíritu del modo de trabajo de los cartógrafos:

- Utilice los nombres existentes en el territorio.
- Excluya las características irrelevantes.
- No añada cosas que no están ahí.

Un modelo del dominio es un tipo de mapa de conceptos o cosas de un dominio. Este espíritu destaca el rol analítico de un modelo del dominio, y sugiere lo siguiente:

- Un cartógrafo utiliza los nombres del territorio —no cambia los nombres de las ciudades en un mapa—. Para un modelo del dominio, significa que *utilice el vocabulario del dominio al nombrar los nombres de las clases conceptuales y los atributos*. Por ejemplo, si estamos desarrollando un modelo para una biblioteca, nombre al cliente como “*Prestatario*” o “*Socio*” —los términos utilizados por el personal de la biblioteca—.
- Un cartógrafo elimina las cosas de un mapa si no se consideran relevantes para el propósito del mapa; por ejemplo, la topografía o la población no es necesario que se muestren. Análogamente, un modelo de dominio podría excluir clases conceptuales del dominio del problema que no son pertinentes para los requisitos. Por ejemplo, podríamos excluir *Bolígrafo* y *BolsaPapel* de nuestro modelo del dominio (para el conjunto de requisitos actual) puesto que no tienen un rol relevante obvio.
- Un cartógrafo no muestra cosas que no estén ahí, como montañas que no existen. Igualmente, el modelo del dominio debería excluir cosas que *no* se encuentran en el dominio del problema que se está estudiando.

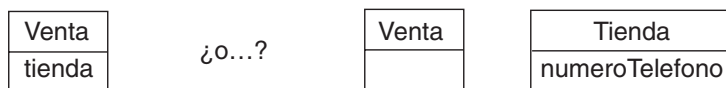
El principio también se conoce como la estrategia *Utilice el Vocabulario del Dominio* [Coad95].

Error típico en la identificación de las clases conceptuales

Quizás el error más típico al crear un modelo del dominio es representar algo como un atributo cuando debería haber sido un concepto. Una regla empírica para ayudar a prevenir este error es:

Si no consideramos alguna clase conceptual X que sea un número o texto en el mundo real, X es probablemente una clase conceptual, no un atributo.

Como ejemplo, ¿debería ser la *tienda* un atributo de *Venta*, o una clase conceptual separada *Tienda*?



En el mundo real, una tienda no se considera un número o un texto —el término sugiere una entidad legal, una organización, y algo que ocupa espacio—. Por tanto, *Tienda* debe ser un concepto.



Otro ejemplo, considere el dominio de las reservas de vuelos. ¿Debería ser el *destino* un atributo de *Vuelo*, o una clase conceptual aparte, *Aeropuerto*?

En el mundo real, un aeropuerto de destino no se considera un número o un texto —es una cosa grande que ocupa espacio—. Por tanto, *Aeropuerto* debería ser un concepto.

En caso de duda, considérela un concepto separado. Los atributos deberían ser bastante raros en un modelo del dominio.

10.5. Resolución de clases conceptuales similares: Registro vs. “TPDV”

TPDV son las siglas de terminal de punto de venta. En el lenguaje informático, un terminal es cualquier dispositivo de un sistema, como un PC cliente, un PDA en red inalámbrico, etcétera. En los primeros tiempos, mucho antes a los TPDVs, una tienda mantenía un *registro* —un libro en el que se apuntaban las ventas y pagos—. Con el tiempo, esto se automatizó en “cajas registradoras” mecánicas. Hoy en día, un TPDV desempeña el rol del registro (ver Figura 10.6).

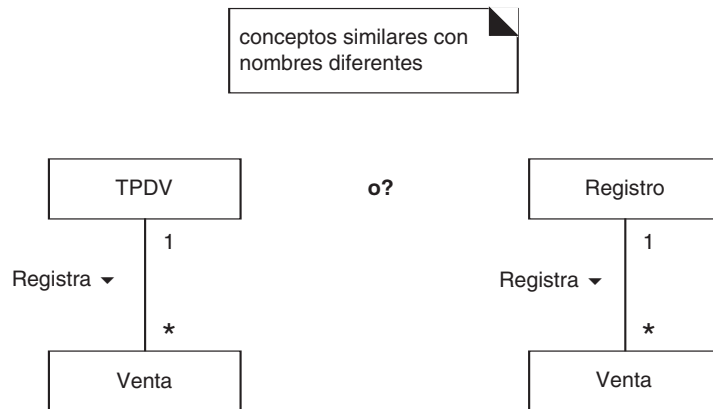


Figura 10.6. TPDV y Registro son clases conceptuales similares.

Un registro es una cosa que recoge las ventas y pagos, pero esto es un TPDV. Sin embargo, el término *registro* parece algo más abstracto y menos orientado a la implementación que un *TPDV*. Por tanto, en el modelo del dominio, ¿debería utilizarse el símbolo *Registro* en lugar de *TPDV*?

Primero, como regla empírica, un modelo del dominio no es absolutamente correcto o equivocado, sino más o menos útil; es una herramienta de comunicación.

Por el principio del cartógrafo, “*TPDV*” es un término familiar en el territorio, de manera que es un símbolo útil desde el punto de vista de la familiaridad y la comunicación. Por el objetivo de la creación de modelos que representan abstracciones y son indepen-

dientes de la implementación, *Registro* es atractivo y útil⁶. Se podría considerar *Registro* justamente, para representar tanto la clase conceptual de un sitio que registra las ventas, como una abstracción de varias clases de terminales, como un TPDV.

Ambas elecciones tienen valor; en este caso de estudio se ha elegido *Registro* algo arbitrariamente, pero el personal involucrado también habría entendido *TPDV*.

10.6. Modelado del mundo *irreal*

Algunos sistemas software son para dominios que encuentran muy poca analogía con dominios naturales o de negocios; el software para las telecomunicaciones es un ejemplo. Todavía es posible crear un modelo del dominio en estos dominios, pero requiere un alto grado de abstracción y olvidarse de diseños familiares.

Por ejemplo, algunas de las clases conceptuales candidatas relacionadas con conmutadores de telecomunicaciones son: *Mensaje*, *Conexion*, *Puerto*, *Dialogo*, *Ruta*, *Protocolo*.

10.7. Clases conceptuales de especificación o descripción

La siguiente discusión podría, al principio, parecer relacionada con un tema raro y altamente especializado. Sin embargo, prueba que la necesidad de las clases conceptuales de especificación (como se definirán) es habitual en muchos modelos del dominio. Por tanto, se destaca.

Asuma lo siguiente:

- Una instancia de un *Articulo* representa un objeto físico de una tienda; como tal, podría incluso tener un número de serie.
- Un *Articulo* tiene una descripción, precio, e identificador del artículo (*articuloID*), que no se recogen en ningún otro sitio.
- Todo el mundo que trabaja en la tienda tiene amnesia.
- Cada vez que se vende un artículo físico real, se elimina una instancia de *Articulo* software correspondiente, del “terreno del software”.

Con estas suposiciones, ¿qué ocurre en el siguiente escenario?

Existe una fuerte demanda de las nuevas hamburguesas vegetales que han tenido mucho éxito —ObjetoHamburguesa—. La tienda las vende todas, lo que implica que se eliminan de la memoria del ordenador todas las instancias de *Articulo* de los ObjetoHamburguesa.

⁶ Nótese que en los primeros tiempos, un *registro* era únicamente una posible implementación del modo de registrar las ventas. El término ha adquirido un significado generalizado a lo largo del tiempo..

Ahora, aquí está lo esencial del problema: Si alguien pregunta: “¿Cuánto cuestan los ObjetosHamburguesa?”, nadie puede contestar, porque el precio se encontraba en las instancias inventariadas, que se eliminaron cuando se vendieron.

Nótese también que el modelo actual, si se implementa en el software tal y como se describe, contiene datos duplicados y es ineficiente en cuanto al espacio, puesto que la descripción, el precio y el ID, se duplican en cada instancia de *Articulo* del mismo producto.

La necesidad de especificación o descripción de las clases conceptuales

El problema anterior ilustra la necesidad de conceptos de objetos que sean especificaciones o descripciones de otras cosas. Para solucionar el problema del *Articulo*, lo que se necesita es una clase conceptual *EspecificacionDelProducto* (o *EspecificacionDelArticulo*, *DescripcionDelProducto*,...) que recoge la información sobre los artículos. Una *EspecificacionDelProducto* no representa un *Articulo*, sino una descripción de la información *sobre* los artículos. Nótese que incluso si todos los elementos inventariados se venden, y sus correspondientes instancias software de *Articulo* se eliminan, permanece todavía la *EspecificacionDelProducto*.

Los objetos de descripción o especificación están fuertemente relacionados con las cosas que describen. En un modelo del dominio, es típico establecer que una *EspecificacionDeX Describe un X* (ver Figura 10.7).

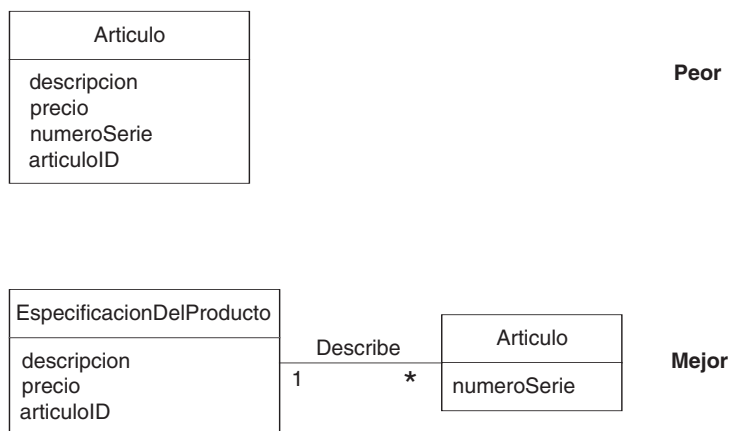


Figura 10.7. Especificaciones o descripciones sobre otras cosas. El “*” significa una multiplicidad de “muchos”. Indica que una *EspecificacionDelProducto* podría describir a muchos (*) *Articulos*.

La necesidad de clases conceptuales de especificación es habitual en los dominios de ventas y productos. También es típica en la fabricación, donde se requiere una *descripción* de la cosa que se fabrica, que es distinto de la cosa en sí misma. Se ha dedicado tiempo y espacio a motivar las clases conceptuales de especificación porque son muy frecuentes; no es un concepto de modelado raro.

¿Cuándo se requieren las clases conceptuales de especificación?

Las siguientes guías sugieren cuándo utilizar las especificaciones:

Añada una clase conceptual de especificación o descripción (por ejemplo, *EspecificacionDelProducto*) cuando:

- Se necesita la descripción de un artículo o servicio, independiente de la existencia actual de algún ejemplo de esos artículos o servicios.
- La eliminación de instancias de las cosas que describen (por ejemplo, *Articulo*) dan como resultado una pérdida de información que necesita mantenerse, debido a la asociación incorrecta de información con la cosa eliminada.
- Reduce información redundante o duplicada.

Otro ejemplo de especificación

Como otro ejemplo, considere una compañía aérea que sufre un accidente catastrófico de uno de sus aviones. Asuma que se cancelan todos los vuelos durante seis meses, pendiente de que se complete la investigación. También asuma que, cuando se cancelan los vuelos, su correspondiente objeto software *Vuelo*, se elimina de la memoria del ordenador. Por tanto, después de la colisión, se eliminan todos los objetos software *Vuelo*.

Si el único registro de los aeropuertos de destino de los vuelos se encuentra en las instancias software *Vuelo*, que representa un vuelo específico en una fecha y hora concreta, ya no habrá un registro de las rutas de los vuelos de la compañía.

Para solucionar este problema, se necesita una *DescripcionDelVuelo* (o *EspecificacionDelVuelo*) que describe un vuelo y su ruta, incluso cuando no se ha planificado ningún vuelo concreto (ver Figura 10.8).

Descripción de servicios

Nótese que el ejemplo anterior trata sobre un servicio (un vuelo) en lugar de un artículo (como una hamburguesa vegetal). Es habitual que se necesiten descripciones de servicios o planes de servicios.

Como otro ejemplo, una compañía de telefonía móvil vende paquetes con denominaciones como “bronce”, “oro”, etcétera. Es necesario tener el concepto de descripción del paquete (un tipo de plan de servicio que describe las tarifas por minuto, acceso a Internet sin cable, el coste, etcétera) separado del concepto del paquete realmente vendido (como el “paquete oro vendido a Craig Larman el 1 de enero de 2002, a \$55 al mes”). El departamento de Marketing necesita definir y registrar este plan de servicios o *DescripcionDelPaqueteDeComunicacionesMoviles* antes de que se venda alguno.

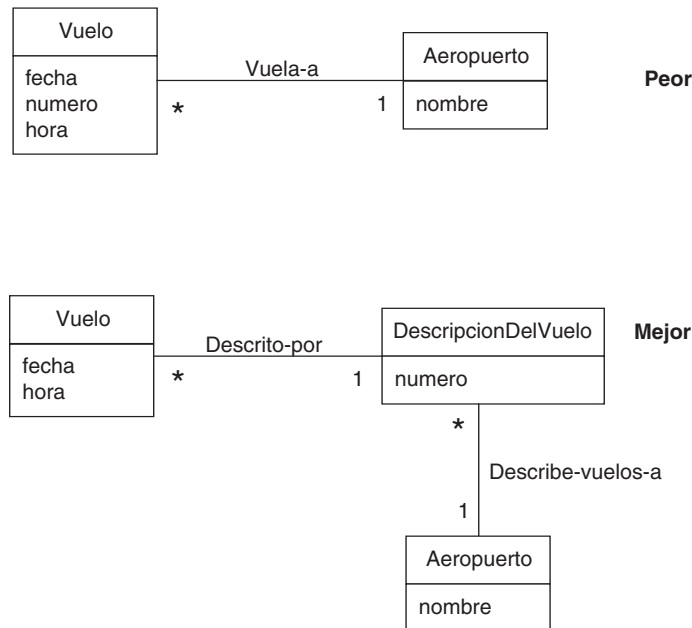


Figura 10.8. Especificaciones sobre otras cosas.

10.8. Notación UML, modelos y métodos: perspectivas múltiples

El UP define algo denominado Modelo del Dominio, que se representa con la notación UML. Sin embargo, no existe un término “Modelo del Dominio” en la documentación oficial de UML. Esto nos revela algo importante:

UML simplemente describe tipos de diagramas, como los diagramas de clases y los diagramas de secuencia. No superpone un método o perspectiva de modelado sobre ellos. Más bien, un proceso (como el UP) aplica la notación de la especificación de UML en el contexto de modelos definidos en el ámbito de una metodología.

Por ejemplo, la notación de los diagramas de clases de UML se puede utilizar para crear representaciones de las clases conceptuales del dominio (un modelo del dominio), clases software, tablas de una base de datos relacional, etcétera.

Por tanto, no debemos confundir la notación básica de los diagramas UML, con su aplicación para visualizar distintos tipos de modelos definidos por los metodólogos (ver Figura 10.9). Este punto es aplicable no sólo a los diagramas de clases UML, sino a la mayoría de la notación UML.

Como otro ejemplo de los diagramas de UML que se interpretan de manera diferente en modelos distintos, los diagramas de secuencia UML se pueden utilizar para representar el paso de mensajes entre los objetos software (como en el Modelo de Diseño del UP), o la interacción entre personas y grupos en el mundo real (como en el Modelo de Objetos del Negocio del UP).

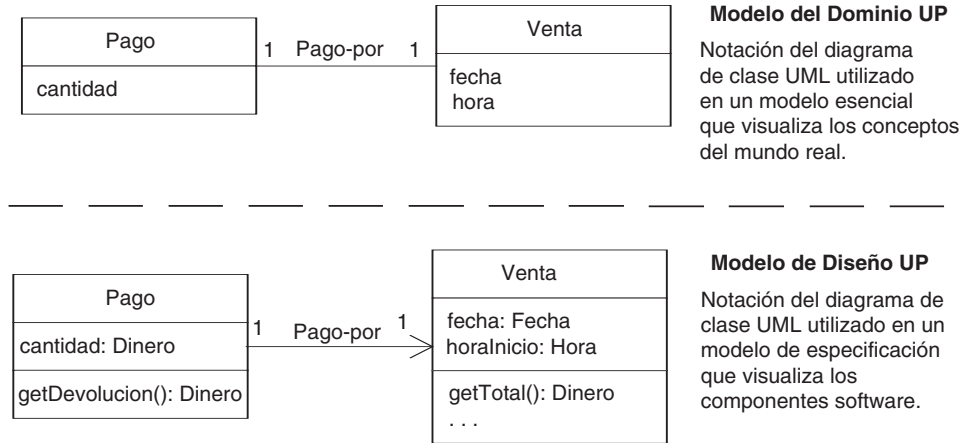


Figura 10.9. La notación de la especificación UML se puede aplicar en diferentes perspectivas y modelos definidos por un proceso o método.

Esta idea se pone de relieve en el método orientado a objetos Syntropy [CD94], y Martin Fowler la reitera en *UML Distilled* [FS00]. Es decir, la misma notación basada en diagramas se puede utilizar en tres perspectivas y tipos de modelos:

1. **Perspectiva esencial o conceptual:** se interpreta que los diagramas describen cosas del mundo real o de un dominio de interés.
2. **Perspectiva de especificación:** se interpreta que los diagramas (utilizando la misma notación de los modelos esenciales) describen abstracciones software o componentes con especificaciones e interfaces, pero no comprometidas a ninguna implementación en particular (por ejemplo, no específicamente una clase C# o Java).
3. **Perspectiva de implementación:** se interpreta que los diagramas (utilizando la misma notación de los modelos esenciales) describen implementaciones software con una tecnología y lenguaje particular (como Java).

Superposición de terminología: UML vs. métodos

En la especificación UML, las cajas rectangulares mostradas en la Figura 10.9 se denominan **clases**, pero nótese que en UML, este término abarca una variedad de fenómenos —cosas físicas, cosas del software, eventos, etcétera⁷—. Un proceso o método superpondrá una terminología alternativa sobre UML. Por ejemplo, en el UP, cuando las cajas de UML se dibujan en el Modelo del Dominio, podrían llamarse **conceptos del dominio** o **clases conceptuales**; el Modelo del Dominio ofrece una perspectiva conceptual. En el UP, cuando las cajas UML se dibujan en el Modelo de Diseño, se denominan oficialmente **clases de diseño**: el Modelo de Diseño ofrece una perspectiva de especificación o implementación, como quiere el modelador.

⁷ Una clase UML es un caso especial de un elemento de modelado UML muy general el **clasificador** —algo con características estructurales y/o comportamiento, incluyendo clases, actores, interfaces y casos de uso—.

Independientemente de la definición, la cuestión importante es que es útil distinguir entre la perspectiva de un analista que mira conceptos del mundo real tal como una venta (una perspectiva conceptual), y los diseñadores de software que especifican componentes software como la clase software *Venta* (una perspectiva de especificación o implementación).

UML se puede utilizar para ilustrar ambas perspectivas con una notación y perspectiva muy similar, así que es muy importante tener en cuenta qué perspectiva se está tomando.

Para mantener las cosas claras, este libro utilizará los términos relacionados con las clases de la siguiente forma, que es consistente con UML y el UP:

- **Clase conceptual:** concepto o cosa del mundo real. Una perspectiva conceptual o esencial. El Modelo del Dominio del UP contiene clases conceptuales.
- **Clase software:** una clase que representa una perspectiva de especificación o implementación de un componente software, independientemente del proceso o método.
- **Clase de diseño:** un miembro del Modelo de Diseño del UP. Es un sinónimo de clase software, pero por alguna razón deseo resaltar que es una clase del Modelo de Diseño. El UP permite que una clase de diseño tenga perspectiva de especificación o implementación, según desee el modelador.
- **Clase de implementación:** una clase implementada en un lenguaje orientado a objetos como Java.
- **Clase:** como en UML, el término general que representa o una cosa del mundo real (una clase conceptual) o del software (una clase software).

10.9. Reducción del salto en la representación

Por favor, mire atentamente la Figura 10.10. ¿Por qué los libros y educadores que presentan el diseño de objetos común sólo muestran el uso de clases software cuyos nombres reflejan el vocabulario del dominio? ¿Por qué eligen un nombre de clase software como *Venta*, y qué hace una *Venta*?

Simplemente, eligiendo nombres que reflejan el vocabulario del dominio (*Venta*) se favorece la rápida comprensión y proporciona una pista acerca de lo que se espera del trozo de código de la clase software *Venta*. Tenemos un modelo mental del dominio en cuestión (por ejemplo, una tienda que vende cosas). En el mundo real, sabemos que una venta tiene una fecha. En consecuencia, si creamos una clase Java llamada *Venta*, y le damos la responsabilidad de conocer sobre una venta real y su fecha, entonces la clase Java *Venta* se corresponde de alguna manera con nuestro modelo mental del dominio real; es decir, apela a nuestra “intuición” del dominio.

El Modelo del Dominio proporciona un diccionario visual del vocabulario y conceptos del dominio a partir de los cuales nos inspiramos para nombrar algunas cosas del diseño software.

Esto se relaciona con el tema del **salto de la representación** o salto semántico —el salto entre nuestro modelo mental del dominio y su representación en el software—.

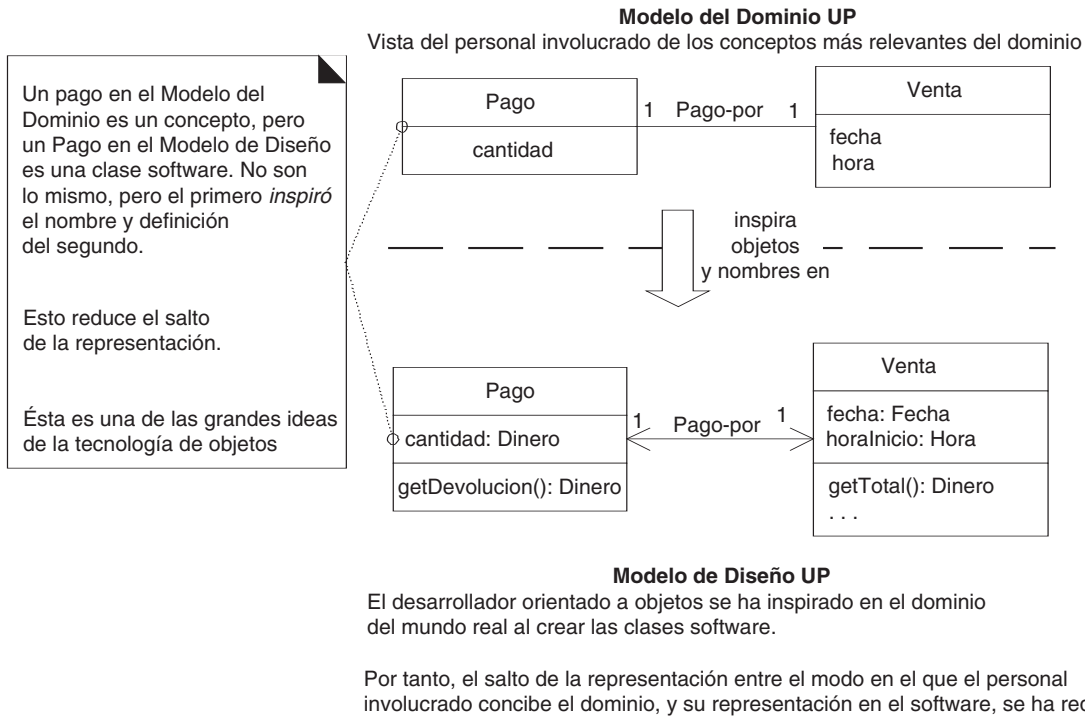


Figura 10.10. En el diseño y programación de objetos, es normal crear clases software cuyos nombres e información se inspira en el dominio del mundo real.

En un extremo, podríamos programar directamente la aplicación de PDV NuevaE-
ra en código binario para invocar el conjunto de instrucciones del procesador. Enten-
demos que el salto en las representaciones es enorme, y existirá en el software un coste
real —aunque difícil de cuantificar— con un salto en la representación tan grande,
porque es difícil de entender y relacionar con el dominio del problema. Cercano al otro
extremo del espectro, están las tecnologías de objetos que nos permiten partir el código en
clases cuyos nombres reflejan el tipo de partición que percibimos en el dominio. En el
mundo real percibimos una “parte” (o hecho) denominada venta, luego en el terreno del
software tenemos una clase software denominada *Venta*. Esta estrecha correspondencia
uno-a-uno entre el vocabulario del dominio y nuestro vocabulario del software y sus par-
ticiones reduce el salto de la representación. Esto acelera la comprensión del código exis-
tente (porque funciona de la manera que esperamos, conociendo el dominio) y sugiere
formas “naturales” para extender el código que se corresponden análogamente con el do-
minio, o apelan a nuestras intuiciones del dominio. Diciéndolo sencillamente, el modelo
software nos recuerda al modelo mental o conceptual, y funciona de manera predecible.

Los modelos software proporcionan una ventaja práctica al reducir el salto de la re-
presentación. La mayoría de los ingenieros del software saben que es verdad, incluso si
es difícil de cuantificar. De hecho, una prueba de esto es que algunos de los programa-
dores Java, de forma deliberada complican el código fuente para que sea difícil hacer in-
geniería inversa a partir de los bytecodes, cambiando los nombres de las clases y los mé-
todos Java de manera que sean ininteligibles y, por tanto, ya no evoquen nuestra visión
del dominio, aunque las estructuras de control y datos no se cambian.

Por supuesto, la tecnología de objetos es valiosa también porque puede favorecer el diseño de sistemas elegantes, débilmente acoplados, que pueden crecer y ampliarse fácilmente, como veremos en lo que queda del libro. Una reducción del salto en la representación es útil, pero se puede sostener que es secundario a la ventaja que ofrecen los objetos de facilitar los cambios y extensiones, y el soporte que ofrecen para el manejo y ocultación de la complejidad.

10.10. Ejemplo: el Modelo del Dominio del PDV NuevaEra

La lista de clases conceptuales generadas para el dominio del PDV NuevaEra se podría representar gráficamente (ver Figura 10.11) para mostrar el comienzo del Modelo del Dominio.



Figura 10.11. Modelo del Dominio inicial.

Las consideraciones acerca de los atributos y asociaciones del Modelo del Dominio se posponen a capítulos siguientes.

10.11. Modelos del Dominio en el UP

Como se sugiere en el ejemplo de la Tabla 10.2, un Modelo del Dominio, normalmente, se inicia y completa en la elaboración.

Inicio

Los modelos del dominio no se incentivan fuertemente en la fase de inicio, puesto que el propósito del inicio no es llevar a cabo un estudio serio, sino decidir si merece la pena un estudio más profundo en el proyecto, en una fase de elaboración.

Elaboración

El Modelo del Dominio se crea sobre todo durante las iteraciones de la elaboración, cuando la necesidad más importante es entender los conceptos relevantes y trasladar algunos a clases software durante el trabajo de diseño.

Tabla 10.2. Muestra de los artefactos UP y evolución temporal. c – comenzar; r - refinar.

<i>Disciplina</i>	<i>Artefacto Iteración →</i>	<i>Inicio I1</i>	<i>Elab. E1...En</i>	<i>Const. C1...Cn</i>	<i>Trans. T1...T2</i>
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso Visión Especificación Complementaria Glosario	c c c c	r r r r		
Diseño	Modelo de Diseño Documento de Arquitectura SW Modelo de Datos		c c c	r r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Aunque, irónicamente, se dedicarán un número significativo de páginas a explicar el modelado de los objetos del dominio, en manos experimentadas el desarrollo de un modelo del dominio (parcial, desarrollado incrementalmente) en cada iteración debería durar sólo unas pocas horas. Esto se acorta mediante el uso de patrones de análisis predefinidos.

El Modelo de Objetos del Negocio del UP vs. el Modelo del Dominio

El Modelo del Dominio del UP es una variación oficial del menos común Modelo de Objetos del Negocio del UP (BOM, *Business Object Model*). El BOM del UP —no confundir con el modo en el que otras personas o métodos podrían definir un BOM, que es un término ampliamente utilizado con significados diferentes— es un tipo de modelo de empresa utilizado para describir el negocio completo. Podría utilizarse al llevar a cabo la ingeniería o reingeniería de procesos del negocio, independiente de cualquier aplicación software (como el PDV NuevaEra). Citando textualmente:

[El BOM del UP] sirve como abstracción del modo en el que los trabajadores y las entidades del negocio necesitan relacionarse y cómo necesitan colaborar para llevar a cabo el negocio. [RUP]

El BOM se representa con varios diagramas diferentes (clase, actividad y secuencia) que muestran cómo funciona (o debería funcionar) toda la empresa. Es más útil si se realiza ingeniería de procesos de negocio por toda la empresa, pero ésta es una actividad menos común que la creación de una única aplicación software.

En consecuencia, el UP define el Modelo del Dominio como un artefacto subconjunto o una especialización del BOM, que se crea normalmente. Citando textualmente:

Puedes elegir desarrollar un modelo de objetos del negocio “incompleto”, enfocado a explicar “cosas” y productos importantes para un dominio. ... A menudo se hace referencia a éste como modelo del dominio. [RUP]

10.12. Lecturas adicionales

Object-Oriented Methods: A Foundation de Odell proporciona una sólida introducción al modelado del dominio conceptual. También resulta útil *Design Object Systems* de Cook and Daniel.

Analysis Patterns de Fowler ofrece importantes patrones de los modelos del dominio, y se recomienda sin ninguna duda. Otro libro bueno, que describe patrones de los modelos del dominio es *Data Model Patterns: Conventions of Thought* de Hay. Los consejos de los expertos en el modelado de datos, que entienden la distinción entre los modelos conceptuales puros y los modelos de los esquemas de bases de datos, pueden ser muy útiles para el modelado de los objetos del dominio.

Java Modeling in Color with UML [CDL99] contiene más consejos relevantes sobre el modelado del dominio de lo que sugiere el título. El autor identifica patrones comunes en los tipos relacionados y sus asociaciones; el aspecto de color es realmente una visualización de las categorías típicas de estos tipos, como *descripción* (azul), *roles* (amarillo), y *momento-intervalos* (rosa). El color se utiliza para ayudar a ver los patrones.

Desde el trabajo original de Abbot, el análisis lingüístico ha adquirido técnicas más sofisticadas para el análisis orientado a objetos, generalmente denominado modelado del lenguaje natural, o una variante. Véase [Moreno97] como un ejemplo.

10.13. Artefactos del UP

La influencia entre los artefactos, destacando el Modelo del Dominio, se muestra en la Figura 10.12.

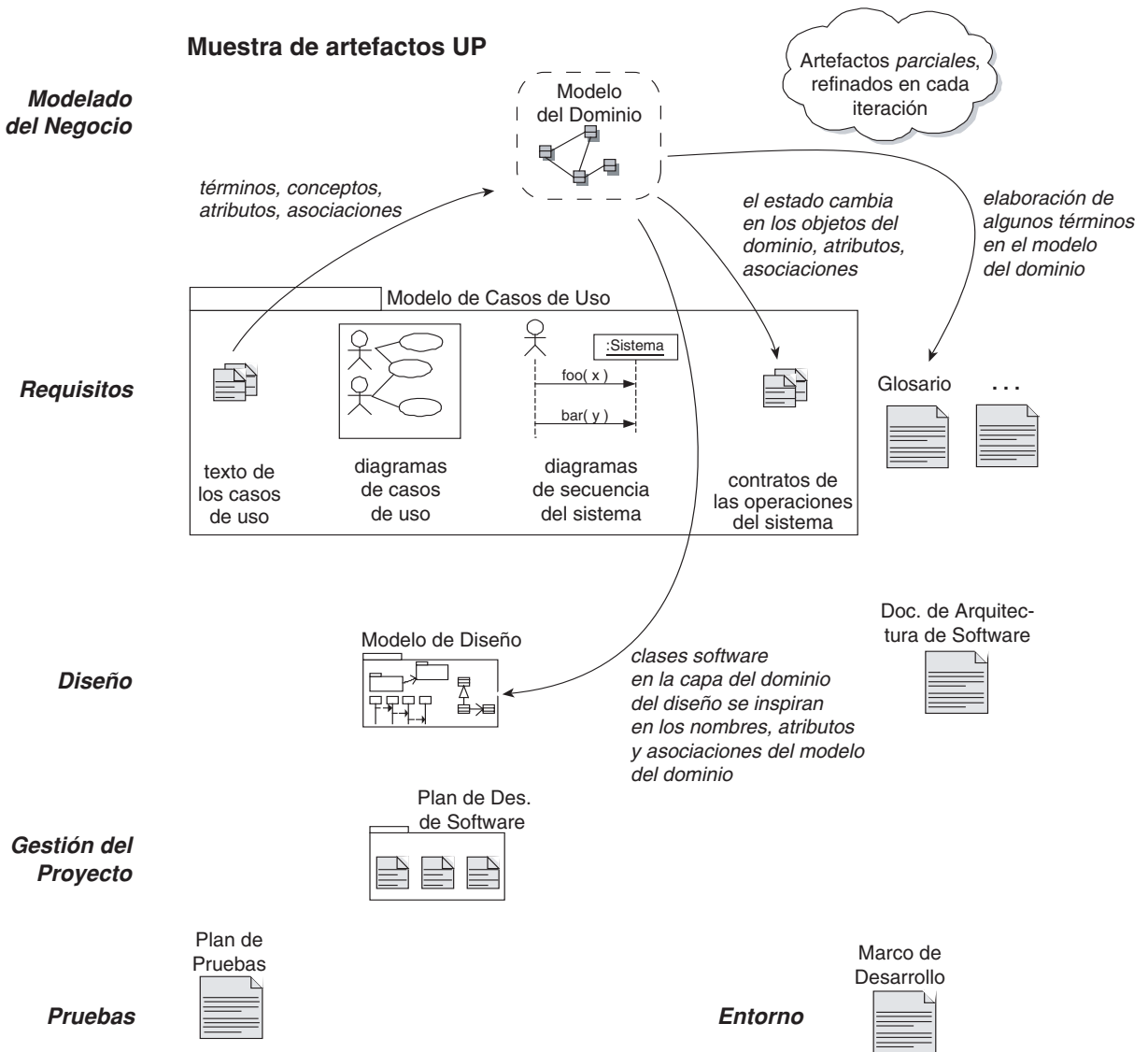


Figura 10.12. Muestra de la influencia entre los artefactos UP.

MODELO DEL DOMINIO: AÑADIR ASOCIACIONES

Objetivos

- Identificar las asociaciones del modelo del dominio.
- Distinguir entre las asociaciones necesito-conocer y sólo-comprensión.

Introducción

Resulta útil identificar aquellas asociaciones entre clases conceptuales que son necesarias para satisfacer los requisitos de información de los escenarios actuales que se están desarrollando, y que ayudan a entender el modelo del dominio. Este capítulo explora la identificación de las asociaciones adecuadas, y añade las asociaciones al modelo del dominio del caso de estudio NuevaEra.

11.1. Asociaciones

Una **asociación** es una relación entre tipos (o más concretamente, instancias de estos tipos) que indica alguna conexión significativa e interesante (ver Figura 11.1).

En UML, las asociaciones se definen como “la relación semántica entre dos o más clasificadores que implica conexiones entre sus instancias”.

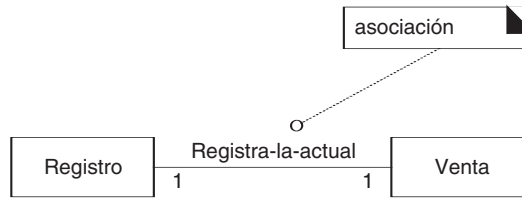


Figura 11.1. Asociaciones.

Criterio para asociaciones útiles

Las asociaciones que merece la pena registrar, normalmente, implican conocimiento de una relación que es necesario conservar durante algún tiempo —podría ser milisegundos o años, dependiendo del contexto—. En otras palabras, ¿entre qué objetos necesitamos mantener en memoria una relación? Por ejemplo, ¿necesitamos recordar qué instancias de *LineaDeVenta* están asociadas con una instancia de *Venta*? Sin ninguna duda, de otra manera no sería posible reconstruir una venta, imprimir un recibo o calcular el total de la venta.

Considere la inclusión de las siguientes asociaciones en un modelo del dominio:

- Asociaciones de las que es necesario conservar el conocimiento de la relación durante algún tiempo (asociaciones “necesito-conocer”).
- Asociaciones derivadas de la Lista de Asociaciones Comunes.

Por el contrario, ¿necesitamos recordar una relación entre la *Venta* actual y un *Encargado*? No, los requisitos no dan a entender que se necesite ninguna relación de este tipo. No es incorrecto mostrar una relación entre una *Venta* y un *Encargado*, pero no es convincente o útil en el contexto de nuestros requisitos.

Éste es un punto importante. En un modelo del dominio con n clases del dominio diferentes, pueden existir $n(n-1)$ asociaciones entre diferentes clases conceptuales —un número potencialmente grande—. Muchas líneas en un diagrama añadirán “ruido visual” y lo hará menos comprensible. Por tanto, sea cuidadoso al añadir líneas de asociación. Utilice como criterio las guías que se sugieren en este capítulo.

11.2. Notación de las asociaciones en UML

Una asociación se representa como una línea entre clases con un nombre de asociación. La asociación es inherentemente bidireccional, lo que significa que desde las instancias de cualquiera de las dos clases, es posible el recorrido lógico hacia la otra.

Este recorrido es puramente abstracto; *no* se trata de una sentencia sobre conexiones entre entidades software.

Los extremos de la asociación podrían contener una expresión de multiplicidad que indica la relación numérica entre las instancias de las clases.

Una “flecha de dirección de lectura” opcional indica la dirección de la lectura del nombre de la asociación; no indica la dirección de la visibilidad o navegación.

Si no está presente, la convención es leer la asociación de izquierda a derecha o de arriba hacia abajo, aunque no es una regla de UML (ver Figura 11.2).

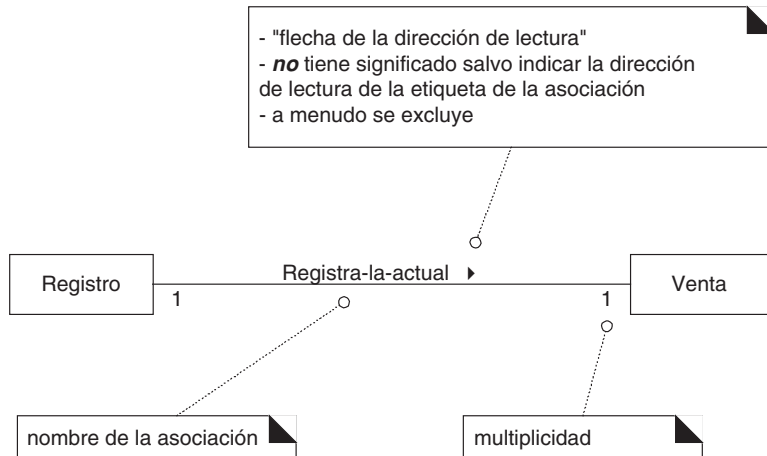


Figura 11.2. Notación UML para las asociaciones.

La flecha de dirección de lectura no tiene significado en términos del modelo; sólo es una ayuda para el lector del diagrama.

11.3. Localización de las asociaciones—lista de asociaciones comunes

Comience la inclusión de asociaciones utilizando la lista de la Tabla 11.1.

Contiene categorías comunes que, normalmente, merece la pena tener en cuenta. Los ejemplos se han extraído de los dominios de ventas y reservas de vuelos.

Tabla 11.1. Lista de asociaciones comunes.

Categoría	Ejemplos
A es una parte física de B	<i>Cajon-Registro (o más concretamente, TPDV) Ala-Avion</i>
A es una parte lógica de B	<i>LineaDeVenta-Venta</i> <i>EtapasVuelo-RutaVuelo</i>
A está contenido físicamente en B	<i>Registo-Tienda, Artículo-Estanteria</i> <i>Pasajero-Avion</i>
A está contenido lógicamente en B	<i>DescripcionDelArticulo-Catalogo</i> <i>Vuelo-PlanificacionVuelo</i>

continúa

Tabla 11.1. Lista de asociaciones comunes. (Continuación)

<i>Categoría</i>	<i>Ejemplos</i>
A es una descripción de B	<i>DescripcionDelArticulo-Articulo</i> <i>DescripcionDelVuelo-Vuelo</i>
A es un línea de una transacción o informe de B	<i>LineaDeVenta-Venta</i> <i>TrabajoMantenimiento-RegistroDe-Mantenimiento</i>
A se conoce/registra/recoge/informa/captura en B	<i>Venta-Registro</i> <i>Reserva-ListaPasajeros</i>
A es miembro de B	<i>Cajero-Tienda</i> <i>Piloto-CompañíaAerea</i>
A es una subunidad organizativa de B	<i>Departamento-Tienda</i> <i>Mantenimiento-CompañíaAerea</i>
A utiliza o gestiona B	<i>Cajero-Registro</i> <i>Piloto-Avion</i>
A se comunica con B	<i>Cliente-Cajero</i> <i>AgenteDeReservas-Pasajero</i>
A está relacionado con una transacción B	<i>Cliente-Pago</i> <i>Pasajero-Billete</i>
A es una transacción relacionada con otra transacción B	<i>Pago-Venta</i> <i>Reserva-Cancelacion</i>
A está al lado de B	<i>LineaDeVenta-LineaDeVenta</i> <i>Ciudad-Ciudad</i>
A es propiedad de B	<i>Registro-Tienda</i> <i>Avion-CompañíaAerea</i>
A es un evento relacionado con B	<i>Venta-Cliente, Venta-Tienda</i> <i>Salida-Vuelo</i>

Asociaciones de prioridad alta

A continuación, presentamos algunas de las categorías de asociaciones de prioridad alta que son, invariablemente, útiles incluirlas en un modelo del dominio:

- A es una *parte lógica o física* de B.
- A está *contenida física o lógicamente* en B.
- A *se registra* en B.

11.4. Guías para las asociaciones

- Céntrese en aquellas asociaciones para las que se necesita conservar el conocimiento de la relación durante algún tiempo (asociaciones “necesito-conocer”).
- Es más importante identificar *clases conceptuales* que identificar asociaciones.

- Demasiadas asociaciones tienden a confundir un modelo del dominio en lugar de aclararlo. Su descubrimiento puede llevar tiempo, con beneficio marginal.
- Evite mostrar asociaciones redundantes o derivadas.

11.5. Roles

Cada extremo de una asociación se denomina **rol**. Los roles pueden tener opcionalmente:

- Nombre.
- Expresión de multiplicidad.
- Navegabilidad.

La multiplicidad se presenta a continuación, y las otras dos características se discutirán en capítulos posteriores.

Multiplicidad

La **multiplicidad** define cuántas instancias de una clase A pueden asociarse con una instancia de una clase B (ver Figura 11.3).

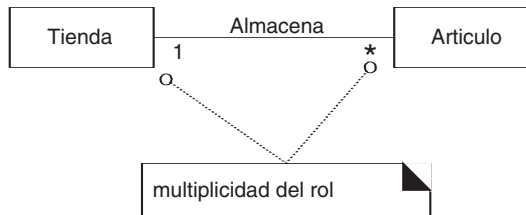


Figura 11.3. Multiplicidad de una asociación.

Por ejemplo, una instancia individual de una *Tienda* puede asociarse con “muchas” (cero o más, indicado por el “*”) instancias de *Artículo*.

En la Figura 11.4 se muestran algunos ejemplos de expresiones de multiplicidad.

El valor de la multiplicidad indica cuántas instancias se puede asociar legalmente con otra, en un momento concreto, en lugar de a lo largo de un periodo de tiempo. Por ejemplo, es posible que un coche usado pudiera a lo largo del tiempo ser vendido repetidamente a comerciantes de coches usados. Pero en un momento concreto, el coche sólo es *Abastecido-por* un comerciante. El coche no es *Abastecido-por* muchos comerciantes en un momento concreto. Análogamente, en países con leyes monógamas, una persona puede estar *Casado-con* sólo una persona en un momento dado, aunque a lo largo del tiempo, puedan casarse con muchas personas.

El valor de la multiplicidad depende de nuestros intereses como modeladores y desarrolladores de software, porque pone de manifiesto una restricción de diseño que será (o podrá ser) reflejada en el software. La Figura 11.5 muestra un ejemplo y explicación.

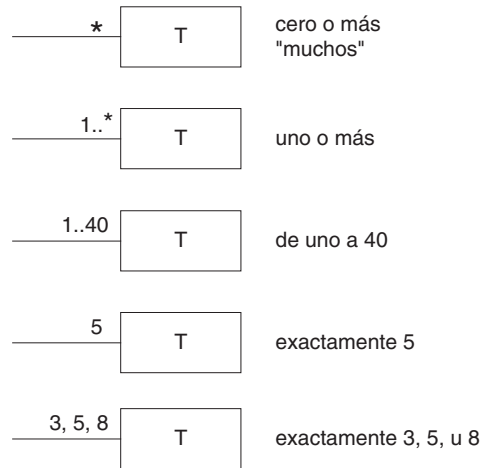
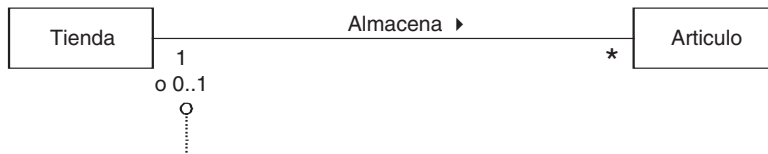


Figura 11.4. Valores de la multiplicidad.



¿La multiplicidad debería ser “1” o “0..1”?

La respuesta depende de nuestro interés al utilizar el modelo. Típica y prácticamente, la multiplicidad denota una restricción del dominio que nos preocupamos de ser capaces de comprobar en el software, si esta relación se implementase o reflejase en los objetos software o en la base de datos. Por ejemplo, un concreto artículo podría llegar a venderse o desecharse, y por tanto, no se almacenaría en la tienda por más tiempo. Desde este punto de vista, “0..1” es lógico, pero...

¿Nos preocupa este punto de vista? Si se implementa esta relación en el software, probablemente querríamos asegurar que una instancia software de *Articulo* siempre se relacionara con una instancia concreta de *Tienda*, de otra manera, indica un error o corrupción en los elementos software o datos.

Este modelo de dominio parcial no representa objetos software, sino que las multiplicidades registran restricciones cuyo valor práctico está, normalmente, relacionado con nuestro interés en la construcción del software o bases de datos (que reflejan nuestro dominio del mundo real) con comprobaciones de validez. Desde este punto de vista, el valor deseable podría ser “1”.

Figura 11.5. La multiplicidad es dependiente del contexto.

Rumbaugh proporciona otro ejemplo de *Persona* y *Compañía* en la asociación *Trabaja-para* [Rumbaugh91]. El que indiquemos si una instancia de *Persona* trabaja para una o varias instancias de *Compañía*, es dependiente del contexto del modelo; el departamento de impuestos está interesado en *muchas*; un sindicato probablemente sólo en *una*. Normalmente, la elección, prácticamente, depende de para quién estamos construyendo el software, y de esta manera, las multiplicidades válidas de una implementación.

11.6. ¿Cómo de detalladas deben ser las asociaciones?

Las asociaciones son importantes, pero un error típico al crear los modelos del dominio, es dedicar demasiado tiempo durante el estudio intentando descubrirlas.

Es fundamental entender lo siguiente:

Es más importante encontrar las *clases conceptuales* que las asociaciones. La mayoría del tiempo dedicado a la creación del modelo del dominio debería emplearse en la identificación de las clases conceptuales, no de las asociaciones.

11.7. Asignación de nombres a las asociaciones

Nombre una asociación en base al formato *NombreTipo-FraseVerbal-NombreTipo* donde la frase verbal crea una secuencia que es legible y tiene significado en el contexto del modelo.

Los nombres de las asociaciones deben comenzar con una letra mayúscula, puesto que una asociación representa un clasificador de enlace entre las instancias; en UML, los clasificadores deben comenzar con una letra mayúscula. Dos formatos típicos e igualmente válidos para un nombre de asociación compuesta son:

- *Pagado-mediante.*
- *PagadoMediante.*

En la Figura 11.6, la dirección por defecto para la lectura de los nombres de las asociaciones es de izquierda a derecha o de arriba abajo. No se corresponde con la dirección por defecto en UML, sino que se trata de una convención común.

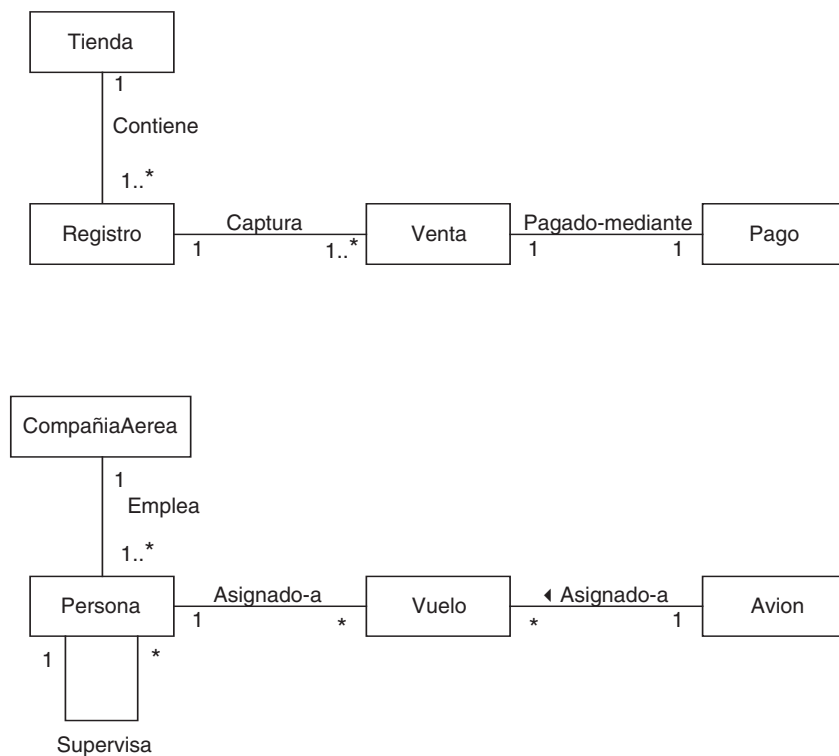


Figura 11.6. Nombres de asociaciones.

11.8. Múltiples asociaciones entre dos tipos

Dos tipos podrían tener múltiples asociaciones entre ellos; no es extraño. No existe ningún ejemplo destacado en nuestro caso de estudio del PDV, pero un ejemplo del dominio de la compañía aérea podría ser las relaciones entre un *Vuelo* (o quizás de manera más precisa, una *Etapavuelo*) y un *Aeropuerto* (ver Figura 11.7); las asociaciones *vuela-a* y *vuela-desde* son relaciones claramente diferentes, que se deberían mostrar de manera separada.

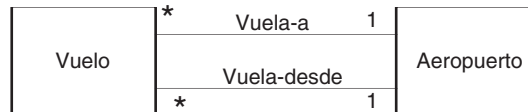


Figura 11.7. Múltiples asociaciones.

11.9. Asociaciones e implementación

Durante el modelado del dominio, una asociación *no* es una declaración sobre el flujo de datos, variables de instancia o conexiones entre objetos en una solución software; es una manifestación de que una relación es significativa en un sentido puramente conceptual —en el mundo real—. Desde un punto de vista práctico, muchas de estas relaciones se implementarán generalmente en software como caminos de navegación y visibilidad (tanto en el Modelo de Diseño como en el Modelo de Datos), pero su presencia en una vista conceptual (o esencial) de un modelo de dominio no requiere su implementación.

Al crear un modelo de dominio, podríamos definir asociaciones que no son necesarias durante la implementación. A la inversa, podríamos descubrir asociaciones que necesitan implementarse pero que se obviaron durante el modelado del dominio. En estos casos, el modelo del dominio puede actualizarse para reflejar estos descubrimientos.

Sugerencia

¿Deberían actualizarse los anteriores modelos estudiados, como el modelo del dominio, con las apreciaciones descubiertas durante el trabajo de implementación? No se moleste a menos que haya algún uso práctico del modelo en el futuro. Si es únicamente (como es el caso algunas veces) un artefacto temporal que se utiliza para inspirar las etapas siguientes, y no se utilizará de manera significativa más adelante, ¿por qué actualizarlo? Evite hacer o actualizar cualquier modelo o documentación a menos que exista una justificación concreta para su futura utilización.

Posteriormente estudiaremos formas de implementar la asociaciones en un lenguaje de programación orientado a objetos (los más habitual es utilizar un atributo que referencia una instancia de la clase asociada), pero de momento, es conveniente pensar en ellas como expresiones puramente conceptuales, *no* declaraciones sobre la solución de base de datos o software. Como siempre, al posponer las consideraciones de diseño se nos libera de informaciones y decisiones extrañas mientras hacemos un estudio de “análisis” puro y maximiza nuestras opciones de diseño más adelante.

11.10. Asociaciones del Modelo del Dominio del PDV NuevaEra

Ahora podemos añadir las asociaciones a nuestro modelo del dominio del PDV. Deberíamos añadir aquellas asociaciones que los requisitos (por ejemplo, los casos de uso) sugieren o implican una necesidad de recordar, o que, de otra manera, recomienda fuertemente nuestra percepción del dominio del problema. Cuando abordamos un nuevo problema, las categorías comunes de asociaciones, presentadas anteriormente, deberían revisarse y tenerse en cuenta, puesto que representan muchas de las asociaciones relevantes que normalmente necesitan recogerse.

Relaciones evidentes de la Tienda

La siguiente muestra de asociaciones es justificable en términos de necesito-conocer. Se basa en los casos de uso que se están considerando actualmente.

<i>Registro Registra Venta</i>	Para conocer la venta actual, generar el total, imprimir recibo
<i>Venta Pagada-mediante Pago</i>	Para conocer si se ha pagado la venta, relacionar la cantidad entregada con el total de la venta, e imprimir un recibo
<i>CatalogoDeProductos Registra EspecificacionDelProducto</i>	Para recuperar una <i>Especificacion DelProducto</i> a partir de un articuloID

Aplicación de la lista de comprobación de categorías de asociaciones

Recorreremos la lista de comprobación, basada en los tipos identificados previamente, considerando los requisitos del caso de uso actual.

<i>Categoría</i>	<i>Sistema</i>
A es una parte física de B	<i>Registro—Caja</i>
A es una parte lógica de B	<i>LineaDeVenta—Venta</i>
A está contenido físicamente en B	<i>Registro—Tienda</i> <i>Articulo—Tienda</i>
A está contenido lógicamente en B	<i>EspecificacionDelProducto—Catalogo-DeProductos</i> <i>CatalogoDeProductos—Tienda</i>
A es una descripción de B	<i>EspecificacionDelProducto-Articulo</i>

continúa

<i>Categoría</i>	<i>Sistema</i>
A es una línea de una transacción o informe de B	<i>LineaDeVenta—Venta</i>
A se conoce/registra/recoge/informa/captura en B	<i>(completa)Venta—Tienda</i> <i>(actual)Venta—Registro</i>
A es miembro de B	<i>Cajero—Tienda</i>
A es una subunidad organizativa de B	<i>no aplicable</i>
A utiliza o gestiona B	<i>Cajero—Registro</i> <i>Encargado—Registro</i> <i>Encargado—Cajero, aunque probablemente no es aplicable</i>
A se comunica con B	<i>Cliente—Cajero</i>
A está relacionado con una transacción B	<i>Cliente—Pago</i> <i>Cajero—Pago</i>
A es una transacción relacionada con otra transacción B	<i>Pago—Venta</i>
A está al lado de B	<i>LineaDeVenta—LineaDeVenta</i>
A es propiedad de B o informe de B	<i>Registro—Tienda</i>

11.11. Modelo del Dominio del PDV NuevaEra

El modelo del dominio de la Figura 11.8 muestra un conjunto de clases conceptuales y asociaciones que son candidatas para nuestra aplicación de PDV. Las asociaciones se derivaron sobre todo a partir de la lista de comprobación de asociaciones candidatas.

¿Conservamos sólo las asociaciones necesito-conocer?

El conjunto de asociaciones que se muestran en el modelo del dominio de la Figura 11.8 se derivaron de forma mecánica, la mayor parte, a partir de la lista de comprobación de asociaciones. Sin embargo, sería deseable que fuéramos más cuidadosos con las asociaciones que incluimos en nuestro modelo del dominio. Vista como una herramienta de comunicación, no es deseable sobrecargar el modelo del dominio con asociaciones que no se necesitan forzosamente y que no favorecen nuestra comprensión. Demasiadas asociaciones no imprescindibles oscurecen en lugar de aclarar.

Como se sugirió previamente, se recomienda el siguiente criterio para mostrar las asociaciones:

- Céntrese en aquellas asociaciones para las que el conocimiento de la relación necesita conservarse durante algún tiempo (asociaciones “necesito-conocer”).
- Evite mostrar asociaciones redundantes o derivadas.

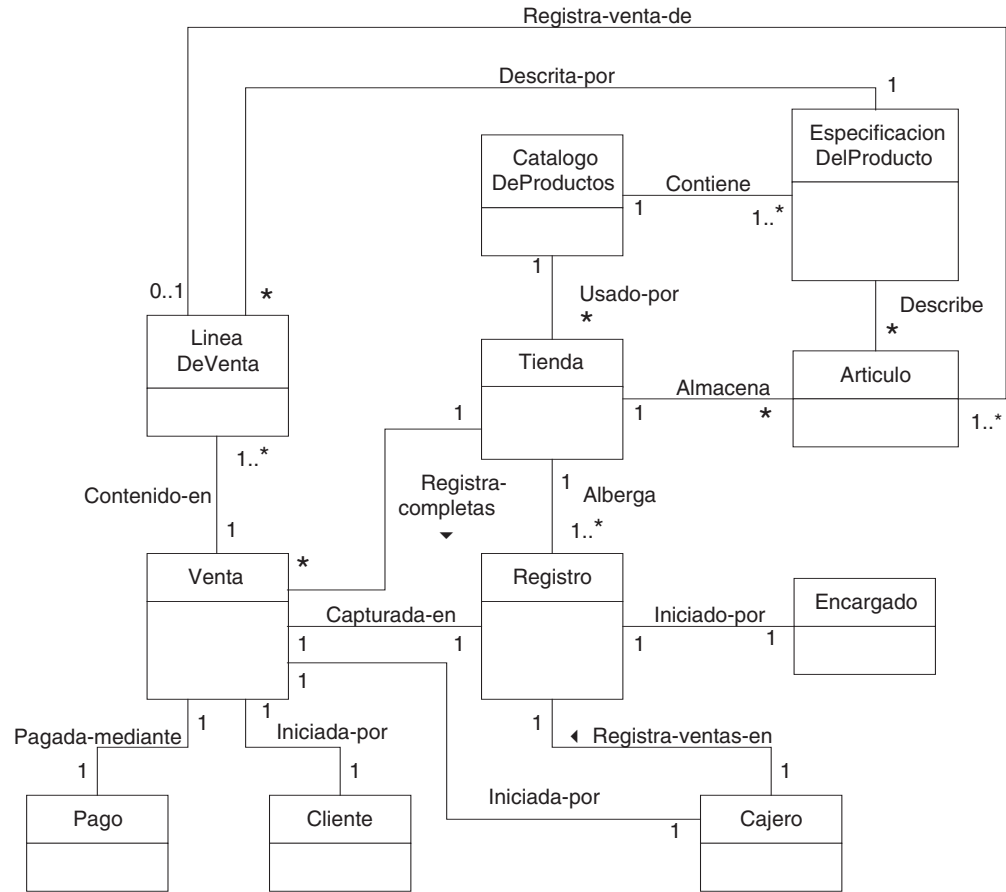


Figura 11.8. Modelo del dominio parcial.

En base a este consejo, no todas las asociaciones que se acaban de mostrar son imprescindibles. Considere lo siguiente:

Asociación	Comentario
<i>Venta Insertada-por Cajero</i>	Los requisitos no indican que necesito-conocer o registrar al cajero actual. Además, puede derivarse si está presente la asociación <i>Registro Usado-por Cajero</i> .
<i>Registro Usado-por Cajero</i>	Los requisitos no indican que necesito-conocer o registrar al cajero actual.
<i>Registro Iniciado-por Encargado</i>	Los requisitos no indican que necesito-conocer o registrar al encargado que pone en marcha un <i>Registro</i> .
<i>Venta Iniciada-por Cliente</i>	Los requisitos no indican que necesito-conocer o registrar al cliente que inicia una venta.
<i>Tienda Almacena Articulo</i>	Los requisitos no indican que necesito-conocer o mantener información del inventario.
<i>LineaDeVenta Registra-venta-de Articulo</i>	Los requisitos no indican que necesito-conocer o mantener información del inventario.

Nótese que la capacidad de justificar una asociación en función de necesito-conocer depende de los requisitos; obviamente, un cambio en éstos —como que se necesite mostrar en el recibo el ID del cajero— cambia la necesidad de recordar una relación.

Basado en el análisis anterior, *podría* justificarse la eliminación de las asociaciones en cuestión.

Asociaciones necesito-conocer vs. comprensión

Un criterio necesito-conocer estricto para el mantenimiento de las asociaciones generará un “modelo de información” mínimo de lo que se necesita para modelar el dominio del problema —limitado por los requisitos actuales que se están considerando—. Sin embargo, este enfoque podría crear un modelo que no transmite (a nosotros o a los demás) una comprensión completa del dominio.

Además de ser un modelo necesito-conocer de información sobre las cosas, el modelo del dominio es una herramienta de comunicación con la que estamos intentando entender y comunicar a otros los conceptos importantes y sus relaciones. Desde este punto de vista, eliminando algunas asociaciones que no se exigen estrictamente en una base necesito-conocer, puede crear un modelo sin interés —no comunica las ideas claves y relaciones—.

Por ejemplo, en la aplicación del PDV: aunque, tomando como base las relaciones necesito-conocer de manera estricta, podría no ser necesario registrar *Venta Iniciada-por Cliente*, su ausencia deja fuera un aspecto importante para entender el dominio —que un cliente genera las ventas—.

En cuanto a las asociaciones, un buen modelo se sitúa en alguna parte entre un modelo necesito-conocer mínimo y uno que ilustra cada relación concebible. ¿El criterio básico para juzgar su valor? —¿Satisface todos los requisitos necesito-conocer y además comunica claramente un conocimiento esencial de los conceptos importantes en el dominio del problema?—.

Céntrese en las asociaciones necesito-conocer, pero contemple las asociaciones de sólo-comprensión para enriquecer el conocimiento básico del dominio.

MODELO DEL DOMINIO: AÑADIR ATRIBUTOS

Cualquier error tardío es indistinguible de una característica.

Rich Kulawiec

Objetivos

- Identificar los atributos del modelo del dominio.
- Distinguir entre atributos correctos e incorrectos.

Introducción

Resulta útil identificar aquellos atributos de las clases conceptuales que se necesitan para satisfacer los requisitos de información de los actuales escenarios en estudio. Este capítulo explora la identificación de los atributos adecuados, y añade los atributos al modelo del dominio de NuevaEra.

12.1. Atributos

Un **atributo** es un valor de datos lógico de un objeto.

Incluya los siguientes atributos en un modelo del dominio: aquellos para los que los requisitos (por ejemplo, los casos de uso) sugieren o implican una necesidad de registrar la información.

Por ejemplo, un recibo (que recoge la información de una venta) normalmente incluye una fecha y una hora, y la dirección quiere conocer las fechas y horas de las ventas por múltiples motivos. En consecuencia, la clase conceptual *Venta* necesita los atributos *fecha* y *hora*.

12.2. Notación de los atributos en UML

Los atributos se muestran en el segundo compartimento del rectángulo de la clase (ver Figura 12.1). Sus tipos podrían mostrarse opcionalmente.

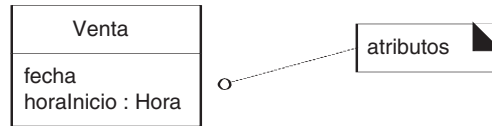


Figura 12.1. Clases y atributos.

12.3. Tipos de atributos válidos

Hay algunas cosas que no deberían representarse como atributos, sino como asociaciones. Esta sección presenta los tipos válidos.

Mantenga atributos simples

Intuitivamente, la mayoría de los atributos simples son los que, a menudo, se conocen como tipos de datos primitivos, como los números. El tipo de un atributo, normalmente, no debería ser un concepto de dominio complejo, como *Venta* o *Aeropuerto*. Por ejemplo, el siguiente atributo *registroActual* en la clase *Cajero* de la Figura 12.2, no es deseable porque su tipo tiene la intención de ser un *Registro*, que no es un tipo de atributo simple (como *Numero* o *String*). La manera más útil para expresar que un *Cajero* utiliza un *Registro* es con una asociación, no con un atributo.

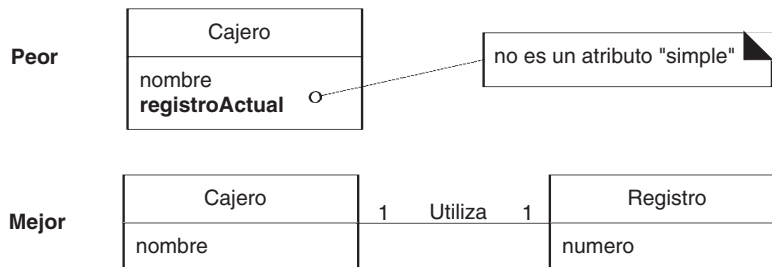


Figura 12.2. Relaciones con asociaciones, no atributos.

Los atributos en un modelo del dominio deberían ser, preferiblemente, **atributos simples** o **tipos de datos**.

Los tipos de datos de los atributos muy comunes incluyen: *Boolean*, *Fecha*, *Numero*, *String (Texto)*, *Hora*.

Otros tipos comunes comprenden: *Direccion*, *Color*, *Geometrico (Punto, Rectangulo)*, *Numero de Telefono*, *Numero de la Seguridad Social*, *Codigo de Producto Universal (UPC; Universal Product Code)*, *SKU*, *ZIP* o *códigos postales*, *tipos enumerados*.

Repitiendo un ejemplo previo, un error típico es modelar un concepto del dominio complejo como un atributo. Para ilustrarlo, un aeropuerto de destino no es realmente una cadena de texto; se trata de una cosa compleja que ocupa muchos kilómetros cuadrados de espacio. Por tanto, *Vuelo* debería relacionarse con *Aeropuerto* mediante una asociación, no con un atributo, como se muestra en la Figura 12.3.

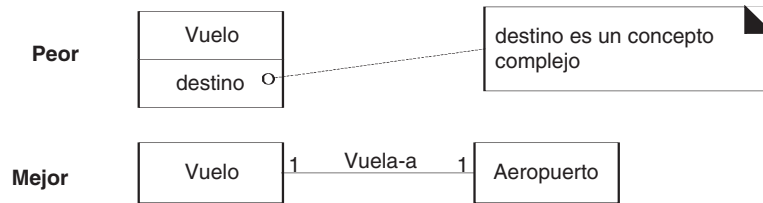


Figura 12.3. Evite la representación de conceptos del dominio complejos como atributos; utilice asociaciones.

Relacione las clases conceptuales con una asociación, no con un atributo.

Perspectiva conceptual vs. implementación: ¿qué sucede con los atributos en el código?

La restricción de que el tipo de los atributos en el modelo del dominio sea sólo un tipo de datos simple *no* implica que los atributos C++ o Java (miembros de datos, campos de una instancia) sólo deban ser de tipos de datos primitivos, o simples. El modelo del dominio se centra en declaraciones conceptuales puras sobre un dominio del problema, no en componentes software.

Posteriormente, durante el trabajo de diseño e implementación, se verá que las asociaciones entre objetos representadas en el modelo del dominio, a menudo, se implementarán como atributos que referencian a otros objetos software complejos. Sin embargo, ésta es sólo una de las posibles soluciones de diseño para implementar una asociación, y, de ahí que la decisión se deba posponer durante el modelado del dominio.

Tipos de datos

Los atributos deben ser, generalmente, **tipos de datos**. Esto es un término UML que implica un conjunto de valores para los cuales no es significativa una identidad única (en el contexto de nuestro modelo o sistema) [RJB99]. Por ejemplo, no es (normalmente) significativo distinguir entre:

- Diferentes instancias del *Numero* 5.
- Diferentes instancias del *String* 'gato'.

- Diferentes instancias del *NumeroDeTelefono* que contiene el mismo número.
- Diferentes instancias de la *Direccion* que contiene la misma dirección.

Por el contrario, es significativo distinguir (por identidad) entre dos instancias de *Persona* cuyos nombres son, en los dos casos, “Luis García”, puesto que las dos instancias pueden representar individuos diferentes con el mismo nombre.

En cuanto al software, existen pocas situaciones donde uno compararía las direcciones de memoria de las instancias de *Numero*, *String*, *NumeroDeTelefono* o *Direccion*; sólo son relevantes las comparaciones basadas en los valores. Por el contrario, es comprensible comparar las direcciones de memoria de las instancias de *Persona*, y distinguirlas, incluso si tienen los mismos valores de los atributos, porque es importante su identidad única.

Por tanto, todos los tipos primitivos (número, string) son tipos de datos UML, pero no todos los tipos de datos son primitivos. Por ejemplo, *NumeroDeTelefono* es un tipo de dato no primitivo.

Estos valores de tipos de datos también se conocen como **objetos valor**.

La noción de tipos de datos puede ser sutil. Como regla empírica, sea fiel a la prueba básica de tipos de atributos “simples”: hágalo un atributo si se considera de manera natural como un número, string, booleano, fecha u hora (etcétera); en otro caso, representélo como una clase conceptual aparte.

En caso de duda, defina algo como una clase conceptual aparte en lugar de como un atributo.

12.4. Clases de tipos de datos no primitivos

El tipo de un atributo podría representarse como una clase no primitiva por derecho propio en un modelo del dominio. Por ejemplo, en el sistema de PDV hay un identificador de artículo. Generalmente se ve simplemente como un número. Así que, ¿se debería representar como una clase no primitiva? Aplique esta guía:

Represente lo que podría considerarse, inicialmente, como un tipo de dato primitivo (como un número o string) como una clase no primitiva si:

- Está compuesto de secciones separadas.
 - Número de teléfono, nombre de persona.
- Habitualmente, hay operaciones asociadas con él, como análisis sintáctico o validación
 - Número de la seguridad social.
- Tiene otros atributos.
 - el precio de promoción podría tener una fecha (efectiva) de comienzo y fin.
- Es una cantidad con una unidad.

- La cantidad del pago tiene una unidad monetaria.
- Es una abstracción de uno o más tipos con alguna de estas cualidades.
 - El identificador del artículo en el dominio de ventas es una generalización de tipos como el Código de Producto Universal (UPC) o el Número de Artículo Europeo (EAN)

Aplicando estas guías a los atributos del modelo del dominio del PDV llegamos al siguiente análisis:

- El identificador del artículo es una abstracción de varios esquemas de codificación comunes, incluyendo UPC-A, UPC-E y la familia de esquemas EAN. Estos esquemas de codificación numéricos tienen subpartes que identifican al fabricante, producto, país (para EAN), y un dígito de control para validarlos. Por tanto, debería existir una clase no primitiva *ArticuloID*, puesto que satisface muchas de las guías anteriores.
- Los atributos del *precio* y *cantidad* deberían ser clases no primitivas *Cantidad* o *Moneda* porque se trata de cantidades en una unidad monetaria.
- El atributo de la *direccion* debe ser una clase no primitiva *Direccion* porque tiene secciones diferentes.

Las clases *ArticuloID*, *Direccion* y *Cantidad* son tipos de datos (no es significativa la identidad única de las instancias) pero merece la pena considerarlas como clases independientes debido a sus cualidades.

¿Dónde representamos las clases de tipos de datos?

¿Debería mostrarse la clase *ArticuloID* como una clase conceptual independiente en el modelo del dominio? Depende de lo que quiera resaltar en el diagrama. Puesto que *ArticuloID* es un *tipo de datos* (no es importante la identidad única de las instancias), se podría mostrar en el compartimento de los atributos del rectángulo de la clase, como se muestra en la Figura 12.4. Pero, puesto que es una clase no primitiva, con sus propios atributos y asociaciones, podría ser interesante mostrarla como una clase conceptual en su propio rectángulo. No existe una respuesta correcta; depende de cómo se esté utilizando el modelo del dominio como herramienta de comunicación, y la importancia de los conceptos en el dominio.

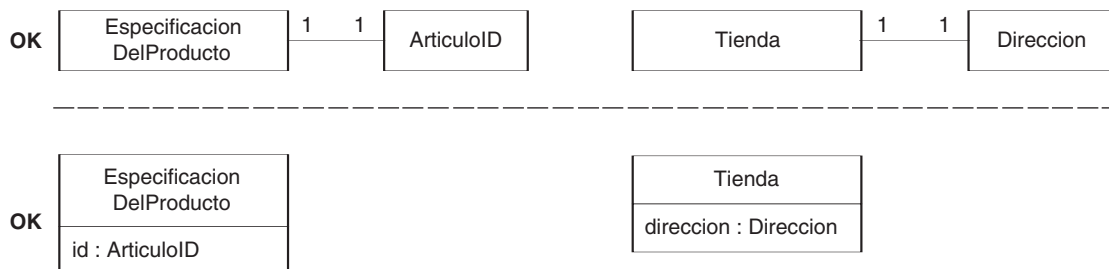


Figura 12.4. Si la clase del atributo es un tipo de datos, podría mostrarse en el rectángulo del atributo.

Un modelo del dominio es una herramienta de comunicación; las elecciones sobre lo que se muestra deben hacerse con esa consideración en mente.

12.5. Deslizarse al diseño: ningún atributo como clave ajena

No se deberían utilizar los atributos para relacionar las clases conceptuales en el modelo del dominio. La violación más típica de este principio es añadir un tipo de **atributo de clave ajena**, como se hace normalmente en el diseño de bases de datos relacionales, para asociar dos tipos. Por ejemplo, en la Figura 12.5, no es deseable el atributo *numeroRegistroActual* en la clases *Cajero* porque el propósito es relacionar el *Cajero* con un objeto *Registro*. La mejor manera de expresar que un *Cajero* utiliza un *Registro* es con una asociación, no con un atributo de clave ajena. Una vez más, relacione los tipos con una asociación, no con un atributo.

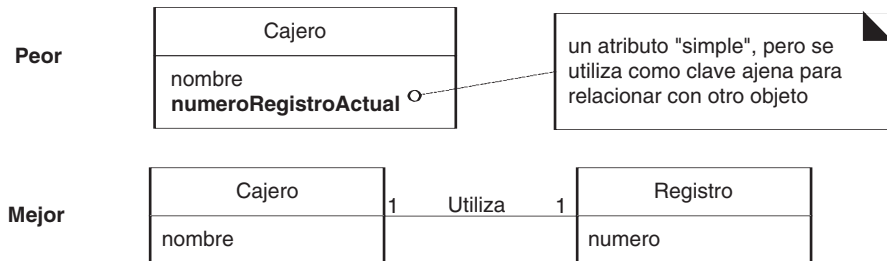


Figura 12.5. No utilice atributos como claves ajenas.

Hay muchas formas de relacionar objetos —siendo las claves ajenas una de ellas— y pospondremos el modo de implementar la relación hasta el diseño, para evitar el **deslizamiento al diseño**.

12.6. Modelado de cantidades y unidades de los atributos

La mayoría de las cantidades numéricas no deberían representarse simplemente como números. Considere el precio o la velocidad. Son cantidades con unidades asociadas, y es habitual que se necesite conocer la unidad y dar soporte a las conversiones. El software del PDV NuevaEra es para un mercado internacional y necesita soportar los precios en diferentes monedas. En el caso general, la solución consiste en representar la *Cantidad* como una clase conceptual aparte, con una *Unidad* asociada [Fowler96]. Puesto que las cantidades se consideran tipos de datos (no es importante la identidad única de las instancias), es aceptable recoger su representación en la sección de atributos del rectángulo de clase (ver Figura 12.6). También es común mostrar especializaciones de *Cantidad*. El *Dinero* es una clase de *Cantidad* cuyas unidades son monedas. El *Peso* es una cantidad cuyas unidades son kilogramos o libras.

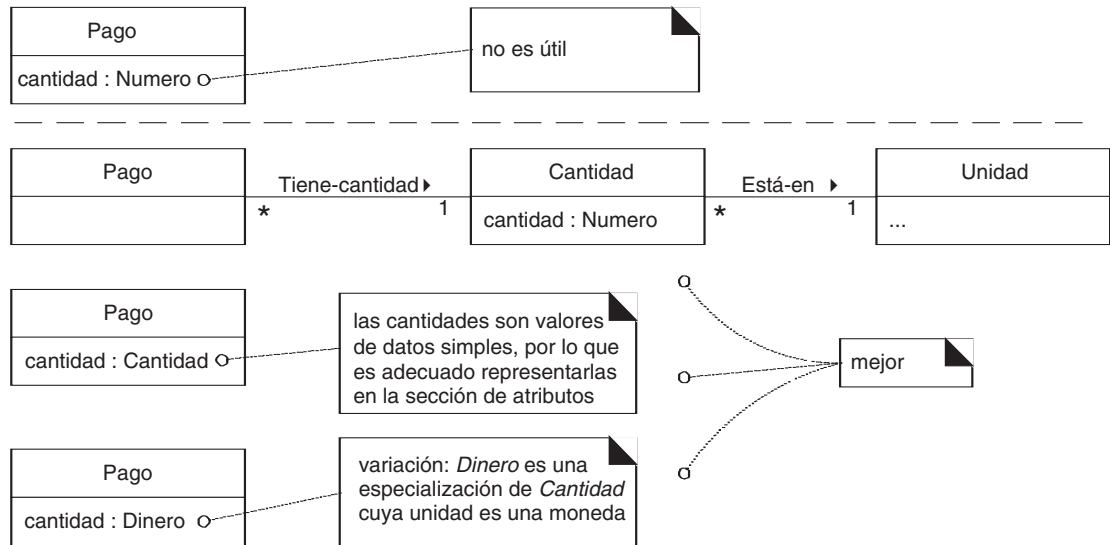


Figura 12.6. Modelado de cantidades.

12.7. Atributos en el Modelo del Dominio de NuevaEra

Los atributos elegidos reflejan los requisitos de esta iteración —los escenarios de *Procesar Venta* de esta iteración—.

<i>Pago</i>	<i>cantidad</i> : Se debe capturar una cantidad (también conocida como “cantidad entregada”) para determinar si se proporciona el pago suficiente y calcular el cambio.
<i>Especificacion DelProducto</i>	<i>descripcion</i> : Para mostrar la descripción en una pantalla o recibo. <i>id</i> : Para buscar una <i>EspecificacionDelProducto</i> , dado un artículoID introducido, es necesario relacionarlas con un <i>id</i> . <i>precio</i> : Para calcular el total de la venta y mostrar el precio de la línea de venta.
<i>Venta</i>	<i>fecha, hora</i> : Un recibo es un informe en papel de una venta. Normalmente muestra la fecha y la hora de la venta.
<i>LineaDeVenta</i>	<i>cantidad</i> : Para registrar la cantidad introducida, cuando hay más de un artículo en una línea de venta (por ejemplo, <i>cinco</i> paquetes de tofu).
<i>Tienda</i>	<i>direccion, nombre</i> : El recibo requiere el nombre y la dirección de la tienda.

12.8. Multiplicidad de la LineaDeVenta al Articulo

Es posible que el cajero reciba un grupo de artículos iguales (por ejemplo, seis paquetes de tofu), introduzca el *articuloID* una vez, y después introduzca una cantidad (por

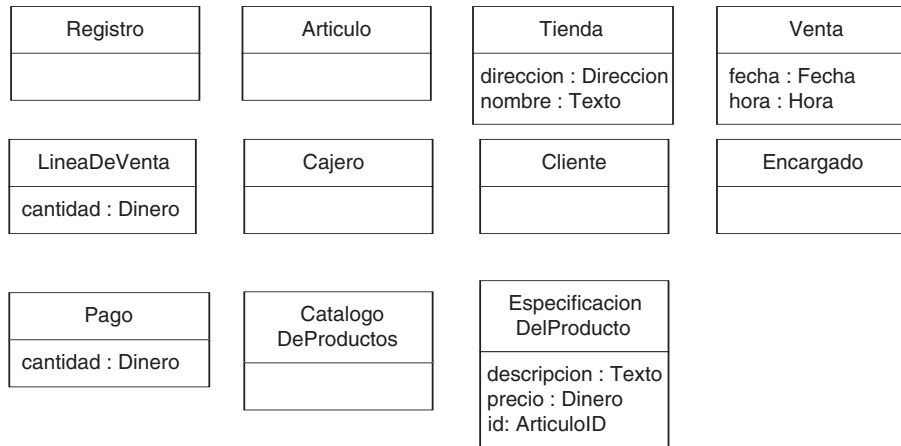


Figura 12.7. Modelo del dominio que muestra los atributos.

ejemplo, seis). En consecuencia, una *LineaDeVenta* individual se puede asociar con más de una instancia de un artículo.

La cantidad introducida por el cajero podría registrarse como un atributo de la *LineaDeVenta* (Figura 12.8). Sin embargo, la cantidad se puede calcular a partir del valor de la multiplicidad actual de la relación, de ahí que pudiera caracterizarse como un **atributo derivado** —uno que puede derivarse a partir de otra información—. En UML, un atributo derivado se indica con el símbolo “/”.

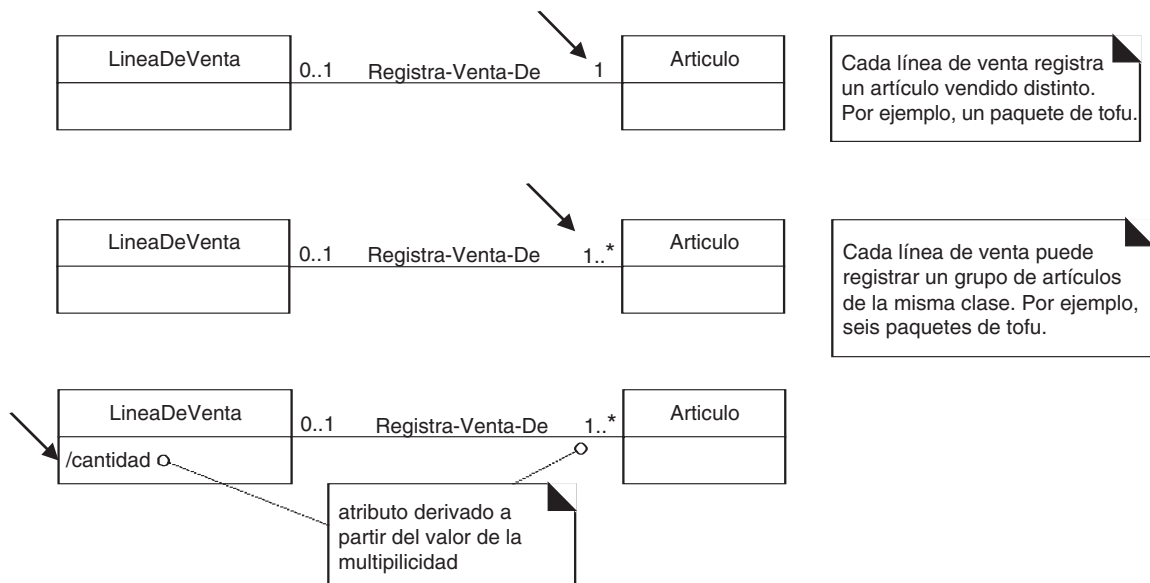


Figura 12.8. Registro de la cantidad de artículos vendidos en una línea de venta.

12.9. Conclusión del Modelo del Dominio

La combinación de las clases conceptuales, asociaciones y atributos, descubiertos en el estudio anterior, da lugar al modelo que se presenta en la Figura 12.9.

Se ha creado un modelo del dominio, relativamente útil, para el dominio de una aplicación de PDV. No existe un único modelo correcto. Todos los modelos son aproximaciones del dominio que estamos intentando entender. Un buen modelo del dominio captura las abstracciones y la información esenciales necesarias para entender el dominio en el contexto de los requisitos actuales, y ayuda a la gente a entender el dominio —sus conceptos, terminología y relaciones—.

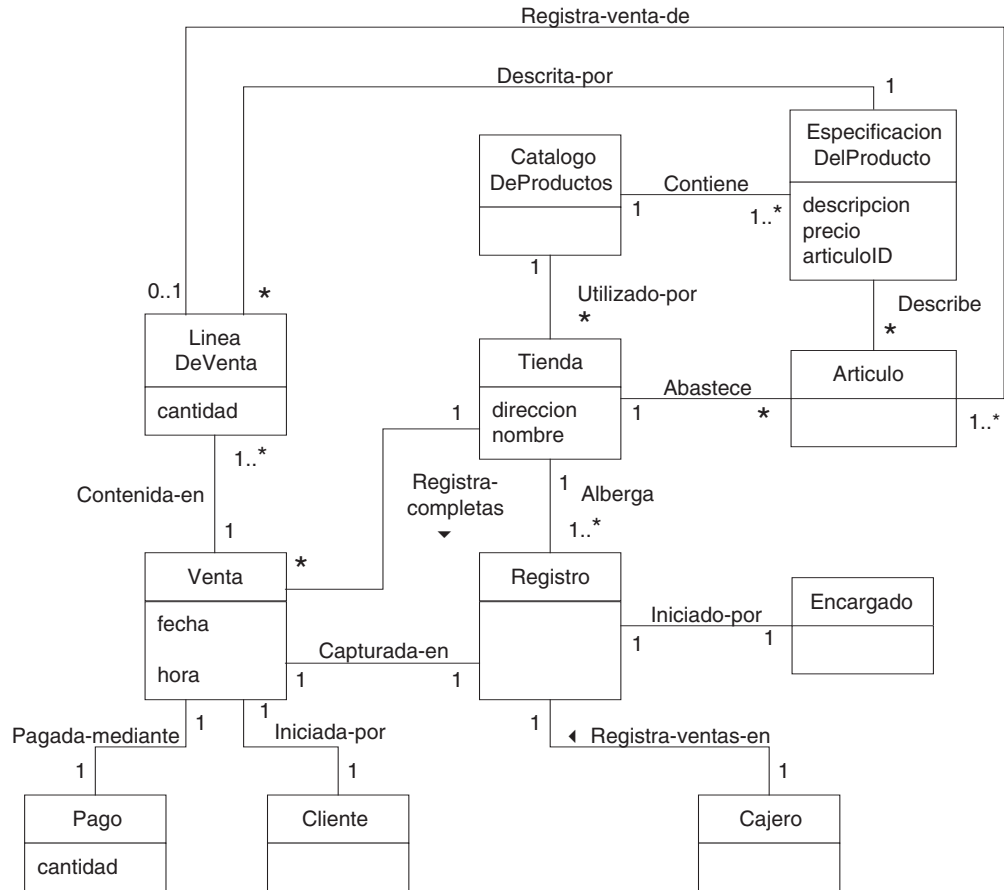


Figura 12.9. Un modelo del dominio parcial.

MODELO DE CASOS DE USO: AÑADIR DETALLES CON LOS CONTRATOS DE LAS OPERACIONES

Rápido, barato, bueno: elija dos cualquiera.

Anónimo

Objetivos

- Crear los contratos para las operaciones del sistema.

Introducción

Los contratos de las operaciones pueden ayudar a definir el comportamiento del sistema; describen el resultado de la ejecución de las operaciones del sistema en función de los cambios de estado de los objetos del dominio. Este capítulo explora su uso.

13.1. Contratos

Los casos de uso son el principal mecanismo del UP para describir el comportamiento del sistema y, normalmente es suficiente. Sin embargo, algunas veces se necesita una descripción más detallada del comportamiento del sistema. Los contratos describen el comportamiento detallado del sistema en función de los cambios de estado de los objetos del Modelo del Dominio, después de la ejecución de una operación del sistema.

Operaciones del sistema y la interfaz del sistema

Se pueden definir contratos para las **operaciones del sistema** —operaciones que el sistema, como una caja negra, ofrece en su interfaz pública para manejar los eventos del sistema entrantes—. Las operaciones del sistema se pueden identificar descubriendo estos eventos del sistema, como se muestra en la Figura 13.1.

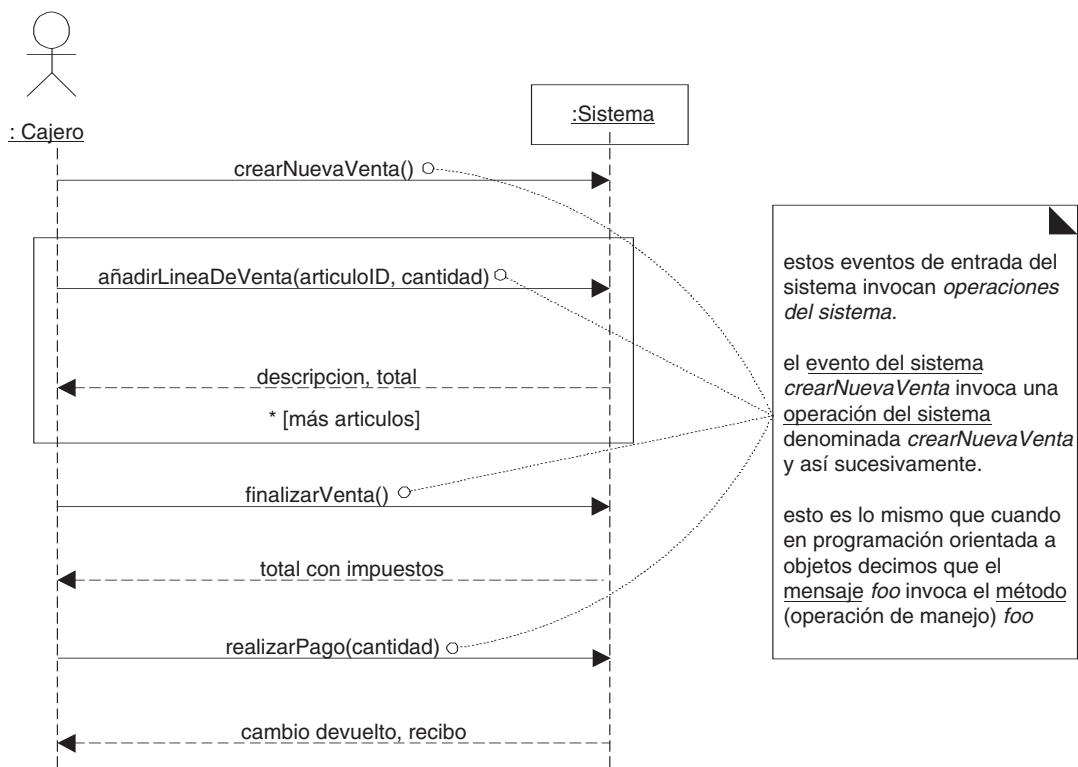


Figura 13.1. Las operaciones del sistema manejan los eventos de entrada del mismo.

El conjunto completo de operaciones del sistema, de todos los casos de uso, define la interfaz pública del sistema, viendo al sistema como un componente o clase individual. En UML, el sistema como un todo se puede representar mediante una clase.

13.2. Ejemplo de contrato: introducirArticulo

Antes de examinar las razones para la escritura de un contrato, merece la pena presentar un ejemplo. A continuación, se presenta un contrato para la operación del sistema *introducirArticulo*.

Contrato CO2: introducirArticulo

Operación:	introducirArticulo(articuloID:ArticuloID, cantidad:integer)
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- Se creó una instancia de LineaDeVenta ldv (creación de instancias). - ldv se asoció con la Venta actual (formación de asociaciones). - ldv.cantidad pasó a ser cantidad (modificación de atributos). - ldv se asoció con una EspecificacionDelProducto, en base a la coincidencia del articuloID (formación de asociaciones).

13.3. Secciones del contrato

La descripción de cada una de las secciones del contrato se muestra en el siguiente esquema.

Operación:	Nombre de la operación y parámetros.
Referencias cruzadas:	(opcional) Casos de uso en los que pueden tener lugar esta operación.
Precondiciones:	<i>Suposiciones</i> relevantes sobre el estado del sistema o de los objetos del Modelo del Dominio, antes de la ejecución de la operación. No se comprobará en la lógica de esta operación, se asume que son verdad, y son suposiciones no triviales que el lector debe saber que se hicieron.
Postcondiciones:	- El estado de los objetos del Modelo del Dominio después de que se complete la operación. Se discute con detalle en la siguiente sección.

13.4. Postcondiciones

Nótese que cada una de las postcondiciones del ejemplo *introducirArticulo* incluía una categorización como *creación de instancias* o *formación de asociaciones*. He aquí un punto clave:

La postcondición describe cambios en el estado de los objetos del Modelo del Dominio. Los cambios de estado del Modelo del Dominio comprenden la creación de instancias, formación o rotura de asociaciones y cambio en los atributos.

Las postcondiciones no son acciones que se ejecutarán durante la operación; más bien, son declaraciones sobre los objetos del Modelo del Dominio que son verdad cuando la operación ha terminado —*después de que el humo se haya despejado*—.

En resumen, las postcondiciones se dividen en estas categorías:

- Creación y eliminación de instancias.
- Modificación de atributos.
- Formación y rotura de asociaciones (siendo precisos, *enlaces* UML).

Como ejemplo de postcondición que rompe una asociación, considere una operación que permite la eliminación de líneas de venta. La post-condición podría decir “Se rompió la asociación seleccionada de la *LineaDeVenta* con la *Venta*.”. En otros dominios, cuando se cancela un préstamo o cuando alguien deja de ser socio de alguna organización, se rompen las asociaciones.

La postcondición de la eliminación de instancias es más rara, porque en el mundo real uno, normalmente, no se preocupa de forzar explícitamente la destrucción de una cosa. Sin embargo, como ejemplo: en muchos países, después de que una persona se

haya declarado en bancarrota y hayan pasado siete o diez años, se deben destruir todos los registros de su declaración de bancarrota, por ley. Nótese que esto es una perspectiva conceptual, no de implementación. Éstos no son declaraciones sobre liberar la memoria del ordenador ocupada por objetos software.

La cualidad importante es ser declarativo y enunciar con un estilo orientado al cambio en lugar de orientado a la acción, puesto que las postcondiciones son declaraciones sobre los estados o resultados, en lugar de una descripción de las acciones a ejecutar, o un diseño de una solución.

Las postcondiciones se relacionan con el Modelo del Dominio

Estas postcondiciones se expresan en el contexto de los objetos del Modelo del Dominio. ¿Qué instancias se pueden crear? —aquellas del Modelo del Dominio; ¿qué asociaciones se pueden formar?— las que se encuentran en el Modelo del Dominio; y así sucesivamente.

Una ventaja de las postcondiciones: detalle analítico

Expresados en un estilo declarativo de cambio de estado, los contratos son una herramienta excelente para el análisis de requisitos que describen los cambios de estado que requiere una operación del sistema (en función de los objetos del Modelo del Dominio) sin tener que describir *cómo* se van a llevar a cabo. En otras palabras, el diseño del software y la solución se puede diferir, y uno puede centrarse, analíticamente en *qué* debe suceder, en lugar de en *cómo* se va a realizar. Además, las postcondiciones soportan detalles de grano fino y una declaración más específica de cuál debe ser el resultado de la operación.

También es posible expresar este nivel de detalle en los casos de uso, pero normalmente no es deseable, puesto que entonces pasan a ser excesivamente elocuentes y detallados.

Considere la postcondición:

Postcondiciones:	- Se creó una instancia de <i>LineaDeVenta</i> <i>ldv</i> (creación de instancias). - <i>ldv</i> se asoció con la <i>Venta</i> actual (formación de asociaciones). - <i>ldv.cantidad</i> pasó a ser <i>cantidad</i> (modificación de atributos). - <i>ldv</i> se asoció con una <i>EspecificacionDelProducto</i>, en base a la coincidencia del <i>articuloID</i> (formación de asociaciones).
-------------------------	---

No se hace ningún comentario sobre el modo de crear una instancia de *LineaDeVenta*, o cómo se asocia con una *Venta*. Esto podría ser una declaración sobre escribir en folios y que se grapen, la utilización de la tecnología Java para crear objetos software y conectarlos, o la inserción de filas en una base de datos relacional.

El espíritu de las postcondiciones: el escenario y telón

Expresa las postcondiciones en pasado, para resaltar que son declaraciones sobre un cambio de estado en el pasado. Por ejemplo:

- (mejor) Se **creó** una *LineaDeVenta*.

en lugar de

- (peor) Cree una *LineaDeVenta*.

Piense en las postcondiciones utilizando la siguiente imagen:

El sistema y sus objetos se presentan en el escenario de un teatro.

1. Antes de la operación, tome una fotografía del escenario.
2. Baje el telón y aplique la operación del sistema (*ruido metálico de fondo de martillos o campanas, gritos, chirridos...*).
3. Suba el telón y tome una segunda fotografía.
4. Compare las fotografías anterior y posterior, y exprese como postcondiciones el cambio en el estado del escenario (*Se creó una LineaDeVenta...*).

Si se utilizan contratos, ¿cómo de completas deben ser las postcondiciones?

Primero, los contratos podrían no ser necesarios. Esta cuestión se discute en una posterior sección. Pero, asumiendo que se desean algunos contratos, no es probable —o incluso necesario— que se genere un conjunto completo y detallado de postcondiciones para una operación del sistema, durante el trabajo de requisitos. Trate su creación como la mejor suposición inicial, entendiendo que los contratos no serán completos. Su temprana creación —incluso si es incompleta— es, ciertamente, mejor que diferir su estudio hasta el trabajo de diseño, cuando los desarrolladores deben preocuparse por el diseño de una solución, en lugar de investigar *qué* se debe hacer.

Algunos de los detalles finos —y quizás incluso los más importantes— se descubrirán durante el trabajo de diseño. Esto no es necesariamente una cosa mala; si el esfuerzo dedicado al análisis de requisitos es demasiado grande el rendimiento decrece. Naturalmente, tiene lugar algún descubrimiento durante el trabajo de diseño, que puede entonces documentar el trabajo de requisitos de una iteración posterior. Ésta es una de las ventajas del desarrollo iterativo: los descubrimientos que se generan en una iteración anterior pueden impulsar el estudio y el trabajo de análisis de la siguiente.

13.5. Discusión: postcondiciones de introducirArticulo

La siguiente sección analiza la motivación de las postcondiciones de la operación del sistema *introducirArticulo*.

Creación y eliminación de instancias

Después de introducir el *articuloID* y la *cantidad* de un artículo, ¿qué nuevo objeto debe haberse creado? Una *LineaDeVenta*. Por tanto:

- Se creó una instancia de *LineaDeVenta* *ldv* (creación de instancias).

Obsérvese el nombre de la instancia. Este nombre simplificará las referencias a la nueva instancia en otras sentencias de la post-condición.

Modificación de atributos

Después de que el cajero haya introducido el *articuloID* y la *cantidad*, ¿qué atributos de los objetos nuevos o de los ya existentes deberían haberse modificado? La *cantidad* de la *LineaDeVenta* debería haber pasado a ser igual que el parámetro *cantidad*. De ahí:

- *ldv.cantidad* pasó a ser *cantidad* (modificación de atributos).

Formación y rotura de asociaciones

Después de que el cajero haya introducido el *articuloID* y la *cantidad*, ¿qué asociaciones entre los objetos nuevos o los ya existentes deberían haberse formado o roto? La nueva *LineaDeVenta* debería haberse relacionado con su *Venta*, y su *EspecificacionDelProducto*. Así:

- *ldv* se asoció con la *Venta* actual (formación de asociaciones).
- *ldv* se asoció con una *EspecificacionDelProducto*, en base a la coincidencia del *articuloID* (formación de asociaciones).

Nótese la indicación informal de que se forma una relación con una *EspecificacionDelProducto* particular —aquella cuyo *articuloID* se corresponda con el parámetro—. Son posibles otros lenguajes más sofisticados y formales, como utilizar el Lenguaje de Restricciones de Objetos (OCL, *Object Constraint Language*). Recomendación: manténgalo simple.

13.6. La escritura de los contratos da lugar a actualizaciones en el Modelo del Dominio

Es normal, durante la creación de los contratos, descubrir la necesidad de registrar nuevas clases conceptuales, atributos o asociaciones en el Modelo del Dominio. No se limite a la definición anterior del Modelo del Dominio; enriquezca cuando haga nuevos descubrimientos mientras piensa en los contratos de las operaciones.

13.7. ¿Cuándo son útiles los contratos?

¿Contratos vs. casos de uso?

Los casos de uso son el principal repositorio de requisitos del proyecto. Podrían proporcionar la mayoría o todos los detalles necesarios para saber qué hacer en el diseño, en cuyo caso, los contratos no son útiles. Sin embargo, hay situaciones en las que los detalles y la complejidad de los cambios de estado requeridos, son difíciles de capturar en los casos de uso.

Por ejemplo, considere un sistema de reservas de vuelos y la operación del sistema *añadirNuevaReserva*. La complejidad es muy alta considerando todos los objetos del dominio que se deben cambiar, crear y asociar. Estos detalles de grano fino *se pueden* escribir en detalle en el caso de uso asociado a esta operación, pero dará lugar a un caso de uso extremadamente detallado (por ejemplo, anotando cada atributo que se debe cambiar en todos los objetos).

Obsérvese que el formato de la postcondición del contrato ofrece y promueve un lenguaje muy preciso, analítico y exigente que soporta una detallada minuciosidad.

Si, únicamente basándose en los casos de uso y mediante continuas colaboraciones (verbales) con un experto en la materia de estudio, los desarrolladores pueden entender cómodamente qué hacer, entonces evite la escritura de los contratos.

Sin embargo, en aquellas situaciones donde la complejidad es alta y añade valor la precisión detallada, los contratos son otra herramienta de requisitos.

Muy a menudo, los contratos no estarán muy justificados de manera que si un equipo está creando contratos para todas las operaciones del sistema de cada caso de uso, es una advertencia de que, o bien los casos de uso son algo deficientes, o no hay suficiente y continua colaboración o acceso a los expertos en la materia de estudio, o el equipo está haciendo demasiada documentación innecesaria.

El caso de estudio del PDV NuevaEra muestra más contratos de los que probablemente sean necesarios, por cuestiones pedagógicas. En la práctica, la mayoría de los detalles que recogen se pueden inferir de manera obvia a partir del texto de los casos de uso. Por otro lado, “obvio” es un concepto muy escurridizo.

13.8. Guías: contratos

Aplique el siguiente consejo para crear los contratos:

Para hacer contratos:

1. Identifique las operaciones del sistema a partir de los DSSs.
2. Construya un contrato para las operaciones del sistema complejas y quizás sutiles en sus resultados, o que no están claras en el caso de uso.
3. Para describir la postcondiciones utilice las siguientes categorías:
 - creación y eliminación de instancias
 - modificación de atributos
 - formación y rotura de asociaciones

Consejos acerca de la escritura de contratos

- Establezca las postcondiciones de forma declarativa, con una sentencia pasiva expresada en pasado para destacar que se trata de una declaración de un cambio de estado en lugar del diseño de la manera en la que se va a realizar. Por ejemplo:
 - (mejor) Se creó una *LineaDeVenta*
 - (peor) Cree una *LineaDeVenta*.
- Recuerde establecer una relación entre los objetos existentes o aquellos creados recientemente mediante la definición de la formación de asociaciones. Por ejemplo, no es suficiente que se cree una instancia de *LineaDeVenta* cuando tenga lugar la operación *introducirArticulo*. Después de que se complete la operación, también debería cumplirse que la nueva instancia creada se asoció con la *Venta*; de ahí que:
 - La *LineaDeVenta* se asoció con la *Venta* (formación de asociaciones).

El error más habitual en la creación de contratos

El problema más común es olvidarse de incluir la *formación de asociaciones*. En particular, cuando se crean nuevas instancias, es muy probable que se necesiten establecer asociaciones con varios objetos. ¡No lo olvide!

13.9. Ejemplo del PDV NuevaEra: contratos

Operaciones del sistema de Procesar Venta

Contrato CO1: crearNuevaVenta

Operación:	crearNuevaVenta()
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Ninguna
Postcondiciones:	- Se creó una instancia de Venta v (creación de instancias). - v se asoció con el Registro (formación de asociaciones). - Se inicializaron los atributos de v.

Obsérvese la descripción vaga de la última post-condición. Si es suficiente, está bien.

En un proyecto, todas estas postcondiciones particulares son tan obvias a partir del caso de uso que, probablemente, no se debería escribir el contrato de *crearNuevaVenta*.

Recuerde uno de los principios que guían un buen proceso y el UP: Manténgalo tan ligero como sea posible, y evite todos los artefactos a menos que realmente añadan valor.

Contrato CO2: introducirArticulo

Operación:	introducirArticulo(articuloID:ArticuloID, cantidad:integer)
Referencias cruzadas:	Caso de Uso: Procesar Venta

Precondiciones:	Hay una venta en curso
Postcondiciones:	- <i>Se creó una instancia de LineaDeVenta ldv (creación de instancias).</i> - <i>ldv se asoció con la Venta actual (formación de asociaciones).</i> - <i>ldv.cantidad pasó a ser cantidad (modificación de atributos).</i> - <i>ldv se asoció con una EspecificacionDelProducto, en base a la coincidencia del articuloID (formación de asociaciones).</i>

Contrato CO3: finalizarVenta

Operación:	finalizarVenta()
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- <i>Venta.esCompleta pasó a ser verdad (modificación de atributos).</i>

Contrato CO4: realizarPago

Operación:	realizarPago(cantidad: Dinero)
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- <i>Se creó una instancia de Pago p (creación de instancias).</i> - <i>p.cantidadEntregada pasó a ser cantidad (modificación de atributos).</i> - <i>p se asoció con la Venta actual (formación de asociaciones).</i> - <i>La Venta actual se asoció con la Tienda (formación de asociaciones); (para añadirlo al registro histórico de las ventas completadas).</i>

13.10. Cambios en el Modelo del Dominio

Hay un dato sugerido por estos contratos que todavía no está representado en el modelo del dominio: la terminación de la entrada del artículo en la venta. La especificación de *finalizarVenta* lo modifica, y probablemente es una buena idea comprobarlo después, durante el trabajo de diseño, en la operación *realizarPago*, para rechazar los pagos hasta que se complete una venta.

Una manera de representar esta información es con un atributo *esCompleta* en la *Venta*, de tipo de dato booleano:

Venta
esCompleta: Boolean fecha hora

Existen alternativas, que se tienen en cuenta, especialmente, durante el trabajo de diseño. Una técnica se denomina el **patrón de Estado**, que se estudiará en el Capítulo 34. Otra es el uso de objetos “sesión” que siguen la pista del estado de una sesión y rechazan operaciones improcedentes; esto también se estudiará más tarde.

13.11. Contratos, operaciones y UML

Contratos en UML: especificación de operaciones

UML define las **operaciones** formalmente. Citando textualmente:

Una operación es una especificación de una transformación o consulta que se puede invocar para que la ejecute un objeto. [RJB99]

Por ejemplo, los elementos de una interfaz son operaciones, en términos de UML. Una operación es una abstracción, no una implementación. Por el contrario, un **método** (en UML) es una implementación de una operación.

Una operación UML tiene una **signatura** (nombre y parámetros), y también una **especificación de operación**, que describe los efectos producidos por la ejecución de la operación; esto es, la postcondición. El formato de la especificación de operación en UML es flexible, y no tiene que ser el formato de contrato que se muestra en este capítulo. Sin embargo, los documentos de UML proporcionan como ejemplos el estilo de contratos con pre- y postcondiciones, ya que éste es el enfoque más conocido para las especificaciones formales de las operaciones.

Resumiendo: UML define especificaciones de operaciones, que se pueden especificar con el estilo de los contratos con pre- y postcondiciones. Nótese que, como se subraya en este capítulo, una especificación de operación UML podría *no* mostrar un algoritmo o solución, sino únicamente los cambios de estado o efectos de las operaciones.

Además de utilizar los contratos para especificar las operaciones públicas del *Sistema* completo (operaciones del sistema), los contratos se pueden aplicar a las operaciones de cualquier nivel de granularidad: las operaciones públicas (o interfaz) de un subsistema, una clase abstracta, etcétera. Las operaciones presentadas en este capítulo pertenecen a la clase *Sistema*. En UML, las operaciones pertenecen a las clases. Más aún, en UML, los “subsistemas” se modelan como clases (y simultáneamente también como paquetes). En UML, el “sistema” global es el subsistema de más alto nivel, y se modela como una clase denominada *Sistema* (en realidad, cualquier nombre es legal) con operaciones y especificaciones públicas.

Contratos de las operaciones expresados con OCL

Existe un lenguaje formal asociado con UML denominado Lenguaje de Restricciones de Objetos (OCL, *Object Constraint Language*) [WK99], que se puede utilizar para expresar las restricciones en los modelos. OCL podría utilizarse en lugar del lenguaje natural informal que se utiliza en este capítulo; UML permite cualquier formato para una especificación de operación.

Sugerencia

A menos que exista una razón práctica apremiante que requiera que la gente aprenda y utilice OCL, mantenga las cosas simples y utilice el lenguaje natural.

OCL define un formato oficial para la especificación de las pre- y postcondiciones de las operaciones, como se muestra en este fragmento:

```
Sistema::crearNuevaVenta( )  
  pre: <sentencia en OCL>  
  post: ...
```

Detalles adicionales sobre OCL están fuera del alcance de este libro.

Contratos en el Diseño por Contrato

La forma de los contratos con pre- y postcondiciones utilizados para la especificación de las operaciones en UML, se lleva impulsando durante muchos años por Bertrand Meyer, formalizado en una técnica de diseño denominada **Diseño por Contrato** [Meyer97 (primera ed. 1989)], aunque su origen procede de un trabajo anterior en los años sesenta sobre los lenguajes de especificación formal. En el Diseño por Contrato, también se escriben los contratos para las operaciones de las clases de grano fino, no sólo para las operaciones públicas de los sistemas y subsistemas.

Además, el Diseño por Contrato fomenta la inclusión de una sección *invariante*, como es habitual en especificaciones de contrato completas. Los invariantes definen las cosas que no deben cambiar de estado antes y después de que se ejecute una operación. Los invariantes no se han utilizado en este capítulo por simplicidad.

Soporte de los lenguajes de programación para los contratos

Algunos lenguajes, como Eiffel, incluyen soporte a nivel de expresiones del lenguaje para los invariantes, las pre- y postcondiciones. Existen pre-procesadores en Java que proporcionan un soporte parecido.

13.12. Contratos de las operaciones en el UP

Un contrato con pre- y postcondiciones es un estilo bien conocido para especificar una operación en UML. En UML, las operaciones existen a muchos niveles, desde el *Sistema* hasta las clases de grano fino, como *Venta*. Los contratos de especificación de operaciones para el nivel del *Sistema* forman parte del Modelo de Casos de Uso, aunque no se pusieron de relieve en la documentación original del RUP o el UP; su inclusión en este modelo se verificó con los autores del RUP¹.

Fases

Inicio: Los contratos no se justifican durante la fase de inicio, son demasiado detallados.

Elaboración: Si es que se utilizan, los contratos se escribirán durante la elaboración, cuando se escriben la mayoría de los casos de uso. Escriba solamente los contratos para las operaciones del sistema más complejas y sutiles.

Relaciones entre los artefactos

En las Figuras 13.2 y 13.3 se muestran las relaciones entre los contratos y otros artefactos, con diferentes niveles de detalle.

¹ Comunicación privada.

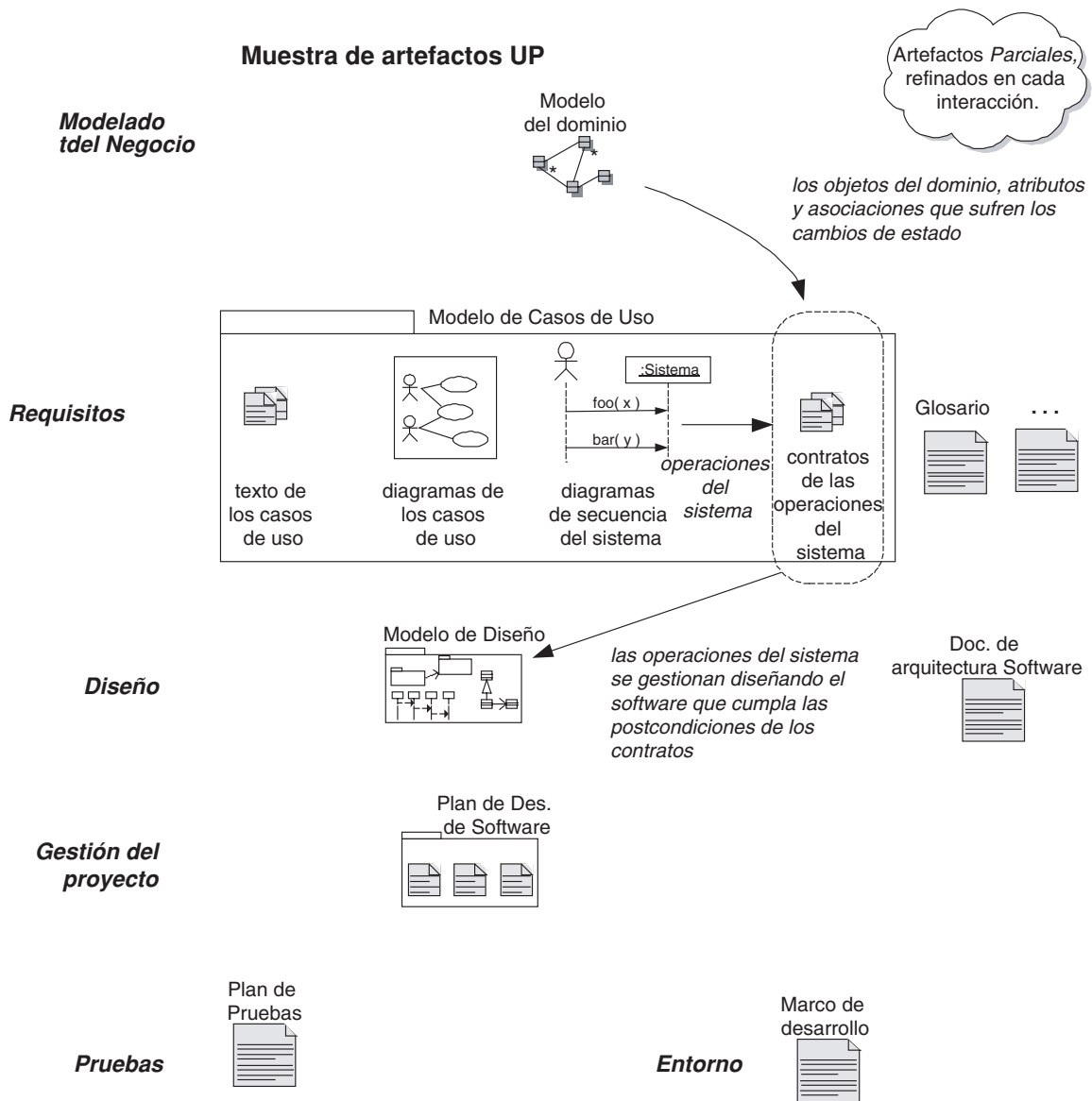


Figura 13.2. Muestra de la influencia entre los artefactos UP.

13.13. Lecturas adicionales

Los contratos de las operaciones proceden del área de las especificaciones formales, y se llevan utilizando y refinando desde los años sesenta, como en el Método de Desarrollo Viena (VDM, *Vienna Development Method*) [BJ78]; existe literatura abundante sobre VDM y otros lenguajes de especificación formal.

Bertrand Meyer contribuyó a un reconocimiento mucho más amplio de las especificaciones formales y los contratos con la inclusión de las pre- y postcondiciones en el lenguaje Eiffel; su *Construcción de Software Orientado a Objetos* proporciona los detalles. Es el creador del **Diseño por Contrato**.

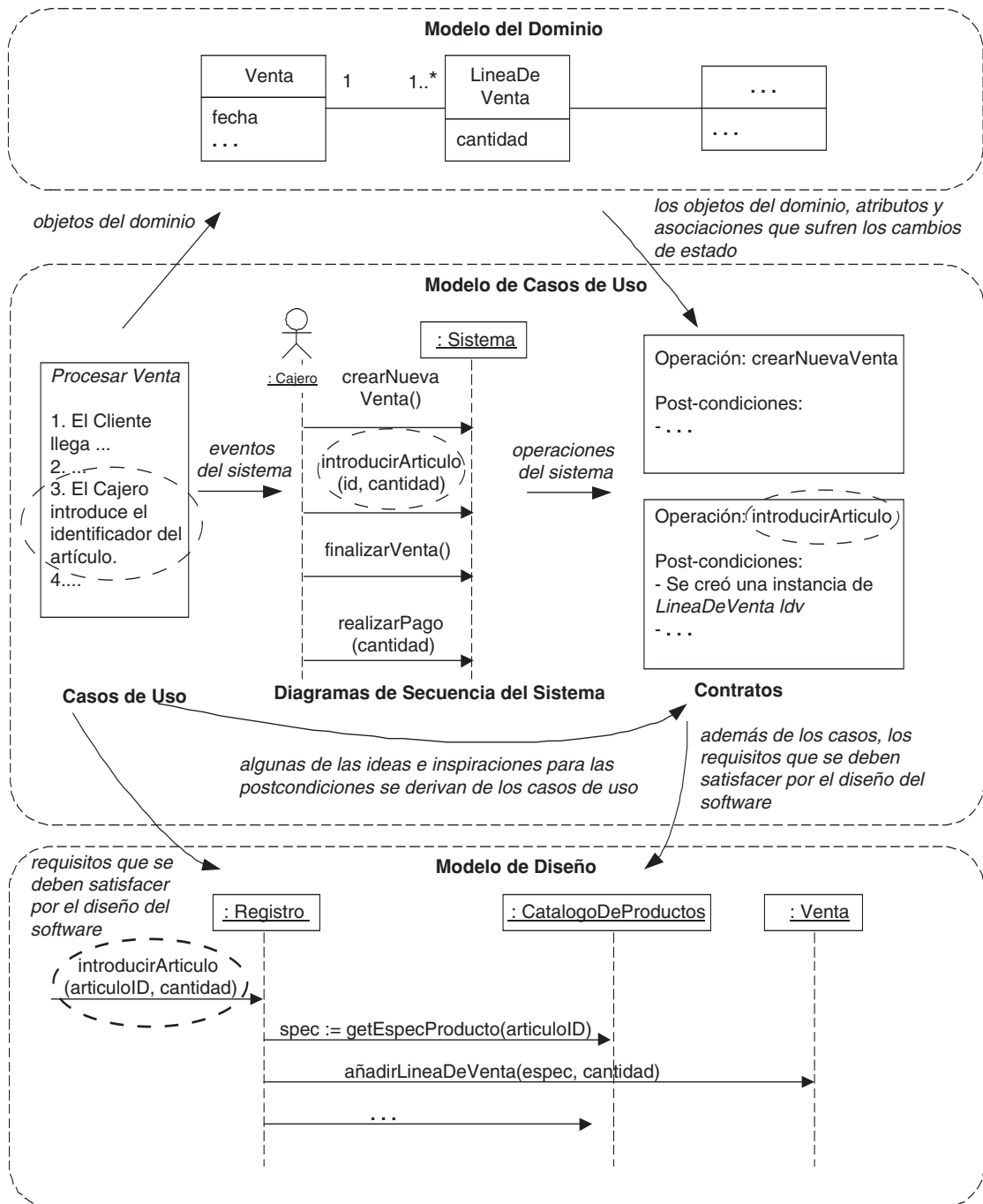


Figura 13.3. Relación del contrato con otros artefactos.

En UML, los contratos de las operaciones también pueden especificarse de manera más rigurosa en el Lenguaje de Restricciones de Objetos (OCL), para lo que se requiere leer *The Object Constraint Language: Precise Modeling with UML*, de Warmer y Kleppe.

DE LOS REQUISITOS AL DISEÑO EN ESTA ITERACIÓN

Hardware, n.: Las partes de un ordenador a las que se pueden dar patadas.

Anónimo

Objetivos

- Motivar la transición a las actividades de diseño.
- Contrastar la importancia de las técnicas de diseño de objetos frente al conocimiento de la notación UML.

Introducción

Hasta el momento, el caso de estudio ha hecho hincapié en el estudio de los requisitos, conceptos y operaciones relacionadas con un sistema. Siguiendo las guías del UP, quizás se investigaron el 10% de los requisitos en la fase de inicio, y se comenzó un estudio ligeramente más profundo en esta primera iteración de la elaboración. Los capítulos siguientes cambian el énfasis hacia el diseño de una solución para esta iteración en función de objetos software que colaboran.

14.1. Iterativamente hacer lo correcto, y hacerlo correcto

Los requisitos y el análisis orientado a objetos se han centrado en aprender a *hacer lo correcto*; es decir, en entender algunos de los objetivos importantes del PDV NuevaEra, y las reglas y restricciones relacionadas. Por el contrario, el siguiente trabajo de diseño pondrá de relieve *hacerlo correcto*; esto es, diseñar con destreza una solución que satisfaga los requisitos de esta iteración.

En el desarrollo iterativo, en cada iteración tendrá lugar una transición desde un enfoque centrado en los requisitos a un enfoque centrado en el diseño y la implementación.

Además, es normal y saludable descubrir y cambiar algunos requisitos de las *primeras* iteraciones durante el trabajo de diseño e implementación. Estos descubrimientos clarificarán los objetivos del trabajo de diseño de esta iteración, y refinarán la comprensión de los requisitos para las iteraciones futuras. A lo largo de estas primeras iteraciones de elaboración el descubrimiento de los requisitos se estabilizará, de manera que al final de la elaboración quizás se definan con detalle y de manera fiable el 80% de los requisitos.

14.2. ¿No lleva eso semanas en hacerse? No, no exactamente

Después de muchos capítulos de discusión detallada, seguramente, debe parecer que el modelado anterior debe llevar semanas de esfuerzo. No es así. Cuando uno se encuentra cómodo con las técnicas de escritura de casos de uso, modelado del dominio, etcétera, el tiempo necesario para llevar a cabo el modelado real que hemos estudiado hasta el momento, de modo realista, será sólo de unos *pocos* días.

Sin embargo, eso no significa que sólo hayan pasado unos días desde el comienzo del proyecto. Muchas otras actividades, como programación de pruebas de conceptos, búsqueda de recursos (personas, software,...), planificación, acondicionamiento del entorno, etcétera, podrían consumir unas pocas semanas de preparación.

14.3. Pasar al diseño de objetos

Durante el diseño de objetos, se desarrolla una solución lógica basada en el paradigma orientado a objetos. Lo esencial de esta solución es la creación de los **diagramas de interacción**, que representan el modo en el que los objetos colaboran para satisfacer los requisitos.

Después de —o en paralelo con— la elaboración de los diagramas de interacción, se pueden representar los **diagramas de clases** (del diseño). Éstos resumen la definición de las clases software (e interfaces) que se van a implementar en el software.

Por lo que se refiere al UP, estos artefactos forman parte del **Modelo de Diseño**.

En la práctica, la creación de los diagramas de interacción y clases tiene lugar en paralelo y con sinergia entre ellos, aunque se introducen de manera lineal en este caso de estudio, por simplicidad y claridad.

La importancia de las técnicas del diseño de objetos vs. las técnicas de la notación UML

Los capítulos siguientes estudian la creación de estos artefactos, o de manera más precisa, las técnicas del diseño de objetos que subyacen a sus creaciones. Lo que es importante es saber cómo pensar y diseñar con objetos, que es muy diferente y una capa-

cidad más importante, que conocer la notación de los diagramas de UML. Al mismo tiempo, un lenguaje visual estándar es importante y, por tanto, se presenta la notación UML necesaria para dar soporte al trabajo de diseño.

De los dos artefactos que se estudiarán, los diagramas de interacción son los más importantes —desde el punto de vista del desarrollo de un buen diseño— y requiere el mayor grado de esfuerzo creativo. La creación de los diagramas de interacción requiere la aplicación de los principios para la asignación de **responsabilidades** y el uso de los **principios y patrones de diseño**. Por tanto, el énfasis de los siguientes capítulos se pone en estos principios y patrones del diseño de objetos.

Las técnicas del diseño de objetos vs. las técnicas de la notación UML

La representación de los diagramas de interacción UML es un reflejo de la toma de decisiones sobre el diseño de objetos.

Las técnicas del diseño de objetos son lo que realmente importan, en lugar de saber cómo dibujar los diagramas UML.

El diseño de objetos básico requiere que se tenga conocimiento sobre:

- Los principios de asignación de responsabilidades.
- Patrones de diseño.

NOTACIÓN DE LOS DIAGRAMAS DE INTERACCIÓN

Los gatos son más listos que los perros. No puedes conseguir que ocho gatos empujen un trineo por la nieve.

Jeff Valdez

Objetivos

- Leer la notación de los diagramas de interacción (secuencia y colaboración) UML básicos.

Introducción

Los siguientes capítulos estudian el diseño de objetos. El lenguaje utilizado para ilustrar los diseños es, principalmente, los diagramas de interacción. Por tanto, es aconsejable, por lo menos, examinar superficialmente los ejemplos de este capítulo y familiarizarse con la notación antes de avanzar.

UML incluye los **diagramas de interacción** para ilustrar el modo en el que los objetos interaccionan por medio de mensajes. Este capítulo introduce la notación, mientras que los capítulos siguientes se centran en su uso en el contexto del aprendizaje y realización del diseño de objetos para el caso de estudio del PDV NuevaEra.

Lea los siguientes capítulos para las guías de diseño

Este capítulo introduce la notación. Para crear objetos bien diseñados, también se deben entender los principios de diseño. Después de familiarizarse algo con la notación de los diagramas de interacción, es importante estudiar los siguientes capítulos sobre estos principios y cómo aplicarlos mientras se dibujan los diagramas de interacción.

15.1. Diagramas de secuencia y colaboración

EL término *diagrama de interacción* es una generalización de dos tipos de diagramas UML más especializados; ambos pueden utilizarse para representar de forma similar interacciones de mensajes:

- Diagramas de colaboración.
- Diagramas de secuencia.

A lo largo del libro, se utilizarán ambos tipos, para remarcar la flexibilidad en la elección.

Los **diagramas de colaboración** ilustran las interacciones entre objetos en un formato de grafo o red, en el cual los objetos se pueden colocar en cualquier lugar del diagrama, como se muestra en la Figura 15.1.

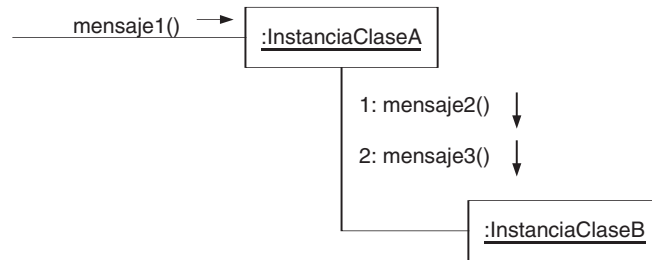


Figura 15.1. Diagrama de colaboración.

Los **diagramas de secuencia** ilustran las interacciones en un tipo de formato con el aspecto de una valla, en el que cada objeto nuevo se añade a la derecha, como se muestra en la Figura 15.2.

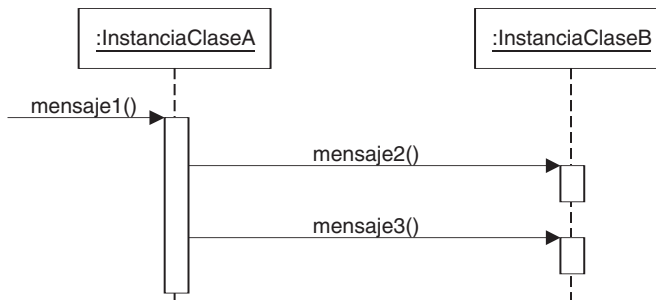


Figura 15.2. Diagrama de secuencia.

Cada tipo tiene puntos fuertes y débiles. Cuando se dibujan los diagramas para publicarlos en páginas estrechas, los diagramas de colaboración tienen la ventaja de permitir la expansión vertical para los nuevos objetos; los objetos adicionales en un diagrama de secuencia deben extenderse hacia la derecha, lo que supone una limitación. Por otro lado, los ejemplos de diagramas de colaboración dificultan que se vea fácilmente la secuencia de mensajes.

La mayoría prefieren los diagramas de secuencia cuando utilizan una herramienta CASE para hacer ingeniería inversa del código fuente a diagramas de interacción, puesto que muestran claramente la secuencia de mensajes.

<i>Tipo</i>	<i>Puntos fuertes</i>	<i>Puntos débiles</i>
secuencia	muestra claramente la secuencia u ordenación en el tiempo de los mensajes notación simple	fuerza a extender por la derecha cuando se añaden nuevos objetos; consume espacio horizontal
colaboración	economiza espacio, flexibilidad al añadir nuevos objetos en dos dimensiones es mejor para ilustrar bifurcaciones complejas, iteraciones y comportamiento concurrente	difícil ver la secuencia de mensajes notación más compleja

15.2. Ejemplo de diagrama de colaboración: realizarPago

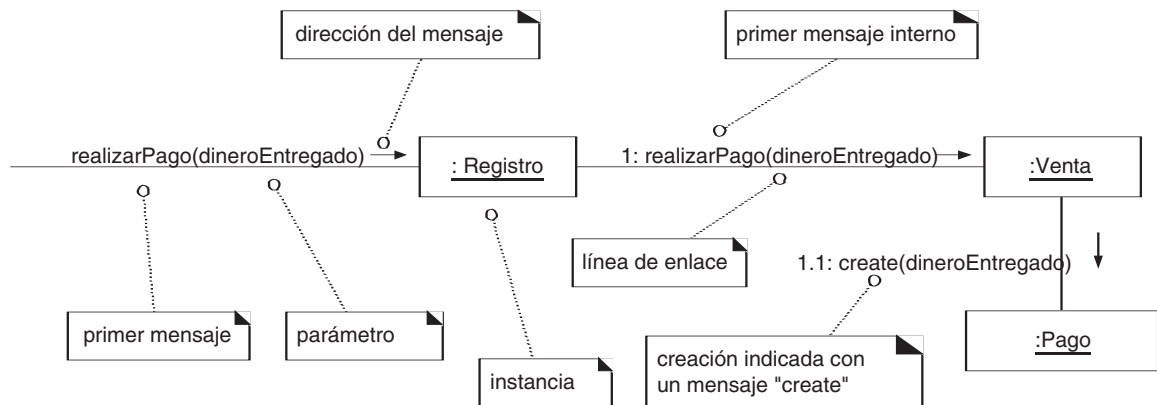


Figura 15.3. Diagrama de colaboración.

El diagrama de colaboración que se muestra en la Figura 15.3 se lee como sigue:

1. Se envía el mensaje *realizarPago* a una instancia de *Registro*. No se identifica al emisor.
2. La instancia de *Registro* envía el mensaje *realizarPago* a una instancia de *Venta*.
3. La instancia de *Venta* crea una instancia de *Pago*.

15.3. Ejemplo de diagrama de secuencia: realizarPago

El objetivo del diagrama de secuencia que se muestra en la Figura 15.4 es el mismo que el del anterior diagrama de colaboración.

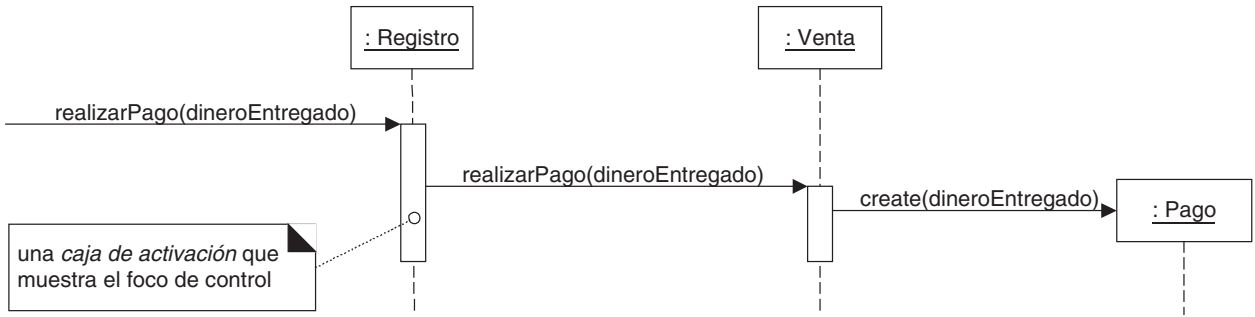


Figura 15.4. Diagrama de secuencia.

15.4. Los diagramas de interacción son importantes

Un problema típico en los proyectos de tecnología de objetos es que no aprecian el valor de llevar a cabo el diseño de objetos mediante el uso de los diagramas de interacción. Un problema relacionado es hacerlos de una manera vaga, como mostrando mensajes hacia objetos que realmente necesitan mucha elaboración adicional; por ejemplo, mostrando el mensaje *ejecutarSimulacion* a algún objeto *Simulacion*, pero sin continuar con el diseño más detallado, como pensando que en virtud de un mensaje con un buen nombre el diseño se completa mágicamente.

Se debería dedicar un tiempo y esfuerzo no trivial a la creación de diagramas de interacción, como reflejo de que se ha estudiado cuidadosamente los detalles del diseño de objetos. Por ejemplo, si la duración de la iteración es de dos semanas, quizás medio día o uno entero, cerca del comienzo de la iteración, se debería dedicar a la creación de los diagramas de interacción (y en paralelo, los diagramas de clases), antes de pasar a la programación. Sí, es cierto, el diseño que se representa en los diagramas será imperfecto y especulativo, y se modificará durante la programación, pero proporcionará un punto de partida serio, consistente y común, que sirve de inspiración durante la programación.

Sugerencia

Cree los diagramas de interacción por parejas, no solo. El diseño colaborando con otro será mejor, y los compañeros aprenderán rápidamente los unos de los otros.

Nótese que es principalmente durante esta etapa donde se requiere la aplicación de las técnicas de diseño, en términos de patrones, estilos y principios. La creación de los casos de uso, modelos del dominio, y otros artefactos, es más sencilla que la asignación de responsabilidades y la creación de diagramas de interacción bien diseñados. Esto es debido a que existen un gran número de principios de diseño sutiles y “grados de libertad” que subyacen a un diagrama de interacción bien diseñado, que a la mayoría de los otros artefactos de A/DOO.

La realización de los diagramas de interacción (en otras palabras, decidir sobre los detalles del diseño de objetos) es una etapa muy creativa del A/DOO. Se pueden aplicar patrones codificados, principios y estilos para mejorar la calidad de sus diseños.

Los principios de diseño que se necesitan para la construcción con éxito de los diagramas de interacción *pueden* codificarse, explicarse y aplicarse de forma sistemática. Este enfoque para entender y utilizar los principios de diseño se basa en los **patrones** —guías y principios estructurados—. Por tanto, después de introducir la sintaxis de los diagramas de interacción, volveremos la atención (en los siguientes capítulos) hacia los patrones de diseño y su aplicación a los diagramas de interacción.

15.5. Notación general de los diagramas de interacción

Representación de clases e instancias

UML ha adoptado un enfoque simple y consistente para representar las **instancias** frente a los clasificadores (ver Figura 15.5):

- Para cualquier tipo de elemento UML (clase, actor,...), una instancia utiliza el mismo símbolo gráfico que el tipo, pero con la cadena de texto que lo designa subrayada.

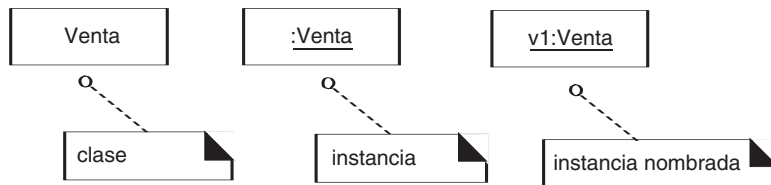


Figura 15.5. Clases e instancias.

En consecuencia, para mostrar una instancia de una clase en un diagrama de interacción, se utiliza el símbolo gráfico para una clase, esto es el rectángulo, pero con el nombre subrayado.

Se puede utilizar un nombre para identificar de manera única la instancia. Si no se utiliza, obsérvese que los “:” preceden al nombre de la clase.

Sintaxis de la expresión de mensaje básica

UML cuenta con una sintaxis básica para las expresiones de los mensajes:

```
return := mensaje(parametro: tipoParametro): tipoRetorno
```

Podría excluirse la información del tipo si es obvia o no es importante. Por ejemplo:

```
espec := getEspecProducto(id)  
espec := getEspecProducto(id:ArticuloID)  
espec := getEspecProducto(id:ArticuloID):EspecificacionDelProducto
```

15.6. Notación básica de los diagramas de colaboración

Enlaces

Un **enlace** es un camino de conexión entre dos objetos; indica que es posible alguna forma de navegación y visibilidad entre los objetos (ver Figura 15.6). De manera más formal, un enlace es una instancia de una asociación. Por ejemplo, existe un enlace —o camino de navegación— desde un *Registro* a una *Venta*, a lo largo del que pueden fluir los mensajes, como el mensaje *realizarPago*.

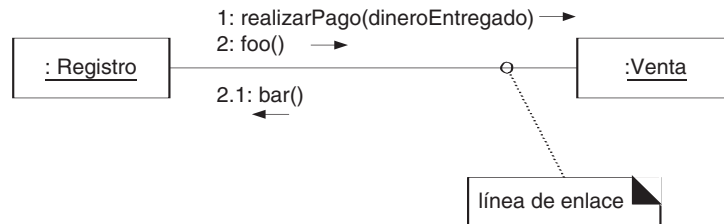


Figura 15.6. Líneas de enlaces.

Obsérvese que, a lo largo del mismo enlace, pueden fluir múltiples mensajes, y mensajes en ambas direcciones.

Mensajes

Cada mensaje entre objetos se representa con una expresión de mensaje y una pequeña flecha que indica la dirección del mensaje. Podrían fluir muchos mensajes a lo largo de este enlace (Figura 15.7). Se añade un número de secuencia para mostrar el orden secuencial de los mensajes en el hilo de control actual.

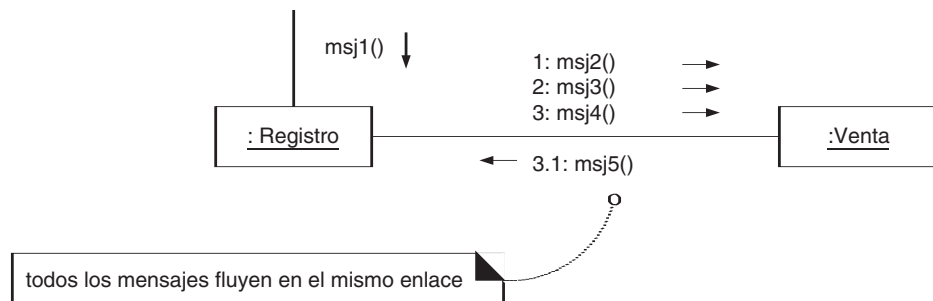


Figura 15.7. Mensajes.

Mensajes a “self” o “this”

Se puede enviar un mensaje desde un objeto a él mismo (Figura 15.8). Esto se representa mediante un enlace a él mismo, con mensajes que fluyen a lo largo del enlace.

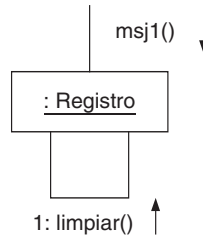


Figura 15.8. Mensajes a “this”.

Creación de instancias

Cualquier mensaje se puede utilizar para crear una instancia, pero en UML existe el convenio de utilizar para este fin el mensaje denominado *create*. Si se utiliza otro nombre de mensaje (quizás menos obvio), se podría añadir al mensaje una característica especial llamada estereotipo UML, como «*create*».

El mensaje *create* podría incluir parámetros, que indican el paso de valores iniciales. Esto indica, por ejemplo, la llamada a un constructor con parámetros en Java.

Además, podría añadirse opcionalmente la propiedad UML {*new*} a la caja de la instancia para resaltar la creación.

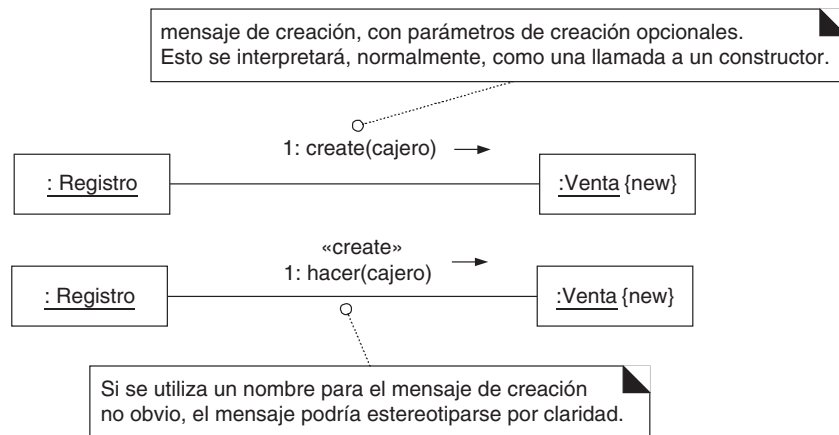


Figura 15.9. Creación de instancias.

Secuencia de números de mensaje

El orden de los mensajes se representa mediante **números de secuencia**, como se muestra en la Figura 15.10. El esquema de numeración es:

1. No se numera el primer mensaje. Por tanto, el *msj1()* no se numera.
2. El orden y anidamiento de los siguientes mensajes se muestran con el esquema de numeración válido en el que los mensajes anidados tienen un número adjunto. El anidamiento se denota anteponiendo el número del mensaje entrante al número del mensaje saliente.

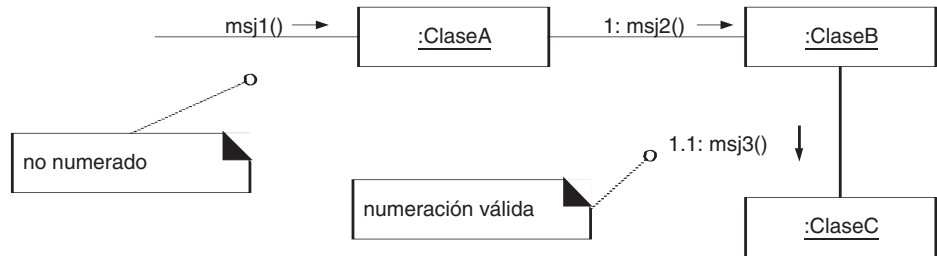


Figura 15.10. Secuencia de numeración.

En la Figura 15.11 se muestra un caso más complejo.

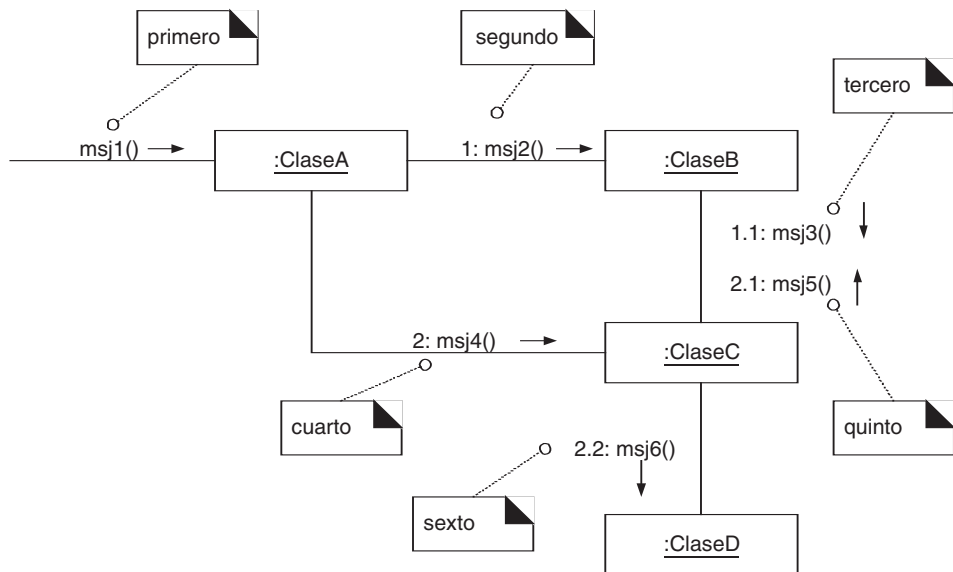


Figura 15.11. Secuencia de numeración compleja.

Mensajes condicionales

Un mensaje condicional (ver Figura 15.12) se muestra con una cláusula condicional, similar a una cláusula de iteración, entre corchetes, a continuación del número de secuencia. El mensaje sólo se envía si la evaluación de la cláusula es *verdad*.

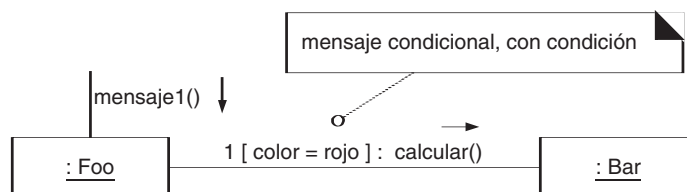


Figura 15.12. Mensaje condicional.

Caminos condicionales mutuamente exclusivos

El ejemplo de la Figura 15.13 ilustra los números de secuencia con caminos condicionales mutuamente exclusivos.

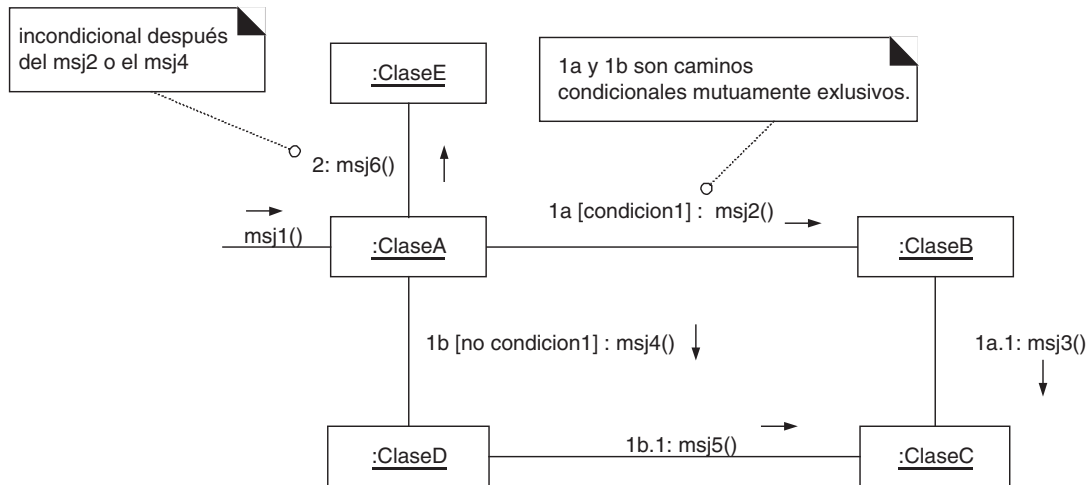


Figura 15.13. Mensajes mutuamente exclusivos.

En este caso es necesario modificar las expresiones de la secuencia con una letra de camino condicional. La primera letra que se utiliza es la **a** por convenio. La Figura 15.13 establece que se podría ejecutar o *1a* o *1b* después de *msj1()*. Ambos tienen el número de secuencia 1 puesto que cualquiera de ellos podría ser el primer mensaje interno.

Nótese que los siguientes mensajes anidados todavía se anteponen de manera consistente con su secuencia de mensaje exterior. De este modo *1b.1* es el mensaje anidado de *1b*.

Iteración o bucle

La notación de las iteraciones se muestra en la Figura 15.14. Si los detalles de la cláusula de iteración no son importantes para el modelador, se puede utilizar simplemente un “*”.

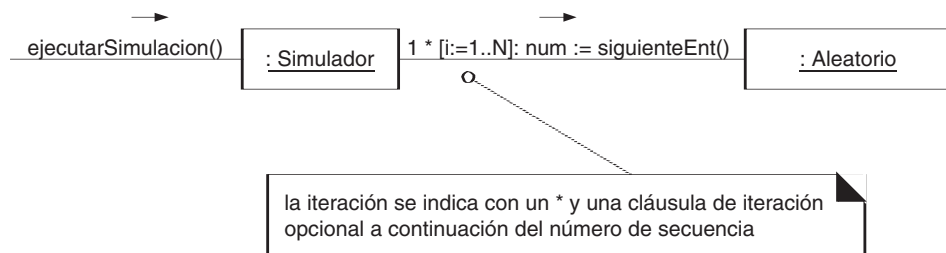


Figura 15.14. Iteración.

Iteración sobre una colección (multiobjeto)

Un algoritmo típico es iterar sobre todos los miembros de una colección (como una lista o una tabla), enviando un mensaje a cada uno de ellos. A menudo, se utiliza, finalmente, algún tipo de objeto iterador, como una implementación de *java.util.Iterator* o el iterador de la librería estándar de C++. En UML, el término **multiobjeto** se utiliza para denotar un conjunto de instancias —una colección—. En los diagramas de colaboración, se puede indicar como se muestra en la Figura 15.15.

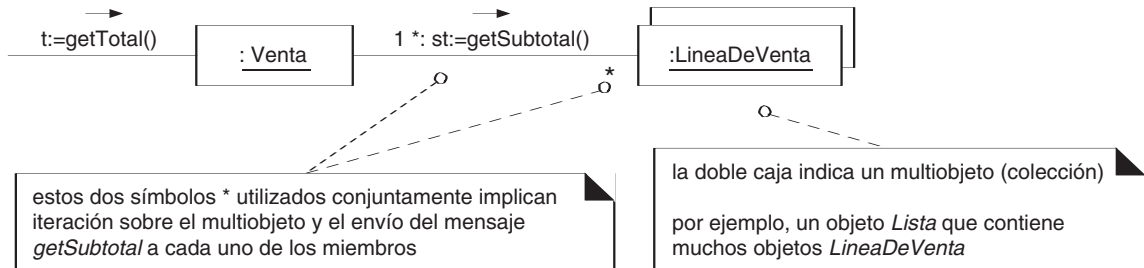


Figura 15.15. Iteración sobre un multiobjeto.

El marcador de multiplicidad “*” al final del enlace, se utiliza para indicar que se va a enviar el mensaje a cada elemento de la colección, en lugar de enviarse repetidamente al propio objeto colección.

Mensaje a un objeto clase

Los mensajes se podrían enviar a las propias clases, en lugar de una instancia, para invocar a los **métodos estáticos** o de clase. Se muestra un mensaje hacia el rectángulo de una clase cuyo nombre no está subrayado, lo que indica que el mensaje se está enviando a una clase en lugar de a una instancia (ver Figura 15.16).

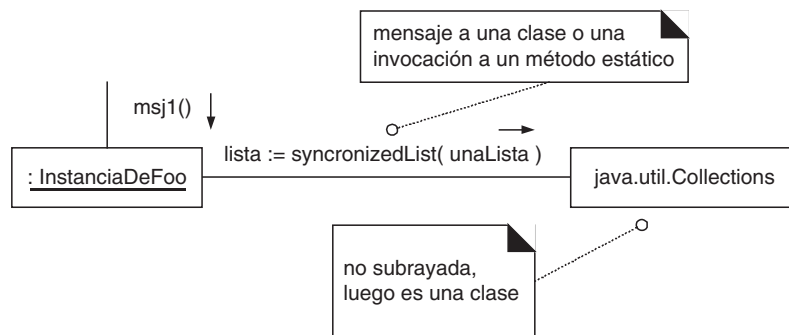


Figura 15.16. Mensaje a un objeto clase (invocación de un método estático).

En consecuencia es importante ser consistente y subrayar los nombre de las instancias cuando lo que se desea es una instancia, de otra manera, se podrían interpretar incorrectamente los mensajes a clases o instancias.

15.7. Notación básica de los diagramas de secuencia

Enlaces

A diferencia de los diagramas de colaboración, los diagramas de secuencia no muestran enlaces.

Mensajes

Cada mensaje entre objetos se representa con una expresión de mensaje sobre una línea con punta de flecha entre los objetos (ver Figura 15.17). El orden en el tiempo se organiza de arriba a abajo.

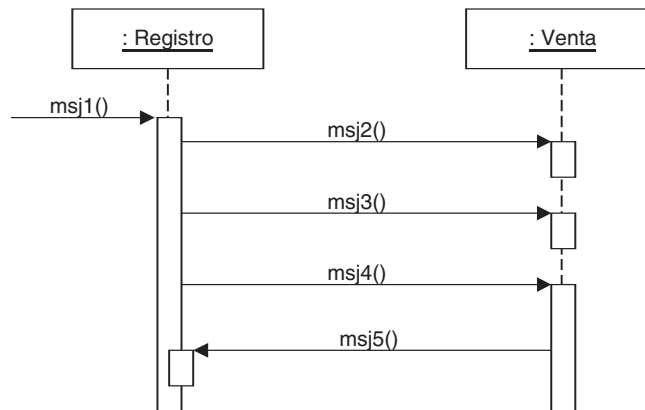


Figura 15.17. Mensajes y focos de control con cajas de activación.

Focos de control y cajas de activación

Como se ilustra en la Figura 15.17, los diagramas de secuencia podrían también mostrar los focos de control (esto es, en una llamada de rutina ordinaria, la operación se encuentra en la pila de llamadas) utilizando una **caja de activación**. La caja es opcional, pero la utilizan habitualmente los modeladores de UML.

Representación de retornos

Un diagrama de secuencia podría opcionalmente mostrar el retorno de un mensaje mediante una línea punteada con la punta de flecha abierta, al final de una caja de activación (ver Figura 15.18). Lo normal es que se excluya por quienes utilizan UML. Algunos anotan la línea de retorno para describir lo que se está devolviendo (si es el caso) a partir del mensaje.

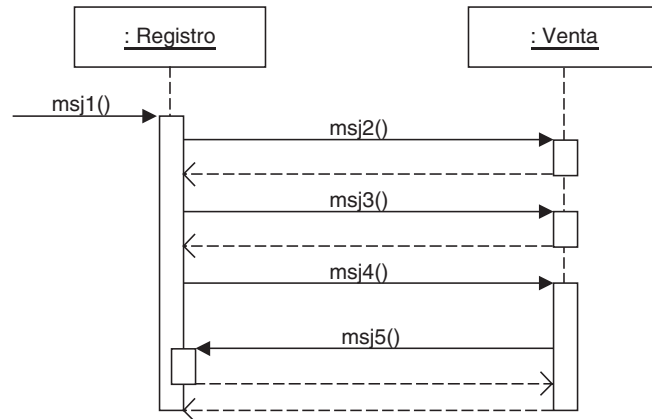


Figura 15.18. Representación de retornos.

Mensajes a “self” o “this”

Se puede representar un mensaje que se envía de un objeto a él mismo utilizando una caja de activación anidada (ver Figura 15.19).

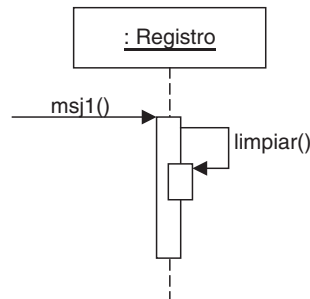


Figura 15.19. Mensajes a “this”.

Creación de instancias

La notación para la creación de instancias se muestra en la Figura 15.20.

Línea de vida del objeto y destrucción de objetos

La Figura 15.20 también ilustra las **líneas de vida de los objetos** —las líneas punteadas verticales bajo los objetos—. Éstas indican la duración de la vida de los objetos en el diagrama. En algunas circunstancias es deseable mostrar la destrucción explícita de un objeto (como en C++, que no tiene recogida de basura); la notación UML para las líneas de vida proporcionan una forma para expresar esta destrucción (ver Figura 15.21).

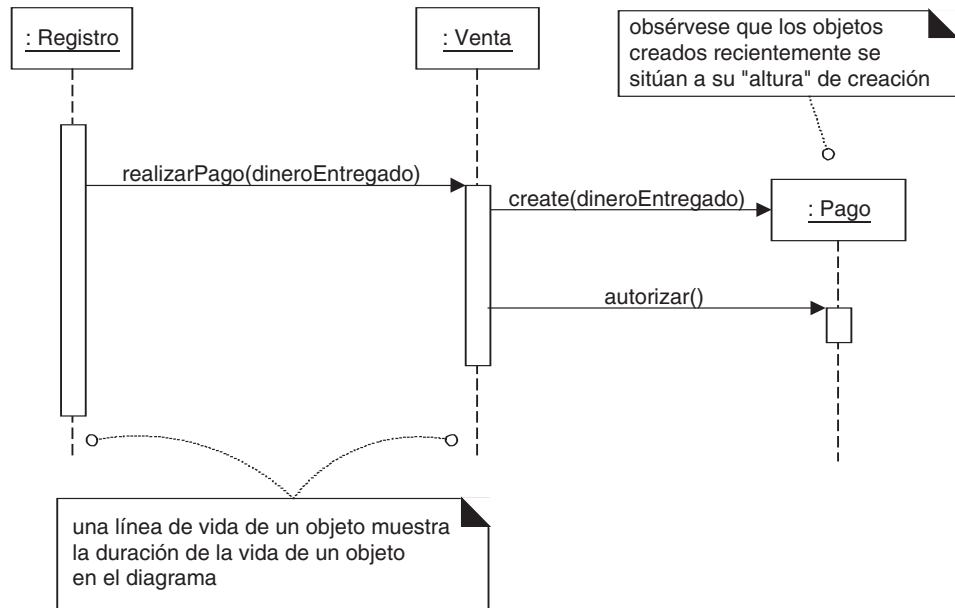


Figura 15.20. Creación de instancias y línea de vida de los objetos.

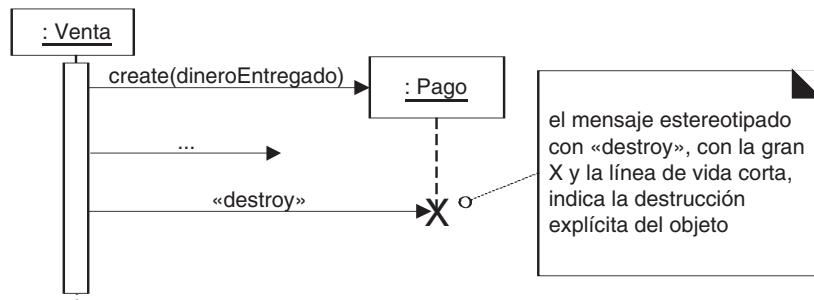


Figura 15.21. Destrucción de objetos.

Mensajes condicionales

La Figura 15.22 muestra un mensaje condicional.

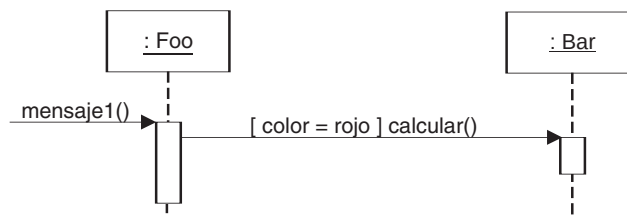


Figura 15.22. Un mensaje condicional.

Mensajes condicionales mutuamente exclusivos

La notación para este caso es un tipo de línea de mensaje con forma de ángulo que nace desde un mismo punto, como se ilustra en la Figura 15.23.

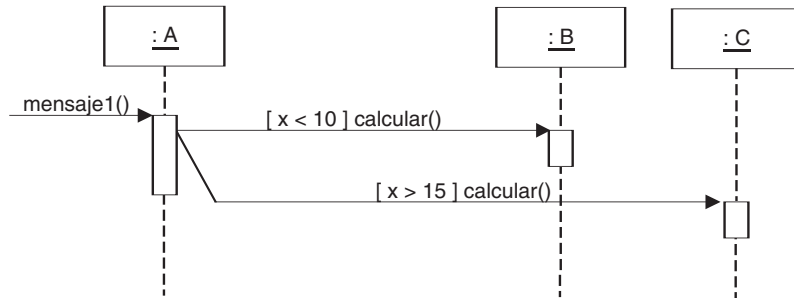


Figura 15.23. Mensajes condicionales mutuamente exclusivos.

Iteración para un único mensaje

La notación para la iteración de un mensaje se muestra en la Figura 15.24.

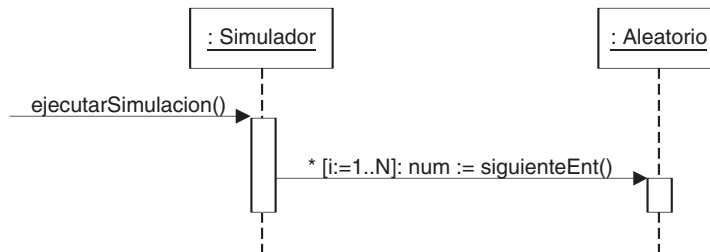


Figura 15.24. Iteración para un mensaje.

Iteración de una serie de mensajes

La Figura 15.25 muestra la notación para indicar la iteración alrededor de una serie de mensajes.

Iteración sobre una colección (multiobjeto)

En un diagrama de secuencia, la iteración sobre una colección se muestra en la Figura 15.26.

Con los diagramas de colaboración de UML, se especifica un marcador de multiplicidad “*” al final del rol (al lado del multiobjeto) para indicar el envío de un mensaje a cada elemento, en lugar de repetidamente a la propia colección. Sin embargo, UML no especifica cómo hacer esto con los diagramas de secuencia.

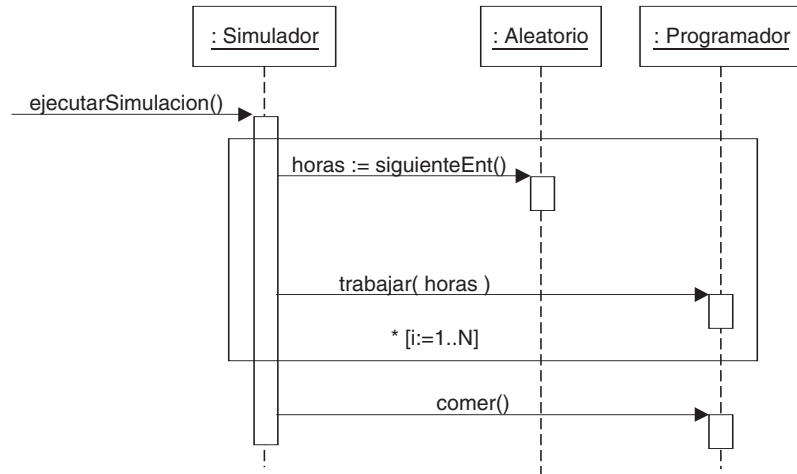


Figura 15.25. Iteración sobre una secuencia de mensajes.

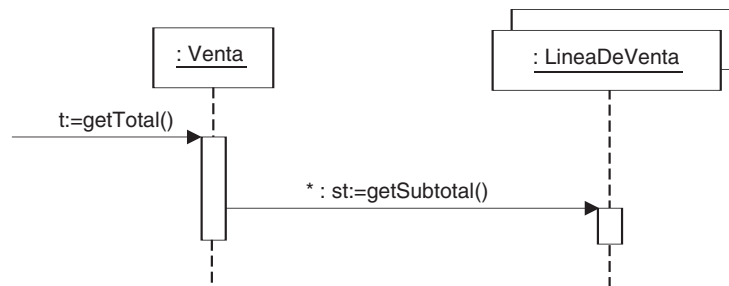


Figura 15.26. Iteración sobre un multiobjeto.

Mensajes a objetos de clase

Como en los diagramas de colaboración, las llamadas a los métodos de clase o estáticos se representan no subrayando el nombre del clasificador, lo que significa que se trata de una clase en lugar de una instancia (ver Figura 15.27).

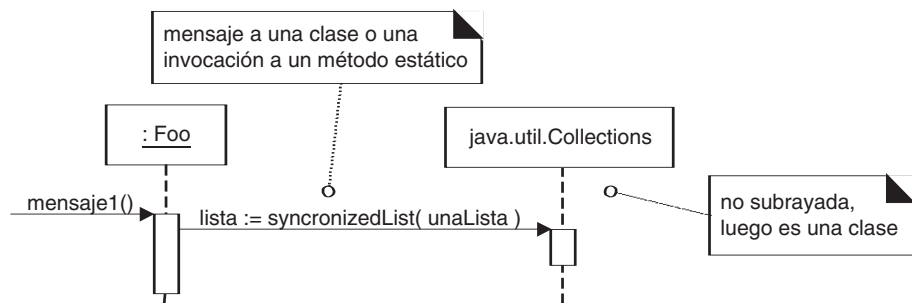


Figura 15.27. Invocación a un método de clase o estático.